

Sicurezza delle Reti

INTRODUZIONE

"Computer Security è la sicurezza offerta da un sistema informatizzato con lo scopo di raggiungere e mantenere l'**integrità**, la **disponibilità** e la **confidenza** delle risorse del sistema"

NIST (*National Institute of Standard and Technology*)

per *risorse* si intende :

- *Hardware / Software*
- *Firmware*
- *Information / Data / Telecommunications*

integrity (integrità), *availability* (disponibilità) e *confidentiality* (confidenza) sono anche chiamate le *CIA triad* e costituiscono il cuore della *computer security* :

- *confidentiality* : *preserva* le restrizioni sull'accesso a informazioni e alla loro divulgazione. Protegge la Privacy e le informazioni *private*. La *perdita* di *confidentiality* è la *divulgazione non autorizzata* di informazioni
 - *Data confidentiality* : le informazioni *private* o *confidenziali* non sono *accessibili* o *divulgate* a utenti *non autorizzati*. Un esempio di *informazione privata* sono le informazioni *protette* da password.
 - *Privacy* : *assicura quali informazioni* di un utente possono essere *raccolte e memorizzate*, chi può *divulgarle* e a chi. Un esempio di *Privacy* sono i risultati degli esami di uno studente.
- *integrity* : *protegge* le informazioni da *modifiche e eliminazioni* improprie. Si *assicura* che le informazioni siano *autentiche* e *non ripudiabili*. La *perdita* di *integrity* è la *modifica non autorizzata* di informazioni
 - *Data integrity* : *assicura* che le *informazioni* e i *programmi* sono *modificabili* solo in un *modo specifico e autorizzato*.
 - *System integrity* : *assicura* che il *sistema* esegua le *proprie funzioni* in maniera corretta, *libero da manipolazioni* accidentali o intenzionali da parte di utenti *non autorizzati*
- *availability* : *assicura* un accesso alle informazioni *tempestivo e affidabile*. La *perdita* di *availability* è l'*impossibilità* di *utilizzo e accesso* alle informazioni del sistema

sebbene l'utilizzo delle *CIA triad* per *definire* gli *obiettivi* della sicurezza sia una *pratica consolidata*, per *completare il quadro* è *necessario* *definire* altri *concetti*, tra i quali :

- *Authenticity* : il sistema deve essere *genuino* e *assicurare* di poter essere *verificato* e reputato *affidabile*. Questo significa poter *verificare* che *l'utente* sia chi *effettivamente dice di essere*, e che ogni *informazione* che arriva al sistema *provenga* da una *fonte affidabile*
- *Accountability* : *consente* di *tracciare in modo univoco* le azioni eseguite da un entità. Dobbiamo essere in grado di *tracciare* tutte le *violazioni di sicurezza* in modo da permetterne la *valutazione* da parte dell'*analisi forense* in caso di *dispute legali*

esistono tre *livelli di impatto* causati dalla *violazione della sicurezza*

- Low : gli effetti *sono limitati*
 - *degrado* delle prestazioni offerte dall'*organizzazione*, la quale riesce comunque a svolgere le sue *funzioni primarie* riducendone *notevolmente* l'efficacia
- Moderate: gli effetti *sono più seri*
 - *infortunio* di un membro del *personale*, il quale però *non comporta lesioni* o *pericolo di vita*
- High: gli effetti sono *catastrofici*
 - l'*organizzazione non è in grado* di svolgere le proprie *funzioni primarie* per un lungo periodo di tempo
 - si *infortuna* un memro del *personale*, il quale *rischia la vita* o *lesioni permanenti*

esempi di applicazione dei *livelli di impatto* sulle *CIA triad* :

- *Confident*
 - i risultati *accademici* sono considerati sotto questo aspetto di *livello High* e quindi questa informazione è *accessibile* solo allo studente stesso, i suoi parenti a gli *impiegati* che necessitano di questa informazione per svolgere il proprio lavoro
 - le *iscrizioni* all'università sono considerate di *livello Moderate* la cui *divulgazione* sarebbe *meno dannosa* rispetto a quella dei *risultati accademici*. Le *informazioni* sulle iscrizioni sono *accessibili* a molte persone, e sicuramente sono *meno appetibili* dal punto di vista delle violazioni *rispetto ai voti*.
 - La *lista degli studenti* , delle *facoltà* o dei *dipartimenti* è considerata di *livello Low* . Le informazioni sono *liberamente disponibili* o pubblicate sul *sito web* della scuola
- *Integrity*
 - le *informazioni* sulle *allergie* dei pazienti sotto questo aspetto vengono considerate di *livello High* in quanto un'informazione *inaccurata* potrebbe comportare *serie lesioni* oppure *la morte* di un paziente esponendo l'ospedale a una *grave responsabilità*
 - il *sito web* che offre un *forum* dove gli *utenti registrati* possono discutere di argomenti *specifici* è considerato di *livello moderate* in quanto se il sito *non è utilizzato* per cose importanti come *la ricerca*, una sua violazione che lo rende *inaccessibile* comporterebbe solo una *lieve perdita* di introiti pubblicitari e il *danno potenziale* sarebbe *molto limitato*
 - i *sondaggi anonimi* invece sono di *livello low* in quanto si *da per scontata* la loro *natura non scientifica* e *innacuratezza*
- *Availability*
 - tutte le *componenti critiche* del servizio richiedono un *livello High*. L'interruzione di questi servizi *impedirebbe* ai clienti di *accedere alle risorse* allo staff di *eseguire le operazioni critiche*. La perdita di *availability* si tradurrebbe in un *ingente perdita finanziaria*, *produttiva* ed eventualmente in una *perdita di clientela*
 - il *sito pubblico* universitario invece è considerato di *livello Moderate* perchè *non contiene informazioni critiche* per l'università, ma la sua *inaccessibilità* potrebbe creare un po' di *disagio* tra gli utenti
 - l'*elenco telefonico* online è considerato di *livello Low* in quanto anche se *l'interruzione del servizio* potrebbe *infastidire*, esistono *molti altri modi* per recuperare un numero telefonico come ad esempio la *copia cartacea* dell'elenco.

La *sicurezza* è un campo affascinante ma *complesso*. Le *sfide* che deve affrontare chi si occupa di *sicurezza* sono molteplici, tra le quali :

- *complessità dei requisiti* : sebbene i *requisiti* siano *identificabili* con semplici *parole chiave* (*confidentially, authentication, non repudiation, integrity*) i *meccanismi* da usare per *soddisfarli* possono essere *molto complessi* e per *capirli* a volte è necessario fare *ragionamenti sottili* per questo la *sicurezza non è così semplice* come potrebbe *apparire* a un novizio
- *attacchi potenziali* : nello *sviluppo* di un particolare *meccanismo* o *algoritmo* di *sicurezza* si deve sempre *considerare* quali possono essere *gli attacchi potenziali*. In molti casi il *successo di un attacco* deriva dal fatto che il *problema* viene visto *sotto un differente punto di vista* e questo permette *all'attaccante* di *sfruttare una debolezza* del sistema che non era *stata considerata* o comunque *inaspettata*
- *procedure controintuitive* : a causa dei *potenziali attacchi* spesso le *procedure* adottate per *fornire* un determinato *servizio* sono *controintuitive*. Dato un *requisito* non è *evidente* che per *soddisfarlo* siano *necessari meccanismi* di *sicurezza così complessi* che richiedono *misure particolarmente elaborate* , infatti per *elaborare meccanismo di sicurezza* sensato è *necessario considerare tutti gli aspetti* della *minaccia dalla quale* ci si vuole *proteggere*
- *sicurezza a livello logico e fisico* : una volta *progettati* i *meccanismi* bisogna *decidere dove implementarli* sia in senso *fisico* che *logico*
 - *logico* : ad esempio *sopra uno strato* del protocollo TCP/IP
 - *fisico* : ad esempio in *punto preciso* della *rete*
- *algoritmi e protocolli coinvolti* : i *meccanismi* di *sicurezza* spesso *coinvolgono più algoritmi e protocolli*. Questo *pone l'accento* sul *problema della creazione, distribuzione e protezione delle informazioni segrete*, ad esempio le *password*. Inoltre i *protocolli di comunicazione* , data la loro *resilienza* , potrebbero *complicare lo sviluppo* del *meccanismo* stesso.
 - Esempio :un *protocollo di comunicazione* o *rete* introduce delle *variabili* o dei *ritardi imprevedibili* . Questo *potrebbe complicare* il *meccanismo di sicurezza* che prevede invece dei *limiti di tempo* relativi alla *durata di transito* di un *messaggio* dal *mittente* al *destinatario*
- *difesa da attacchi* : la *sicurezza informatica* è sostanzialmente una *battaglia* tra chi cerca di *penetrare il sistema* cercando delle *falle* e il *progettista* o *amministratore* che cerca di *chiuderle*.
 - *All'attaccante* basta *trovare una debolezza* nel sistema per *violarlo*. Inoltre *l'attaccante* può *sfruttare una combinazione di debolezze* per *raggiungere i suoi scopi* (es *ottenere i privilegi di amministratore*)
 - il *progettista* o *amministratore* invece deve *trovare ed eliminare tutte le debolezze* del sistema per *raggiungere un ipotetica sicurezza totale*
- *benefici non visibili* : la *sicurezza* viene spesso *sottovalutata* dagli utenti, i quali *tendenzialmente* investono poco in questo senso, in quanto *finché non vengono bucati* non riescono a *percepirne i benefici*
- *la sicurezza è un processo* : la *sicurezza non è un prodotto* ma un *processo* che richiede una *costante e regolare attività di monitoraggio*
- *la sicurezza per ultima* : si pensa alla *sicurezza dopo che il sistema è stato progettato* (e spesso *implementato*) invece che renderla *parte integrante* della *progettazione* stessa
- *meno efficienza e semplicità* : la *sicurezza implica una diminuzione dell'efficienza* e della *semplicità di utilizzo* del sistema e per questo motivo *molti decidono di trascurarla*

la *Request for Comments* n. 2828 definisce i principali termini usati in ambito sicurezza :

- *Adversary (threat agent)* : è l'avversario o l'agente minaccioso, ossia un'entità che attacca o minaccia il sistema
- *Attack* : è l'assalto a un sistema di sicurezza da parte di una minaccia intelligente. Si tratta di un atto deliberato con lo scopo di eludere i servizi di sicurezza e violare il sistema oppure le sue policy (regole)
- *Countermeasure* : è un'azione, dispositivo, procedura o tecnica che riduce la minaccia, la vulnerabilità o l'attacco eliminandolo o prevenendolo. Lo scopo è quello di minimizzare il danno causato oppure scoprire e riportare eventuali azioni correttive
- *Risk* : è un'aspettativa di perdita espressa come la probabilità che una particolare minaccia possa sfruttare una particolare vulnerabilità provocando dei danni
- *Security Policy* : è un insieme di regole e consuetudini che regolano come il sistema o l'organizzazione deve fornire un servizio di sicurezza in grado di proteggere le risorse del sistema sensibili e critiche
- *System Resource (Asset)* : sono le risorse del sistema intese come :
 - Dati memorizzati
 - Servizi forniti
 - capacità del sistema, intesa come capacità di calcolo o larghezza di banda
 - Hardware, Software, Firmware
 - operazioni e attrezzature
- *Threat (minaccia)* : è una potenziale violazione della sicurezza dovuta alla presenza di circostanze, capacità, azioni o eventi che possono bucare la sicurezza e causare un danno. Una minaccia è un possibile danno causato dallo sfruttamento di una vulnerabilità
- *Vulnerability* : è un difetto o una debolezza presente nel progetto, implementazione, operazione o gestione del sistema che può essere sfruttata per violare le policy di sicurezza

la matrice del rischio è una matrice che :

- asse ascisse : esprime la probabilità che un evento accada
- asse ordinate : esprime il danno causato dall'evento nel caso si verificasse

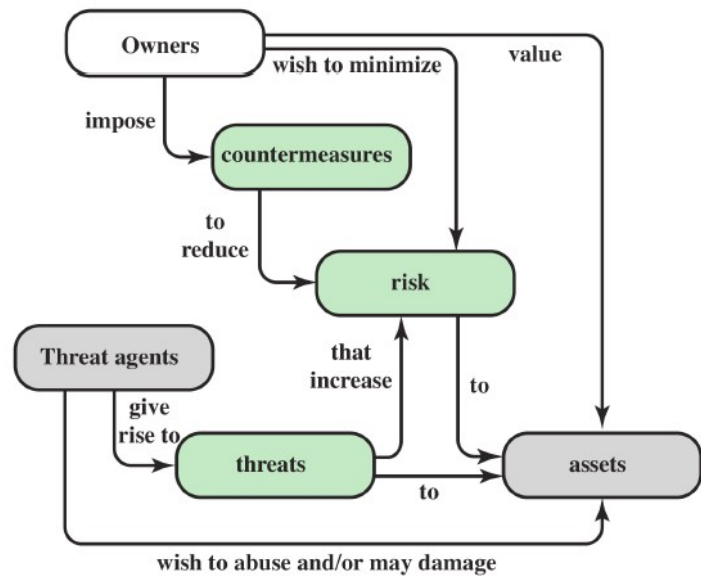
la tabella viene utilizzata per abbassare sia la probabilità che un evento accada, sia per ridurre l'impatto che il suo verificarsi avrebbe sul sistema rendendolo il più leggero possibile. Tuttavia la matrice è legata a tutta una serie di incertezze, di se e ma.

Impact	Likelihood				
	Rare	Unlikely	Possible	Likely	Almost certain
Catastrophic	moderate	moderate	high	critical	critical
Major	low	moderate	moderate	high	critical
Moderate	low	moderate	moderate	moderate	high
Minor	very low	low	moderate	moderate	moderate
Insignificant	very low	very low	low	low	moderate

**RISK
MATRIX**

il *proprietario* inteso come l'amministratore deve valutare l'importanza di una *risorsa* in modo da poter *stabilire* il rischio che essa venga attaccata e decidere quali *contromisure* adottare per minimizzarlo.

L'agente minaccioso sceglie quale risorsa desidera danneggiare e crea una minaccia apposta che aumenta il rischio di un possibile attacco a quella risorsa



nel contesto di *sicurezza*, la preoccupazione *principale* è la *vulnerabilità* del sistema o delle risorse di rete. Esistono *tre categorie generali* di *vulnerabilità* che causano perdita di *integrità*, *confidenza* e *disponibilità*:

- *corrupted* : un sistema/risorsa corrotto ha perso la *proprietà* di *integrità* e quindi da *risposte sbagliate* o ha un *comportamento sbagliato*. Un esempio tipico è la *modifica non autorizzata* dei dati memorizzati sul sistema i quali quindi avranno *valori che differiscono* da quelli che in realtà dovrebbero avere dando origine a *informazioni errate*
- *leaky* : un sistema che *cola* ha perso la *proprietà* di *confidenza* e quindi *persone non autorizzate* riescono ad *accedere a tutte o alcune* informazioni
- *unavailable* : un sistema *indisponibile* ha perso la *proprietà* di *disponibilità* e quindi il suo uso diventa *impossibile* oppure *impraticabile* a causa ad esempio di un *notevole rallentamento* o *diminuzione* delle prestazioni

per ogni tipo di *vulnerabilità* delle risorse del sistema esiste una *minaccia* in grado di *sfruttarla*. Una *minaccia* rappresenta un *possibile danneggiamento* di una risorsa, mentre un *attacco* è la *messa in atto della minaccia* e se ha *successo* porta a una *violazione della sicurezza* o ad *ulteriori minacce*. Colui che *mette in atto* una minaccia è chiamato *attaccante* o *agente minaccioso* :

- *attacco attivo* : tentativo di *alterare* le risorse del sistema o *influenzare* il suo *comportamento* (es. *Man in the middle*)
- *attacco passivo* : tentativo di *raccogliere o fare uso* di informazioni del sistema senza colpirlo (es. *Sniffing*)
- *attacco interno* : è un attacco svolto da una persona *autorizzata all'accesso* delle risorse del sistema, quindi che sta *all'interno del perimetro di sicurezza*, che le *utilizza in modo illecito, non approvato* da chi gli ha concesso tale autorizzazione
- *attacco esterno* : è un attacco svolto da chi sta *all'esterno del perimetro di sicurezza* e quindi *non è autorizzato* oppure è un *utente illegittimo* che accede al sistema. Possono essere degli *amatori delle organizzazioni criminali, terroristi o governi ostili*.

Una *contromisura* è qualunque *tipo di azione* intrapresa per *affrontare un attacco* alla sicurezza. Le *contromisure* possono essere divise in :

- *preventive* : sono *contromisure* adottate per *prevenire* un particolare *tipo di attacco* evitando che questo *faccia successo*
- *individuazione e recupero* (detect and recover) : in caso di *fallimento* delle *contromisure preventive* o quando non è stato *possibile* prevenire un attacco (es. Attacco *sconosciuto*) l'obiettivo diventa quello di *individuare* l'attacco e di *riportare* il *sistema* allo *stato precedente* al manifestarsi degli *effetti* dell'attacco.

Una *contromisura* può introdurre *nuove vulnerabilità* nel sistema, in ogni caso dopo aver *applicato una contromisura* una vulnerabilità non viene *completamente eliminata* ma rimangono *dei residui* : Queste vulnerabilità possono essere *sfruttate* da altri *attaccanti* e questo rappresenta il *livello di rischio residuo* della risorsa. Chi si *occupa di sicurezza* deve *cercare di minimizzare* questo rischio applicando *ulteriori vincoli*.

La *minaccia alla confidenza* è rappresentata dalla Unauthorized Disclosure (*rivelazione non autorizzata*) : si tratta di una *circostanza* per cui un'entità *ottiene l'accesso* a delle informazioni per le quali non aveva *l'autorizzazione* . I *tipi di attacco* che causano *Unauthorized Disclosure* sono :

- *exposure* : può essere un attacco *deliberato* in cui un *insider* diffonde *intenzionalmente* informazioni *sensibili* , ad esempio numeri di carte di credito, a un *outsider* oppure può anche trattarsi di un *errore umano/hardware/software* che *rende nota* un'informazione *sensibile* anche a chi *non è autorizzato* ad accedervi, ad esempio l'università *posta accidentalmente online* le *informazioni confidenziali* dei suoi studenti
- *interception* : è uno degli attacchi *più comuni* nell'ambito della *comunicazione*. In una rete *condivisa* ogni dispositivo collegato *riceve una copia* dei pacchetti *destinati* a un altro dispositivo. Un Haker può quindi (ad esempio su Internet) *ottenere l'accesso* al traffico e-mail creando una *possibile situazione* di *accesso non autorizzato* alle informazioni
- *Inference* : un esempio è *l'analisi del traffico dati* in cui l'attaccante è in gradi di *ottenere informazioni* osservando il *percorso del traffico*, ad esempio la *quantità di traffico tra due host*. Un altro esempio è la capacità di utente che ha un *accesso limitato*, ad esempio a un database, di *ottenere informazioni dettagliate* grazie alla *ripetizione di query* il cui *risultato combinato* permette di fare *inferenza*
- *intrusion* : è un attacco che permette a un utente *non autorizzato* di ottenere informazioni *sensibili superando* i sistemi di *controllo e protezione*

la *minaccia all'integrità* rappresentata dalla Deception (inganno) :

- *masquerade* : è un *tentativo* da parte di un utente *non autorizzato* di ottenere *l'accesso* al sistema *fingendo* ("mascherandosi") di essere un utente *autorizzato*
- *falsification* : è un attacco mirato alla *modifica o sostituzioni* di dati *validi* con informazioni *false* all'interno del *database*. Ad esempio uno studente potrebbe *aumentarsi* i propri voti
- *repudiation* : l'utente *nega* di aver *spedito* informazioni o di aver *ricevuto* o di essere in *possesso* di alcuni dati

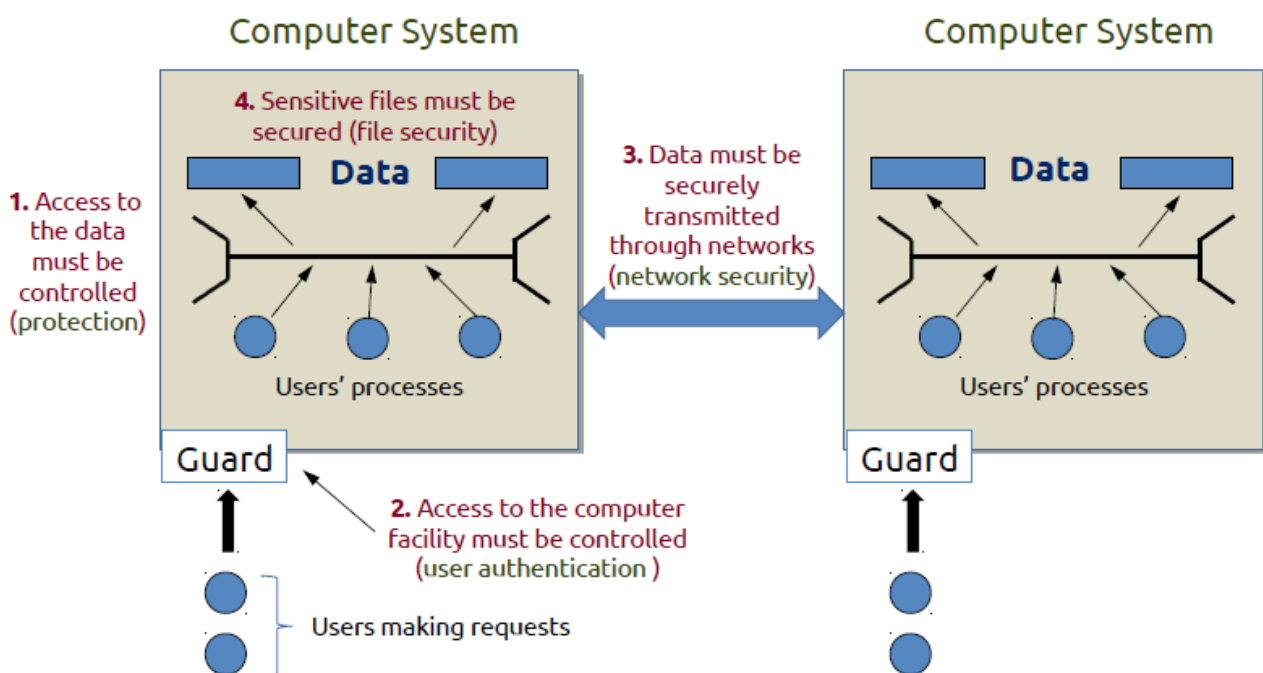
Disruption (rottura) è la minaccia alla disponibilità del sistema :

- *incapacitation* : può essere il risultato della *distruzione fisica* o del *danneggiamento* della componente *hardware*. Più *tipicamente* del *software malevolo* (trojan horse, virus, worm) opera in modo da *disabilitare* il sistema o alcuni dei suoi *servizi*, *pregiudicandone* la *disponibilità*
- *corruption* : in questo contesto invece il *software malevolo* opera in modo da *modificare* il *comportamento* delle *risorse* di sistema o dei *servizi*. Un utente *non autorizzato* quindi riesce ad *accedere* al sistema e *modificare* alcune delle sue *funzioni*, andandone a *intaccare* l'*integrità*. Un esempio è l'*inserimento di backdoor* che consentono di *accedere* al sistema e sfruttare le *risorse* in un modo *non previsto*
- *obstruction* : un modo è quello di *interferire* con la comunicazione *disabilitando* uno dei due *punti* che comunicano, oppure *alterando* le *informazioni* sul controllo della comunicazione. Un'altro modo è quello di *sovraccaricare* il sistema *producendo un eccessivo* traffico di dati o *attivando molti processi*

l'usurpation rappresenta un'altra *minaccia* all'*integrità* del sistema :

- *Misappropriation* : un entità *non autorizzata* assume il *controllo* di risorse *logiche o fisiche* del sistema. Un esempio è l'attacco di tipo *Denial of Service* , dove un *software malevolo* viene installato su *numerosi host* e li usa come *piattaforma di lancio* per "colpire" un *host obiettivo*
- *Misuse* : un entità *non autorizzata* accede al sistema e riesce a fare in modo che un *componente del sistema* *disabiliti* o *danneggi* il sistema di *sicurezza* e quindi *aggirarlo*

questa figura mostra i *punti critici* di un sistema oltre alla sua *sicurezza fisica*. Infatti la *sicurezza* di un sistema non guarda solo la *trasmissione* delle informazioni (*punto 3*) ma anche l'*accesso* ai dati (*punto 1*) , l'*accesso alle macchine* collegate al sistema (*punto 2*) e la *protezione* dei dati sensibili . una *vulnerabilità* in uno di essi comporta il *rischio di fallimento* dell'intero sistema



le *componenti* di un sistema possono essere *suddivise* in 4 *categorie* ognuna delle quali può essere *analizzata* dal punto di vista delle *minacce* all' *integrità*, *confidenzialità* e *disponibilità*

	Availability	Confidentiality	Integrity
Hardware	Il <i>servizio</i> viene negato a causa del <i>furto</i> o la messa <i>fuori servizio</i> di un dispositivo		Viene <i>installato</i> su un dispositivo un <i>malware</i> durante una <i>vendita online</i>
Software	Un programma viene <i>cancellato</i> o viene <i>negato l'accesso</i> agli utenti	Si effettua una <i>copia non autorizzata</i> del programma	Il <i>funzionamento</i> di un programma viene <i>modificato</i> affinché <i>fallisca</i> durante l'esecuzione oppure <i>esegua azioni diverse</i> da quelle <i>previste</i>
Data	Un <i>file</i> viene <i>cancellato</i> o viene <i>negato l'accesso</i> agli utenti, oppure <i>criptato</i> e <i>reso inaccessibile</i>	Vengono <i>lette informazioni</i> alle quali non si <i>dovrebbe avere accesso</i>	I file esistenti vengono <i>modificati</i> oppure se ne creano di <i>nuovi</i> (magari <i>pericolosi</i>)
Communication Line	Il canale di comunicazione viene <i>reso indisponibile</i> oppure i <i>messaggi</i> che passano attraverso di esso vengono <i>cancellati</i> o <i>eliminati</i>	Vengono <i>letti i messaggi</i> che passano sul canale e ne viene <i>osservato e analizzato il traffico</i>	Vengono introdotti <i>messaggi falsi</i> , oppure viene <i>ritardata</i> la ricezione di quelli <i>originali</i> i quali possono essere <i>scambiati di ordine</i> , <i>modificati</i> o <i>duplicati</i>

- Hardware : la *minaccia principale* per l'*hardware* è quella alla *disponibilità*. I dispositivi *fisici* sono i più *vulnerabili* agli attacchi e i meno *suscettibili* al controllo *automatizzato*. Tali minacce *includono* il danneggiamento intenzionale o *deliberato* come ad esempio il *furto*. Altre *minacce* possono portare a una perdita di *confidenzialità* (furto di CD) o la *manomissione* di un *portatile* durante la *consegna* può andare a *minarne l'integrità*.
- Software : la minaccia *chiave* è l'attacco alla *disponibilità* in quanto i *programmi software* spesso sono *facili* da *cancellare*. Inoltre è possibile *alterarli* o *danneggiarli* per renderli *meno utili*. Un modo per mantenere un *livello alto* di disponibilità è quello di mantenere *copie di back up* e la *versione più aggiornata* del programma. Il *problema più difficile* da affrontare è quello rappresentato da un software le *cui funzioni si comportano diversamente* rispetto al solito (attacco all'*integrità*, causato ad esempio da un *virus*). Infine bisogna considerare il problema della *protezione* del software contro *copie illecite*.
- Data : la *minaccia principale* è quella legata alla *sicurezza* dei dati. La prima *preoccupazione* è relativa alla *disponibilità* del dato, il quale può essere *distrutto* sia *accidentalmente* che *intenzionalmente*. È *evidente* che bisogna evitare la *lettura non autorizzata* dei dati o di interi database. È *meno ovvio* invece che una *minaccia* alla *segretezza* dei dati è rappresentata dalla possibilità di *ottenere informazioni segrete* semplicemente *aggregando informazioni* analizzate. La modifica di un file (minaccia all'*integrità*) può avere *conseguenze* che spaziano da *minime* a *disastrose*

- Communication Lines / Network : gli attacchi alle *linee di comunicazione* o più in generale alla *rete* si possono classificare in :
 - *passivi* : un attacco *passivo* è un tentativo di *apprendere* o *fare uso* di informazioni senza *colpire* le risorse del *sistema* . Si tratta di *intercettare* o *monitorare* le trasmissioni : l'*obiettivo* è quello di *ottenere le informazioni* che vengono trasmesse
 - *attacco al contenuto* dei messaggi : si tratta di *intercettare* una comunicazione e *prendere possesso* delle *informazioni scambiate*.
 - *attacco al traffico (analisi)* : per *evitare* di farsi *intercettare il contenuto* dei messaggi è possibile *crittografarli*. Tuttavia questo non *impedisce* all'attaccante di *monitorare il traffico* e di ottenere informazioni sull'*identità e la posizione* degli host che comunicano, la *frequenza* e la *dimensione* dei messaggi scambiati. Con tutte queste informazioni è possibile *indovinare la natura* della *comunicazione* anche se non è *possibile catturarne direttamente* il contenuto.

Gli *attacchi passivi* sono *difficili da individuare* perché il *traffico dati* scorre *regolarmente* e nessuno tra il *mittente* e il *destinatario* è a conoscenza del fatto che *una terza parte* è in *ascolto* o sta *intercettando* i messaggi, anche perché *nessun dato viene alterato*.

La *sicurezza* quindi si concentrerà *sulla prevenzione* degli attacchi passivi piuttosto che sul *rilevamento*

- *attivi* : un attacco *attivo* è un tentativo di *alterare* le risorse del sistema o *influenzare* il loro funzionamento. Si tratta di *modificare* un flusso di dati o di *crearne uno falso*.
 - *Replay* : è un attacco che ha come *obiettivo* quello ottenere un *effetto non autorizzato*, *ritrasmettendo* dei dati che erano stati *intercettati passivamente* in precedenza
 - *masquerade* : è un attacco che *maschera* un *entità* da un *altra* . Di solito viene *combinato* con un altro tipo di attacco. Ad esempio con un *attacco* di tipo *replay* posso catturare la *sequenza di autenticazione* e ritrasmetterla *dopo che* con un attacco di tipo *masquerade* mi sono *finto* un *utente del sistema*. In questo modo il sistema *permette* a un *utente di scalare i privilegi* fino a ottenere anche quelli di *amministratore*
 - *modification of message* : si tratta di *alterare* una porzione del *messaggio* che viene spedito, oppure di *ritardarlo o scambiare l'ordine* dei pacchetti in modo da produrre un *effetto diverso dal previsto*.
 - *denial of service* : è un attacco che *impedisce* il normale uso del *canale di comunicazione*. Questo tipo di attacco ha un *obiettivo specifico* e ad esempio può *impedire* che un *utente riceva messaggi* oppure può *distruggere* un *intera rete* oppure *renderla inutilizzabile* a causa di un *sovraccarico* dovuto all'*elevato numero* di messaggi ricevuti che ne *peggiora drasticamente* le performance.

Gli *attacchi attivi* sono *difficili da prevenire* in quanto bisognerebbe proteggere *fisicamente* tutte le *linee di comunicazione* e i *punti di accesso 24/24h* .

la *sicurezza* si concentrerà quindi *sull'individuazione e il ripristino* di ogni *malfunzionamento*, in quanto questo *funzionerà anche da deterrente* per gli attacchi futuri

esistono dei *principi da seguire* per la *progettazione* di un *buon sistema di sicurezza* :

- *Economy of Mechanism* : il *progetto* deve essere il *più semplice possibile* in modo da *semplificarne* l'implementazione e la verifica. Inoltre un sistema *semplice* presenta *meno vulnerabilità* rispetto a uno *complesso*
- *Fail-Safe default* : le *decisioni* sull'*accesso* al sistema deve basarsi su *permessi* e non su *esclusioni*
- *Complete Mediation* : ogni *tipo* di *accesso* deve *sempre* essere *verificato* e gestito da un sistema di *controllo accessi*
- *Open Design* : il *progetto* non deve restare *segreto* in quanto non deve essere la *segretezza* il suo *punto chiave* (ad esempio gli *algoritmi di cifratura* sono noti a tutti)
- *Separation of Privilege*: i privilegi *devono* essere *separati* e per accedere a *risorse critiche* o per effettuare *operazioni complesse* deve essere necessario *possederne più di uno*
- *Least Privilege* : ogni utente deve possedere il *privilegio minimo* ossia *lo stretto indispensabile* per poter eseguire i propri compiti
- *Least Common Mechanism* : il sistema deve *minimizzare* il numero di *funzioni/informazioni* condivise tra utenti diversi
- *Psychological Acceptability* : i meccanismi di *sicurezza* non devono *interferire* o essere percepiti come un *ostacolo* al lavoro che un utente deve svolgere
- *Isolation* : non deve essere *possibile* usare connessioni *pubbliche* per accedere a *informazioni riservate*, i file di *ogni utente* devono essere *separati* da quelli degli altri e il sistema di *sicurezza* deve essere isolato rispetto al resto del sistema
- *Encapsulation*: come per *l'object oriented* , la struttura *interna* deve essere *nascosta*
- *Modularity* : lo sviluppo delle funzioni di *sicurezza* deve essere visto come un *modulo* *protetto e separato*. La progettazione e l'implementazione deve quindi *utilizzare un architettura modulare*
- *Layering* : la *protezione* del sistema deve prevedere meccanismi *multipli e sovrapposti* (es. Utilizzo di *più firewall* di *produttori diversi*)
- *Least Astonishment* : il programma o la sua *interfaccia* deve sempre *presentarsi* in modo da *non sconvolgere* l'utente

la *security policy* è una *descrizione informale* di come si vorrebbe che *un sistema* si comportasse. Si tratta di un *insieme* di regole e pratiche che *specificano* o *regolano* come un sistema o un'organizzazione deve fornire un *servizio di sicurezza* in modo da proteggere risorse *critiche* o *sensibili*.

Durante lo sviluppo della *security policy* bisogna considerare i seguenti fattori :

- il *valore* delle risorse *da proteggere*
- le *vulnerabilità* del sistema
- le *possibili minacce* e le *probabilità* di ricevere un attacco

inoltre la *sicurezza* di un sistema richiede di *misurarsi* con due grandi *compromessi* :

- *sicurezza vs facilità d'uso* : i *meccanismi* di *sicurezza* *complicano* l'utilizzo del sistema, ad esempio richiedono di *memorizzare una password*, oppure un *firewall* *riduce la capacità* di *trasmissione* o *allunga i tempi* di risposta, gli *antivirus* possono *ridurre* la *potenza* di calcolo o *introdurre malfunzionamenti* causati dall'interazione errata tra *sistemi di sicurezza* e *sistema operativo*
- *costo della sicurezza vs costo del fallimento e riparazione* : il costo che *si sostiene* per *implementare* e *mantenere* le misure di *sicurezza* deve essere *bilanciato* con il costo che si andrebbe a sostenere per *ripristinare* il sistema in caso queste misure *dovessero fallire*

la *policy security* è quindi questione di *business* e può essere *influenzata* da questioni *legali*.
L'*implementazione* di un sistema di sicurezza coinvolge quattro tipi di azione diversa :

- *Prevention* : in un sistema di sicurezza *ideale* nessun attacco *ha successo*. Nel mondo *reale* invece per *certi attacchi* è molto più semplice adottare un *azione preventiva*
 - ad esempio per *contrastare* un attacco che punta alla *confidenzialità* di una trasmissione è possibile adottare una *misura preventiva* che prevede l'utilizzo di un *algoritmo crittografico* forte e una serie di *tecniche* in grado di *proteggere la chiave crittografica* da *accessi non autorizzati*
- *Detection* : in molti altri casi una *protezione completa* non è possibile, ma si può comunque *individuare* l'attacco.
 - Ad esempio si può *progettare* un sistema in grado di *individuare* la presenza di *utenti non autorizzati collegati* al sistema
- *Response* : una volta *individuato* un attacco, il sistema potrebbe essere in grado di *rispondere* in modo da *bloccarlo* e *prevenire danni futuri*
- *Recovery* : è la possibilità di *ripristinare il sistema* a un momento *precedente l'attacco* ad esempio *ricaricando* copie di *back up* che contengono *dati integri*.

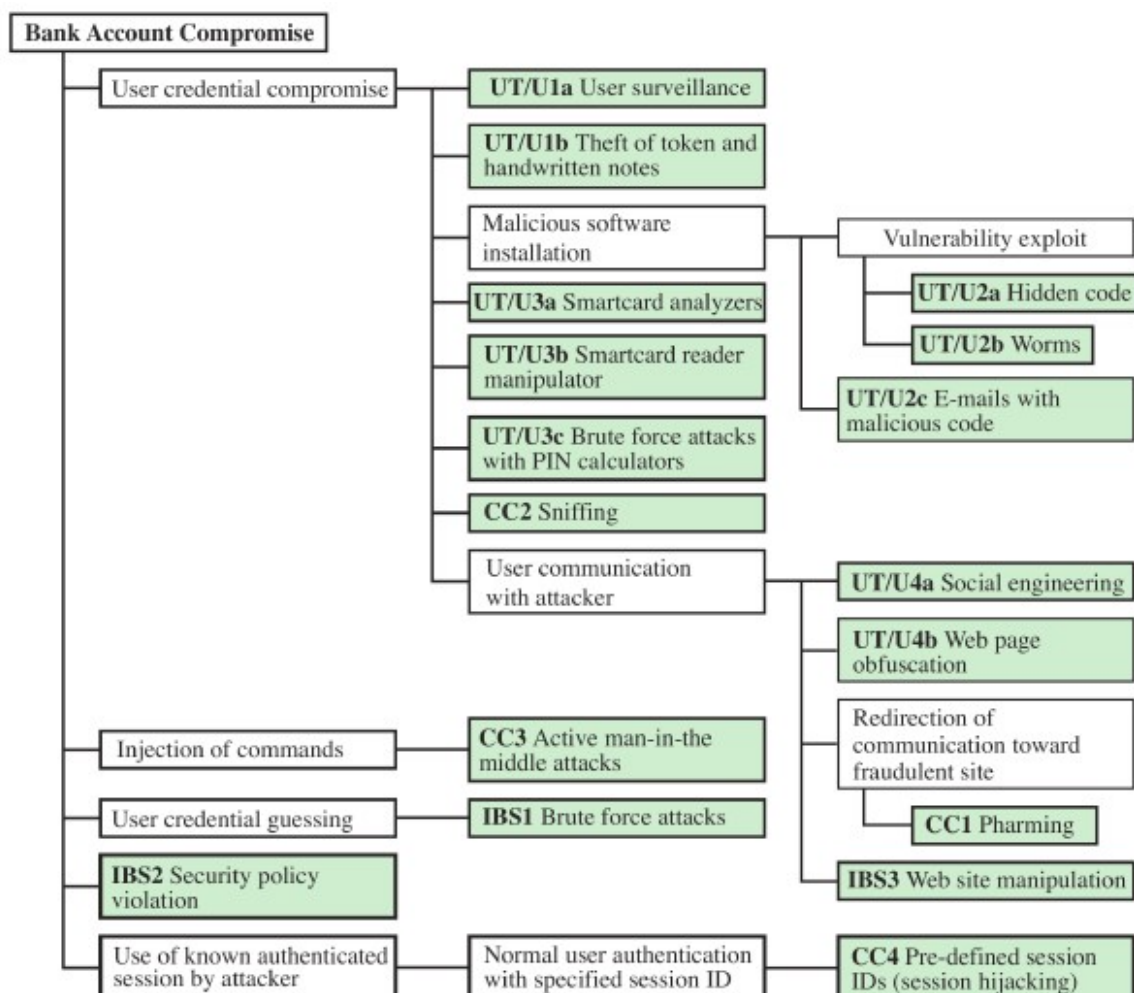
Chi "*acquista*" sicurezza vuole che l'*infrastruttura* del sistema *soddisfi* i requisiti di sicurezza richiesti e *rafforzi* la *security policy*. Queste considerazioni vengono espresse dai *concetti* di *assurance* e *evaluation* :

- *assurance* : è il *grado di confidenza* con cui la *misure di sicurezza* lavorano per *proteggere* il sistema e le informazioni. *L'assurance* risponde alle domande del tipo "il *progetto* del sistema *soddisfa i requisiti?*" , "la sua *implementazione* rispetta le *specifiche?*" *l'assurance* è espressa in *grado di confidenza* , perciò non indica se un *progetto* o un *implementazione* sono *corretti*. Al momento per *tentare* di passare dal *grado di confidenza* a una *prova formale* per stabilirne la *correttezza* , durante lo *sviluppo dei modelli formali* che caratterizzeranno il *progetto* e l'*implementazione* si cerca di *definire i requisiti* utilizzando *tecniche logiche e matematiche*
- *evaluation*: è il *processo* che esamina se un *prodotto* o un *sistema* rispetta certi *criteri*. *Evaluation* comprende delle *fasi di test* e a volte anche delle *analisi* basate su *tecniche formali o matematiche*. Il *punto centrale* è quello di *sviluppare dei criteri* che possano essere *applicati su ogni sistema di sicurezza* in modo da poter *permettere un confronto* tra sistemi diversi

la *superficie* di attacco può essere divisa in tre grandi categorie :

- *network* attack surface : sfrutta le *vulnerabilità* della rete, tra le quali
 - *porte* di rete aperte
 - servizi non protetti da un *firewall*
- *software* attack surface : sfrutta le *vulnerabilità* di un software, ad esempio:
 - nasconde *del codice* all'interno di un *flusso dati* in entrata (e-mail, XML, Flash ..)
 - sfrutta le *interfacce*, SQL e le *form* dei siti web
- *human* attack surface : è la *social engineering*
 - *raggira* i dipendenti che hanno *accesso* a informazioni *sensibili*

l'*analisi* di una *superficie* di attacco deve *valutare* il *grado* e la *gravità* di una minaccia, per questo si utilizza una *struttura dati* gerarchica che *rappresenta* l'insieme delle *possibili vulnerabilità*. L'*obiettivo* è quello di *sfruttare* tutte le *informazioni* disponibili sui *percorsi d'attacco*, ed è usato come *guida* per *progettare* le *contromisure*



CRITTOGRAFIA

"la sicurezza di un crittosistema non deve dipendere dalla segretezza dell'algoritmo usato, ma solo dalla segretezza della chiave"

Kerckhoff. *La cryptographie militaire*, 1883

la *crittografia* viene utilizzata *principalmente* per implementare le seguenti *operazioni* :

- *autenticazione* : consente di *assicurare l'identità* di un utente in un *processo* di comunicazione
- *riservatezza* : consiste nel *proteggere le informazioni* da occhi *indiscreti*
- *integrità* : consente di *certificare l'originalità* di un messaggio o documento, ossia si *certifica* il fatto che *non è stato modificato* in alcun modo
- *anonimato* : consente di *non rendere rintracciabile* una comunicazione

le *operazioni* sono :

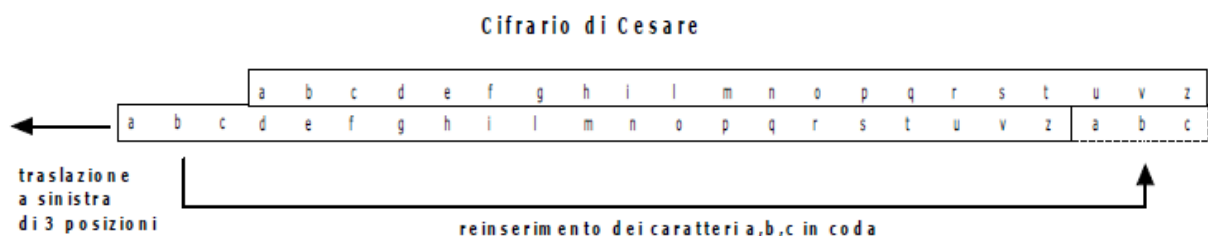
- *cifratura* : un *messaggio in chiaro*, attraverso una *funzione*, viene *trasformato* in un *crittogramma* (messaggio *cifrato*)
 - a *messaggio diverso* corrisponde *crittogramma diverso*
- *decifrazione* : permette, applicando una *funzione* su un *crittogramma*, di *ricavare* il *messaggio in chiaro*

un *cifrario* è un sistema in grado di *trasformare* un *testo in chiaro* in un *testo non intellegibile* (crittogramma)



uno dei *cifrari* più famosi è il *cifrario di cesare* :

- dato un *alfabeto* (es *alfabeto italiano*)
- *sostituisce* ad ogni lettera del messaggio *quella che*, nell'*alfabeto*) *si trova di 3 posizioni avanti* (o *generalizzando di k posizioni*)
- Esempio : "*prova di trasmissione*" viene *trasformato* in "*surbd gn zudvpnvnrqh*"



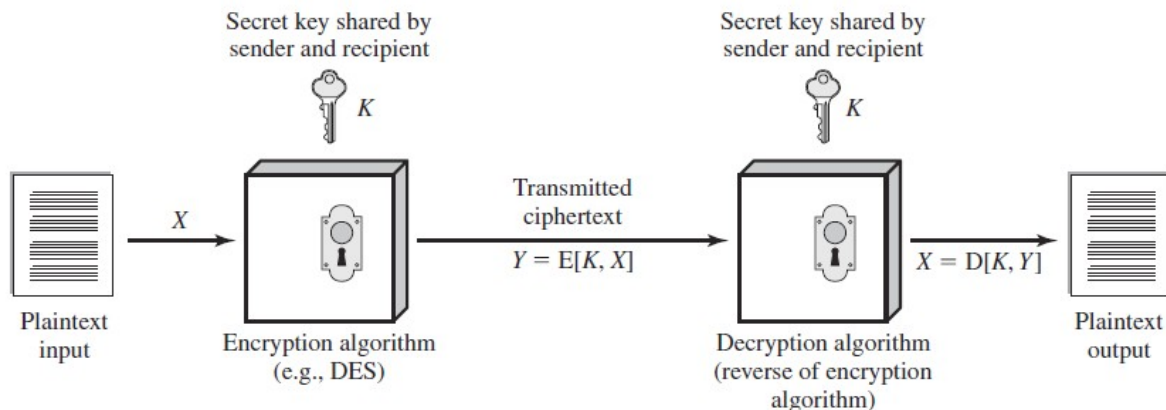
i *cifrari* basati su *trasposizioni* e *traslazioni* sono *facilmente violabili* utilizzando *tecniche statistiche* :

- si *analizzano* le *frequenze relative* dei caratteri presenti nel *crittogramma* e si *confrontano* con le *frequenze relative* di una *lingua conosciuta* (es *italiano*)
- si *procede per tentativi ripetendo l'algoritmo* fino a quando non si ottiene un'*approssimazione soddisfacente*

la *crittografia simmetrica* è la tecnica *universale* utilizzata per garantire la *confidenzialità* dei dati che vengono *trasmessi* oppure *memorizzati* :

- *Data Encryption Standard* (DES, 3DES)
- *Advanced Encryption Standard* (AES)

la *crittografia simmetrica* era l'unica tecnica adottata prima dell'introduzione della *chiave pubblica* ed era utilizzata principalmente per *scopi militari* e ancora oggi è la tecnica *più utilizzata*



- *Plaintext* : è il *testo originale* (o i *dati*) che vengono passati *come input* all'*algoritmo crittografico*
- *Encryption algorithm* : è un *algoritmo* che applica *sostituzioni* e *trasformazioni* al testo o dati *passati in input*
- *Secret Key* : la *chiave segreta* permette di *riapplicare* le *sostituzioni* e *trasformazioni* effettuate e *riottenere* il *messaggio originale*
- *Ciphertext* : è il *testo crittografato* prodotto dall'*Encryption algorithm* . Dipende sia dal *testo originale* che dalla *chiave segreta* : infatti *due chiavi diverse, producono ciphertext diversi*
- *Decryption Algorithm* : utilizzando la *stessa chiave segreta*, applica le *sostituzioni* e *trasformazioni inverse* e restituisce il *messaggio originale*

la *crittografia con chiave simmetrica* per essere *sicura* deve *soddisfare due requisiti* :

1. un *algoritmo forte* : l'*attaccante non deve* essere in grado di *decriptare il ciphertext* o di *trovare la chiave segreta* anche se è in *possesso di alcuni plaintext* e i loro *relativi ciphertext* creati dall'*algoritmo*
2. *scambio sicuro di chiave* : il *mittente* e il *destinatario* devono riuscire a ottenere una *copia della chiave segreta* in *modo sicuro* e la *chiave deve essere protetta* da *attacchi*. Se qualcuno riesce a *ottenere la chiave*, e *conosce l'algoritmo* allora *tutte le comunicazioni* effettuate con *quella chiave* saranno *leggibili*

esistono *due approcci generali* per attaccare la *crittografia con chiave simmetrica* :

- *Cryptanalytic* : fa affidamento sulla *natura dell'algoritmo*, su alcune *caratteristiche generali* del *plaintext* crittografato e su alcuni *esempi di plaintext e relativo ciphertext*. Questo tipo di attacco *sfrutta le caratteristiche* dell'algoritmo e *tenta di dedurre il plaintext* oppure la *chiave segreta* utilizzata. Se l'attaccante riesce a *ottenere la chiave*, sarà in grado di leggere *tutti i messaggi* (passati e futuri) *crittografati* con quella chiave
- *Brute Force* : *tenta tutte le combinazioni di chiave possibili* su un *ciphertext* fino a *ottenere una traduzione intellegibile del plaintext* . In *media* per avere *successo* bastano *la metà dei tentativi*. Il problema più complesso è quello di *riconoscere il plaintext* (se si tratta di un *testo e in che lingua*, se è *codice binario* etc). Chiaramente *più è lunga una chiave* più sarà *difficile* che un *attacco bruteforce* abbia successo (*impiegherebbe troppo tempo*)

l'algoritmo più *comunemente usato* è il Data Encryption Standard e si basa sui *block ciphers* : elabora il *plaintext* in blocchi di *dimensione fissa* e produce dei blocchi di *ciphertext* della *stessa dimensione* . Il DES quindi :

- i *blocchi di plaintext* : *dimensione 64 bit*
- i *blocchi di ciphertext* : *dimensione 64 bit*
- *lunghezza chiave simmetrica* : *56 bit*
 - questa *lunghezza*, invece che *64 bit*, fu suggerita dalla NSA una *dimensione a 48 bit* in quanto *a posteriori* si seppe che a quella *dimensione* era in grado di *scoprire la chiave* in tempi *ragionevoli* grazie a un *attacco brute force* . L'accordo ci fu a metà strada (*56 bit*)

le due *principali preoccupazioni* erano quindi la *lunghezza della chiave* e la *natura dell'algoritmo* in quanto il DES era l'*algoritmo crittografico* più studiato al mondo e nonostante questo nessun attacco di *cryptoanalysis* era riuscito a riportare *debolezze fatali*.

Nel 1998 però la EFF riuscì con un *attacco brute force* a trovare la *chiave* crittografica in *meno di tre giorni* rendendo di fatto il DES *insicuro*.

è importante notare che esistono *molti attacchi* che prevedono tecniche diverse dal *semplice tentativo* con tutte le *possibili chiavi* : senza conoscere il *contenuto* del messaggio, è possibile però *riconoscerlo*. Se un messaggio viene *compressso* prima di venir *decrittato* sarà più difficile *riconoscere* che si tratti di un *testo*, e se il messaggio è composto da tipi di dato *generici*, ad esempio si tratta di un *file numerico*, diventa ancora più complesso riconoscere quel *file numerico* come il vero *testo* . Per questo motivo a *supporto* di un attacco di tipo *Brute Force* è necessario una *conoscenza minima* del messaggio in chiaro (*plaintext*) che ci si aspetta (ad esempio se è un *file numerico*, o se è un *testo*) e una *minima abilità* nel sapere *distinguere automaticamente* un *plaintext* dalla *spazzatura*; Risulta ovvio che un metodo per *contrastare* un attacco *Brute Force* è rappresentato dall'utilizzo di una chiave *abbastanza lunga*

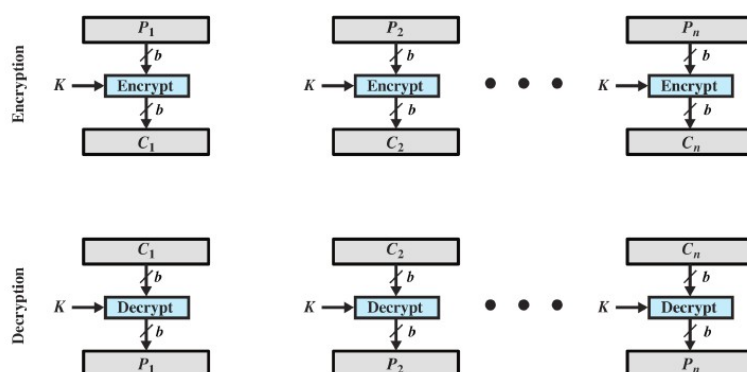
data la sua *insicurezza*, l'algoritmo DES viene *esteso* dal 3DES (*triple DES*) che *riapplica* l'algoritmo DES *tre volte* usando *due o tre chiavi univoche* (112 o 168 *bits*) :

- utilizzando una chiave *più lunga* (168 *bit*) si *eliminano* le vulnerabilità del DES rispetto a un attacco di tipo *Brute Force*
- basandosi sull'algoritmo DES, anche il 3DES risulta *molto resistente* agli attacchi di *cryptoanalysis*

tuttavia riapplicare tre volte DES *rallenta* inevitabilmente le prestazioni e inoltre per questioni di *efficienza e sicurezza* sarebbe desiderabile poter *scegliere* la *dimensione dei blocchi* invece di averne una *fissa* (sia DES che 3DES utilizzano blocchi da 64 *bit*)

nel 1997 il NIST propone l'*Advanced Encryption Standard* (AES) il quale ha lo stesso (se non maggiore) livello di sicurezza del 3DES, ma soprattutto è *molto più efficiente*. AES è un algoritmo *simmetrico* e produce *ciphertext* i cui blocchi hanno *dimensione 128 bits* e supporta *tre dimensioni diverse* per la chiave (128, 192, 256 bits)

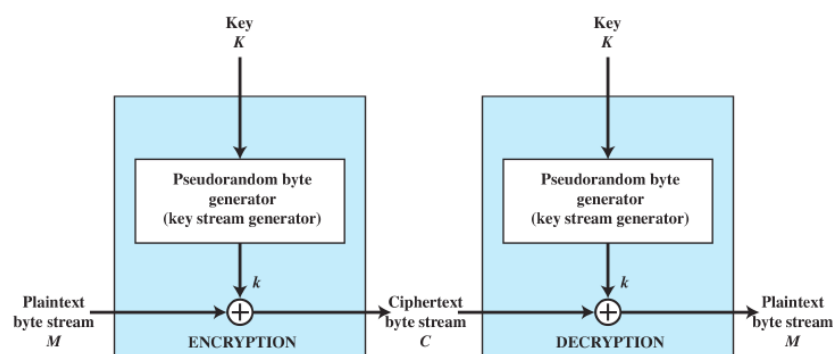
tipicamente la *crittografia simmetrica* avviene *suddividendo* un file in *blocchi* da 64 o 128 bit i quali verranno *crittografati* usando la *stessa chiave* producendo i corrispondenti *ciphertext*. L'approccio *crittografico a multi-blocco* è conosciuto come *Electronic CodeBook*



più un *plaintext* è lungo , più ECB *non è sicuro* : un attacco di *crittoanalisi* potrebbe infatti *sfruttare* le *regolarità* presenti all'interno del *plaintext* (ad esempio sapendo che *tutti i messaggi* cominciano con la stessa frase) per *facilitare* la *decrittatura*.

Una tecnica chiamata *stream cipher* processa gli elementi *in input* in modo *continuo* producendo *uno alla volta* gli elementi di *output* come se si trattasse di un *flusso ininterrotto*.

Uno *stream cipher* processa di solito un byte alla volta (possono essere anche *bit* o altre unità di misura) mentre la *chiave simmetrica* viene data in pasto a un *generatore pseudocasuale* che produce uno *stream* di 8 bit *apparentemente casuale*. Il risultato del *generatore* chiamato *keystream* , che può essere determinato *solo se si conosce* la chiave simmetrica, viene *combinato* un byte alla volta con il *plaintext* usando le operazioni OR e XOR



con un *generatore ben progettato* lo *stream cipher* risulterà essere *più sicuro* del *block cipher* a parità di *lunghezza* della chiave simmetrica . Lo *stream cipher* è *più veloce* e usa *molto meno codice*, i *block cipher* invece possono *riutilizzare* la stessa *chiave simmetrica* :

- *stream cipher* : usato per *crittografare* i canali di comunicazione o i collegamenti web
- *block cipher* : usato per *crittografare* le e-mail, i file trasferiti o database

ad ogni modo ogni applicazioni può essere *crittografata* da *entrambi* i tipi di algoritmo

la *crittografia* viene adottata per *protegersi* dagli attacchi *passivi* (es *intercettazioni*) , tuttavia esistono anche gli attacchi *attivi* (*falsificazioni* di dati o transazioni) e per *protegersi* da essi si utilizza un *meccanismo* di *autenticazione* del dato o del messaggio

- un file è *autentico* quando è *genuino* e *proviene* veramente *dal mittente*

gli aspetti *principali* del processo di *autenticazione* sono la *verifica* che i *contenuti* del messaggio *non siano stati alterati* e che la *sorgente* sia *autentica*. Altri aspetti sono la *verifica della tempistica* (es se un messaggio è stato *ritardato intenzionalmente* oppure *ripetuto più volte*) e la *verifica della corretta sequenza* relativa al *flusso dei messaggi* tra due parti.

È possibile *ottenere autenticazione* in diversi modi

- utilizzando la *crittografia simmetrica* : il *mittente* e *destinatario* *condividono* la *chiave*

solo il *mittente genuino* è in grado di *criptare correttamente* il messaggio in modo che il *destinatario possa riconoscerlo* come *autentico*. Inoltre se il messaggio include un *error-detection code* e un *numero di sequenza* , il *destinatario* potrà *verificare* che il contenuto *non sia stato alterato* e che *l'ordine di arrivo* dei pacchetti *sia quello giusto*. Includendo anche un *timestamp* si potrà avere la *conferma* che il messaggio *non ha subito ritardi* durante la *trasmissione* .

La *sola* *crittografia simmetrica* *non è in grado* di *garantire* un servizio di *autenticazione completo*. Utilizzando un algoritmo ECB, Un *attaccante* potrebbe *riordinare* i blocchi di *ciphertext* alterandone *la sequenza*, e questo *non impedirebbe* al *destinatario* di *decrittarli correttamente* senza accorgersi di nulla (a meno che non sia presente un *numero di sequenza*).

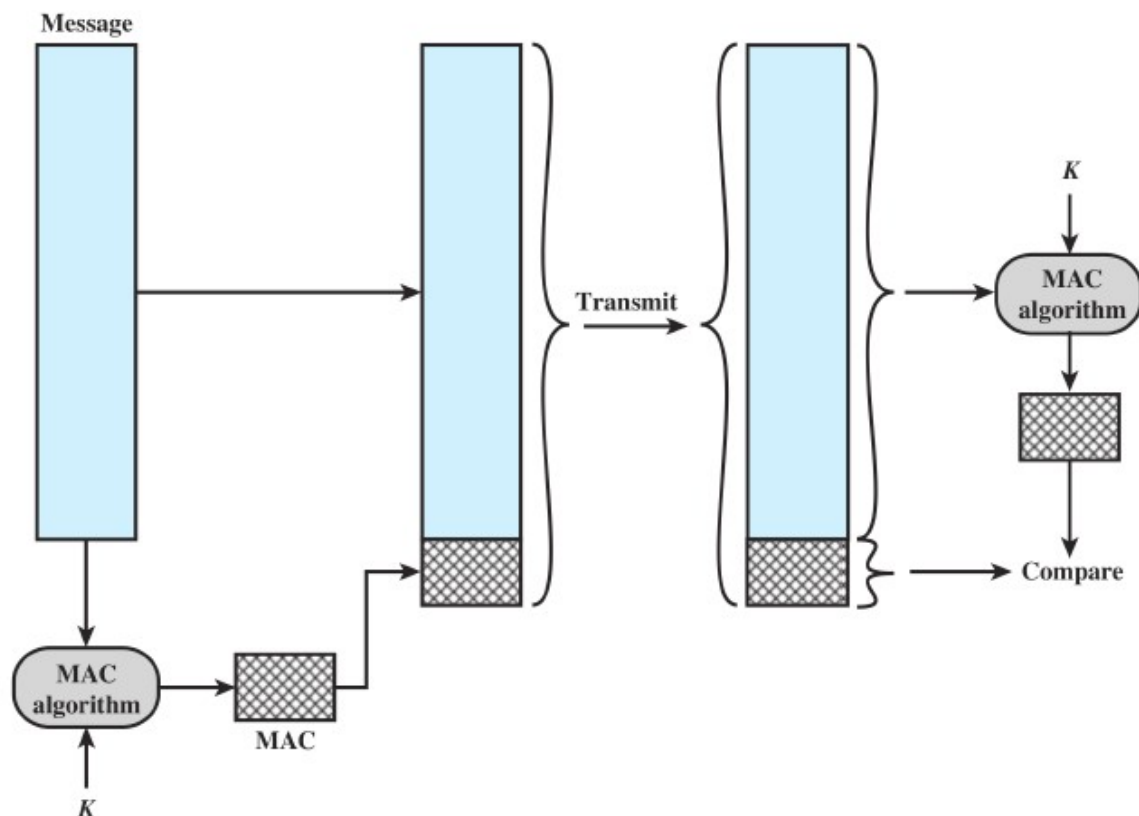
- senza usare *crittografia*

viene *generato* un *authentication tag* e poi *aggiunto* ad ogni messaggio trasmesso. Il *contenuto* del messaggio *non è crittografato* e può essere letto da chiunque *indipendentemente* dalla *funzione di autenticazione* di destinazione. Questo meccanismo quindi *non fornisce confidentiality* ma solo *authentication* . Si può quindi ottenere *sia authentication* che *confidentiality* combinando in un *solo algoritmo* la *crittografia* del messaggio + la *crittografia* dell' *authentication tag*. Di solito però il meccanismo di *autenticazione* viene fornito come una *funzione separata* dalla *crittografia del messaggio*. Esistono infatti alcune situazioni in cui è preferibile *garantire authentication* senza *confidenzialità* :

1. lo stesso messaggio viene *spedito in broadcast* (es. *Notifica rete non disponibile*) : è più economico e affidabile avere *una sola destinazione responsabile* della *verifica dell'autenticità* del messaggio. Il messaggio viene *spedito in chiaro* con associato l'*authentication tag* : se il *responsabile* riconosce *una violazione* allora avviserà *tutti gli altri destinatari*
2. un lato della comunicazione potrebbe *non aver abbastanza tempo* per decrittare *tutti i messaggi* in entrata (causa *sovraccarico*) : l'*autenticazione* avverrebbe solo su *alcuni messaggi* scelti in *maniera casuale*
3. un programma potrebbe essere eseguito senza dover essere *decrittato ogni volta* evitando così un *dispendio* di risorse da parte del processore : se in allegato c'è l'*authentication tag*, allora quando sarà *richiesta* la *verifica dell'integrità* del programma sarà possibile *effettuare un controllo di autenticità*

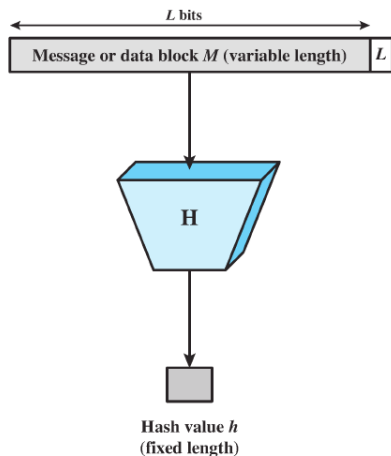
il Message Authentication Code (*MAC*) è una tecnica di autenticazione che utilizza una *chiave segreta* per generare un *piccolo blocco di dati*, chiamato appunto *Message Authentication Code*, che viene aggiunto al messaggio. Questa tecnica prevede che la *chiave segreta* sia *condivisa* tra i due lati della comunicazione e che il *MAC* sia il *risultato* di una *funzione complessa* che viene applicata *sul messaggio* e sulla *chiave segreta*. Il *messaggio + codice* viene trasmesso al destinatario, il quale *usando la stessa chiave* applicherà la *funzione* e *confronterà* il codice ottenuto con quello *allegato* al messaggio ricevuto. *Assumendo che solo il mittente e il destinatario conoscano la chiave*, se i due *codici coincidono* :

- il destinatario ha *la certezza* che il messaggio *non è stato alterato*. Se un *attaccante* avesse alterato il contenuto, senza alterare il codice (abbiamo *assunto* che *non conosca la chiave*) il *codice ottenuto* dal destinatario *non coinciderebbe* con quello *allegato nel messaggio*
- il destinatario ha *la certezza* che il messaggio *arrivi veramente da quel mittente*. Siccome *nessun'altro* conosce la *chiave segreta*, allora *nessuno*, a parte il *mittente giusto*, è in grado di trasmettere un messaggio ed *allegare esattamente quel codice*
- se il messaggio contiene un *numero di sequenza* (es TPC) il destinatario è *sicuro* che il *flusso* dei messaggi ricevuti *non è stato modificato* da nessuno



tipicamente il *codice* (MAC) è composto dagli *ultimi bit* (16 o 32 di solito) del *ciphertext* ottenuto applicando l'algoritmo *crittografico* (DES) al messaggio. Questo processo è *simile* alla *crittografia*, una differenza è che l'algoritmo di *autenticazione* in questo caso *non è reversibile* , e inoltre grazie alle *proprietà matematiche* della funzione di autenticazione questo meccanismo è *meno vulnerabile* rispetto a un semplice *algoritmo crittografico*

la *One-Way Hash Function* è una alternativa alla tecnica di autenticazione MAC.

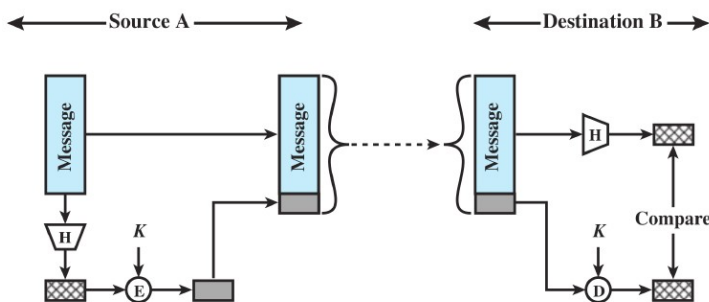


- dato messaggio di una *dimensione variabile* "M" e un *campo* che contiene il *valore (L) della lunghezza in bit* del messaggio *originale*
- Una *hash function* "H" lo accetta *in input* un
- produce *in output* un messaggio *riassunto* (*Hash value*) di *dimensione fissa* $h = H(M)$

la *dimensione* del campo che contiene (L) è una *misura di sicurezza* che aumenta la difficoltà per un attaccante di produrre un *messaggio diverso* che abbia *lo stesso valore hash*.

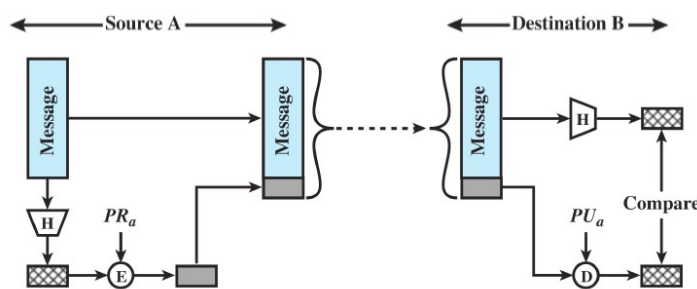
A differenza del "MAC" la *hash function* in input *non usa nessuna chiave segreta*.

Per ottenere autenticazione l'*hash value* viene *spedito assieme* al messaggio *originale* in modo che funga da test per l'*autenticazione* : basterà che il destinatario *ricalcoli* il *valore hash* usando la *stessa funzione* e *confrontarlo* con quello *spedito* assieme al messaggio. L'autenticazione può avvenire *in tre modi diversi* :



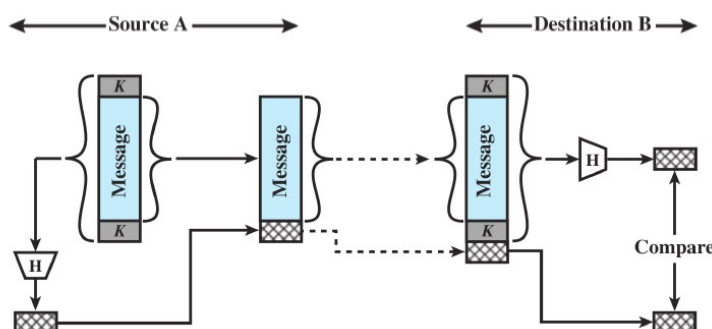
chiave *simmetrica*

- A e B *condividono la stessa chiave*
- B *applica la funzione hash* al messaggio *originale* e *tramite chiave* decrypta l'*hash value* spedito
- assumendo che *solo loro* conoscano la *chiave segreta* è possibile fare *authentication*



chiave *pubblica*

- A usa la sua *chiave privata* per *crittografare* l'*hash value*
- B *decrypta* il valore usando la *chiave pubblica* di A e *confronta* il valore *ottenuto* con quello trovato applicando la *funzione* al messaggio ricevuto



keyed hash MAC

- A e B *condividono una chiave* che viene *utilizzata assieme al messaggio* per produrre il *valore hash*
- B ricevuto il messaggio *riaggiunge* la *chiave* e *ricalcola* il *valore Hash* e poi *confronta*

l'autenticazione basata su *chiave simmetrica* e su *chiave pubblica* invece che crittografare *l'intero messaggio*, operano sull'*hash value* in modo da *evitare computazioni* non richieste. Il metodo più comune però è il *keyed hash MAC* in quanto *evita del tutto* l'uso della *crittografia* :

- *crittografare* il software è un processo *lento*: anche se il messaggio fosse di *piccole dimensione* si creerebbe un *flusso costante* di messaggi in entrata e in uscita dal sistema
- il costo per *crittografare* l'hardware non è trascurabile : l'implementazione su *un'intera rete* ha costi non indifferenti. Inoltre per *blocchi di piccole dimensioni* non converrebbe per l'overhead dovuto a invocazioni e inizializzazioni
- un algoritmo crittografico *di solito* è protetto da un brevetto *che va pagato*

la *one-way hash function*, o più comunemente *secure hash function*, è importante non solo per l'autenticazione ma anche per la *firma digitale* . L'obiettivo della *funzione hash* è quello di produrre un "*fingerprint*" del file, messaggio o blocco di dati. Per poter permettere l'autenticazione del messaggio, una *funzione hash* "H" deve avere le seguenti proprietà :

1. deve poter essere applicata a un *qualunque* blocco di una *qualunque dimensione*
2. deve produrre un risultato di *dimensione fissa*
3. la *funzione* $H(x)$ deve essere *relativamente semplice* da calcolare per *qualunque* x sia per le implementazioni software che hardware

le *prime 3* proprietà sono *richieste* per poter *applicare praticamente* la funzione hash in modo da garantire la *message authentication*.

4. per *qualunque* h deve essere *quasi impossibile* trovare x tale che $H(x) = h$. una funzione hash con questa proprietà è detta *one-way* oppure *preimage resistant* : in altre parole è *semplice* dato un messaggio *generare* un codice ma *virtualmente* dato un codice è *impossibile* ricostruire il messaggio (è detta funzione *one-way* ed è adottata nel metodo *keyed hash MAC*)
5. se per ogni blocco x , è *quasi impossibile* trovare un $y \neq x$ tale che $H(x) = H(y)$ allora la funzione è chiamata *second preimage resistant* oppure *weak collision resistant*. In questo modo si *garantisce* che *non è possibile* creare un messaggio diverso che abbia lo stesso codice dell'originale (proprietà utile per i metodi a *chiave simmetrica* o *pubblica*)
- esempio :
In primo luogo, osservo o intercetto un messaggio più il suo codice hash cifrato . In secondo luogo, genero un codice hash in chiaro dal messaggio *intercettato*. terzo , produco un messaggio alternativo con lo stesso codice hash .
6. se è *quasi impossibile* trovare una coppia qualsiasi (x,y) tale che $H(x) = H(y)$ allora la funzione è chiamata *collision resistant* oppure *strong collision resistant* (in questo modo si evita che un attaccante sia in grado di trovare due messaggi diversi con lo stesso codice
- esempio:
bob arriva a scrivere un messaggio di IOU , inviarlo ad Alice , e lei lo firma . bob trovare due messaggi con lo stesso hash , uno dei quali richiede Alice pagare una piccola quantità e che richiede un pagamento . Alice firma il primo messaggio e Bob è quindi in grado di affermare il *secondo* messaggio è *autentico*

oltre a fornire *authentication*, il *valore hash* consente di verificare l'*integrità* del messaggio ricevuto perché se durante il *transito* viene modificato anche un solo bit del messaggio originale, il destinatario quando ricalcolerà il valore di hash non troverà corrispondenza con quello spedito dal mittente. Come la *crittografia simmetrica* anche la *secure hash function* è soggetta ad attacchi di *criptoanalisi* e *bruteforce* :

- la *criptoanalisi* di una *funzione hash* si basa sullo sfruttamento di falle dell'*algoritmo* utilizzato
- il *bruteforce* invece dipende dalla *lunghezza* dell'*hash code* prodotto dall'*algoritmo*

la *funzione hash* più utilizzata è la *secure hash algorithm* (SHA) sviluppata dal NIST, ed è utilizzata anche per altre funzioni :

- *password* : all'interno del sistema operativo *al posto* della password viene memorizzato il relativo *hash value*. In questo modo se un *attaccante* riuscisse ad accedere al file delle password non riuscirebbe comunque a recuperarla. Quando un *utente* inserisce la password viene applicato SHA e si confronta con l'*hash value* memorizzato. Per questo meccanismo la *funzione hash* deve essere *preimage resistance* o *second preimage resistance*
- *intrusion detection* : bisogna memorizzare la *funzione hash* "H(F)" di ogni file sul sistema, mentre l'*hash value* prodotto andrebbe protetto (ad esempio memorizzato su un CD, o chiavetta). In questo modo se un *attaccante* per avere successo dovrebbe modificare il file senza alterare il suo *hash value*, in quanto all'*utente* basterebbe riapplicare la funzione e confrontare il risultato con quello memorizzato in un posto sicuro

oltre alla *crittografia simmetrica*, basata su *chiave condivisa* esiste un secondo tipo di *crittografia* che viene usata per garantire la *message authentication* e la *key distribution*.

La *crittografia a chiave pubblica* è un tipo di *crittografia asimmetrica* che a differenza di quella *simmetrica*, utilizza due chiavi separate. L'uso di due chiavi genera importanti conseguenze sulla confidenzialità, *key distribution* e *authentication*.

A parte la maggior resistenza agli attacchi di *criptoanalisi* (che dipendono dalla *lunghezza* della chiave e dal *costo computazionale* dovuto al recuperare il testo cifrato) non esiste alcun principio secondo cui la *crittografia asimmetrica* sia migliore di quella *simmetrica* e viceversa. Inoltre il sovraccarico di *costo computazionale* conseguente all'utilizzo di *crittografia asimmetrica* è tale che non prenderà mai completamente il posto di quella *simmetrica* (coesisteranno). Per distribuire la *chiave pubblica* sono necessari dei protocolli che spesso coinvolgono un *agente centrale* e la *procedura* non sono semplici o abbastanza efficienti tanto quanto quelle usate dalla *crittografia simmetrica*.

La *crittografia asimmetrica* è composta da :

- *Plaintext* : è il messaggio in chiaro che verrà poi dato in input all'*algoritmo crittografico*
- *Encryption Algorithm* : è l'*algoritmo crittografico* che trasforma il *plaintext*
- *chiave pubblica* e *privata* : sono le due chiavi che caratterizzano la *crittografia asimmetrica*. Se una è usata per *crittografare* il *plaintext*, l'altra sarà usata per *decrittare* il relativo *ciphertext*
- *Ciphertext* : è il messaggio trasformato dall'*algoritmo crittografato* e dipende dal *plaintext* e dalla *chiave utilizzata*
- *Decryption Algorithm* : è un algoritmo che dato il *ciphertext* e trovata la chiave giusta, restituisce il *plaintext*

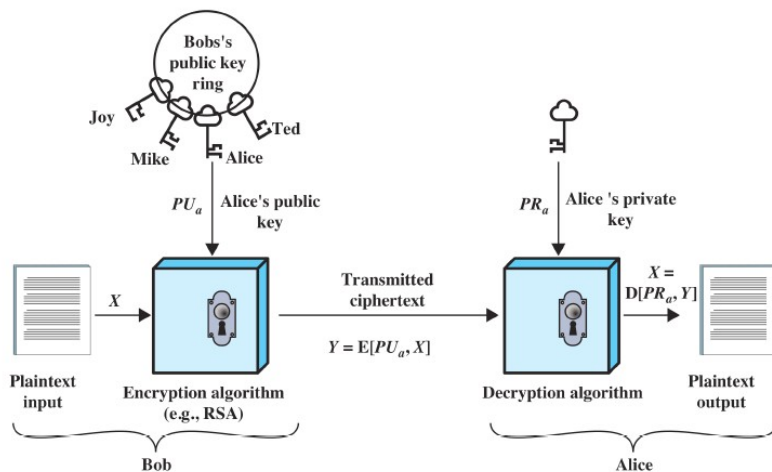
come suggerito dal nome, la *chiave pubblica* deve essere *resa nota* a tutti, mentre la *chiave privata* deve restare a conoscenza *solo del suo proprietario* :

- ogni utente *genera una coppia di chiavi* utilizzate per *crittografare e decrittografare* i messaggi
- ogni utente *pubblica una chiave* in un registro pubblico *accessibile a tutti* mentre *tiene segreta* l'altra.

In questo modo, *tutti i partecipanti* hanno accesso alla *chiave pubblica*, mentre la *chiave privata* viene *generata in locale* da ogni utente e non viene mai distribuita. Finché un utente *manterrà segreta* (protetta) la propria *chiave privata*, la *comunicazione* è sicura.

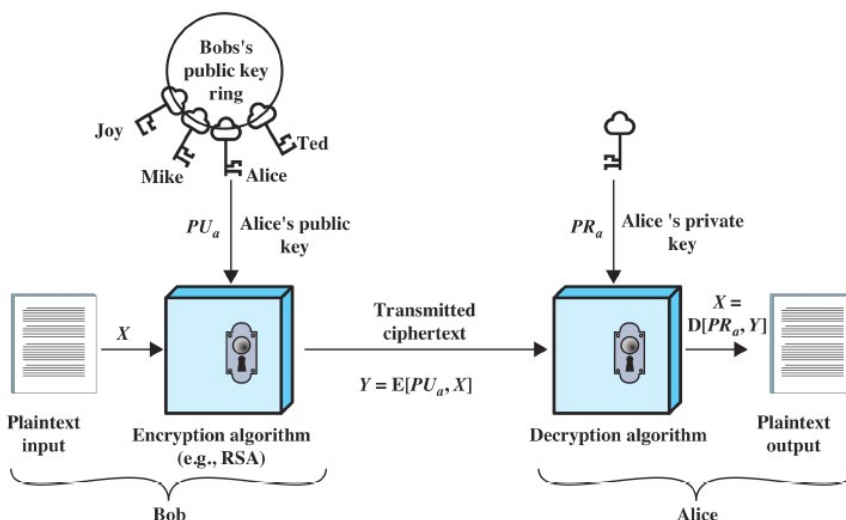
A seconda della *chiave utilizzata* si può ottenere *confidentiality* o *authentication/data integrity*

- *crittografia a chiave pubblica* : Bob (*mittente*) genera il *ciphertext* utilizzando la *chiave pubblica* di Alice (*destinatario*). Alice per poter *decryptare* il messaggio inviatole dovrà utilizzare la *propria chiave privata*.



La *confidentiality* è dovuta dal fatto che *solo il vero destinatario* sarà in grado di *decryptare* il *ciphertext* in quanto è *l'unico* ad avere la *chiave privata* . La confidenzialità però dipende anche da altri fattori, quali la *sicurezza dell'algoritmo*, dei *protocolli utilizzati* e si basa sul fatto che *nessuno al di fuori del proprietario* conosca la *chiave privata*

- *crittografia a chiave privata* : Bob (*mittente*) genera il *ciphertext* utilizzando la *sua chiave privata*. Alice (*destinatario*) per poter *decryptare* il messaggio inviatole dovrà utilizzare la *chiave pubblica* di Bob



la *authentication* o *data integrity* è data dal fatto che se un *utente* riesce a ricavare il *plaintext* dal *ciphertext* utilizzando la *chiave pubblica* di Bob (*mittente*) allora vuol dire che il *plaintext* è stato generato *per forza* utilizzando la *chiave privata* di Bob, che *solo Bob possiede* e quindi è *l'unico* in grado di *criptare* quel messaggio. la *authentication* o *data integrity* dipende comunque anche da altri fattori, *il primo dei quali* riguarda la *certezza* che il *mittente* è chi dice di essere.

Il sistema crittografico illustrato dipende dall'*algoritmo crittografico* basato sulla *coppia di chiavi*. Questo sistema è stato postulato da *Diffie and Hellman* senza però *dimostrare l'esistenza* di un tale algoritmo. Tuttavia hanno *indicato* quali devono essere le *condizioni* che l'*algoritmo crittografico* deve soddisfare :

- deve essere *computazionalmente semplice* per un utente *generare la coppia di chiavi*
- per il *mittente* deve essere *semplice* , una volta recuperata la *chiave pubblica del destinatario* , *generare il ciphertext* del plaintext che vuole crittografare
- per il *destinatario*, una volta ricevuto il *ciphertext*, deve essere *semplice* decrittarlo e recuperare il plaintext originale *usando la propria chiave privata*
- per un *attaccante* deve essere *praticamente impossibile* determinare la *chiave privata* di un utente, *partendo dalla sua chiave pubblica*
- per un *attaccante*, anche *se conosce* la *chiave pubblica* del mittente e il *ciphertext* generato, deve essere *praticamente impossibile* risalire al plaintext
- si *può scegliere quale chiave* utilizzare per *crittografare* e di conseguenza l'altra sarà utilizzata per *decrittografare*

i *principali algoritmi crittografici asimmetrici* sono :

- RSA (*Rivest, Shamir, Adleman*) : è l'*algoritmo più adottato e utilizzato*. Implementa un approccio a *chiave pubblica* (*garantisce confidentiality*) ed è un algoritmo tipo *block cipher* dove il *plaintext* e il *ciphertext* sono *interi* compresi tra 0 e $n - 1$ dato un *certo* n . attualmente la *dimensione della chiave* utilizzata per *crittografare* deve essere di 1024 *byte* per essere *considerata forte abbastanza* per ogni tipo di applicazione.
- Diffie-Hellman Key Agreement : è il primo algoritmo a *chiave pubblica* apparso e definisce la *crittografia a chiave pubblica*. L'obiettivo del Diffie-Hellman key exchange (viene chiamato anche così) è quello di *permettere a due utenti* di *mettersi d'accordo in modo sicuro*, utilizzando un *canale insicuro*, e stabilire una *chiave segreta* che potrà essere utilizzata per la *crittografia simmetrica* del messaggio che si scambieranno. Questo algoritmo infatti viene utilizzato *solo per lo scambio di chiavi*
- Digital Signature Standard : pubblicato dal *NIST* e offre solo la funzione di *digital signature* utilizzando l'algoritmo SHA-1. A *differenza* di RSA *non può essere utilizzato* per la *crittografia* o lo *scambio di chiavi*
- Elliptic Curve Cryptography : è un algoritmo che *offre lo stesso livello di sicurezza* di RSA ma *utilizza una chiave meno lunga*, e di conseguenza ne *riduce l'overhead*

Algoritmo	Digital Signature	Symmetric key distribution	Encrypt of secret key
RSA	Yes	Yes	Yes
Diffie-Hellman	No	Yes	No
DSS	Yes	No	No
Elliptic curve	Yes	Yes	Yes

Gli algoritmi a *chiave pubblica* sono utilizzati in *molte applicazioni*, che possono essere racchiuse in due *grandi categorie* : *digital signature* , e varie tecniche di *key management and distribution*.

Rispetto al *key management and distribution* bisogna considerare *tre aspetti distinti* :

- la *distribuzione sicura* della *chiave pubblica*
- l'*uso* della *crittografia* a chiave pubblica per *distribuire* la *chiave segreta*
- l'*uso* della *crittografia* a chiave pubblica per *creare una chiave temporanea* da utilizzare per *crittografare* il messaggio

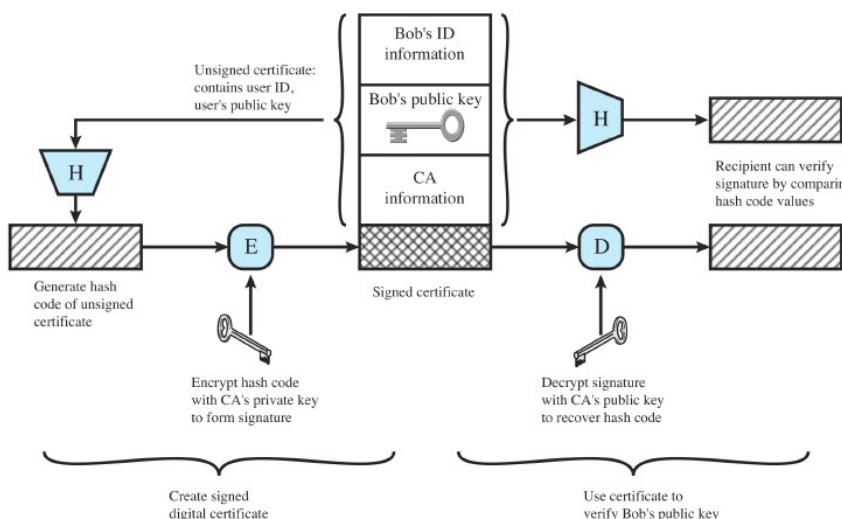
la *crittografia* a *chiave pubblica* può essere utilizzata per ottenere *authentication*. In questo contesto *non è importante* che il *contenuto del messaggio* sia segreto, ma si vuole avere la *certezza* che il messaggio sia *stato veramente spedito* da quel *mittente*. A questo scopo, Bob (il *mittente*) usa una *funzione hash* per generare l'*hash value* del messaggio, *cripta l'hash code* utilizzando la sua *chiave privata* creando quindi una *digital signature*. A questo punto Bob spedisce il messaggio con *allegata* la *digital signature*. Alice (*destinatario*) a sua volta :

- calcola l'*hash value* del messaggio
- utilizza la *chiave pubblica* di Bob (*mittente*) per *decriptare* la *digital signature*
- compara l'*hash* calcolato con quello *decriptato*
- se *coincidono* allora Alice avrà la *certezza* che il messaggio è *stato firmato* da Bob (o più precisamente *con la chiave privata* di Bob)

essendo impossibile alterare il messaggio *senza aver accesso* alla *chiave privata* del mittente, il messaggio è *autentico* sia in termini di *sorgente* (da chi è stato spedito) che di *integrità dei dati* (il contenuto è quello originale). La *digital signature* però *non fornisce confidentiality* in quanto permette di spedire messaggio *senza che questo possa subire alterazioni* ma *non ne impedisce* l'intercettazione da parte di terzi :

- a volte viene *crittografata solo la digital signature* mentre il resto del messaggio viene *spedito in chiaro*
- anche se il messaggio *venisse crittografato* sarebbe comunque possibile decrittarlo con la *chiave pubblica* del mittente

un utente potrebbe comunque *pretendere di essere* Bob e spedire la *chiave pubblica* a un destinatario oppure in broadcast a tutti quanti. Fino a quando il *vero Bob* non si accorge di tale *falsificazione* e non *allerta* gli altri partecipanti, il *Bob fasullo* sarà in grado di *leggere tutti i messaggi crittografati* destinati al *vero Bob* e può usare una *chiave fasulla* per effettuare *authentication*. Per superare questo pericolo, la *chiave pubblica* viene *certificata*: un *certificato* consiste nella *chiave pubblica* più un *user ID* del proprietario, che viene *segnata come affidabile* da una terza parte (*certificate authority*). Il *certificato* include informazioni sulla *certificate authority* che l'ha firmato e il *periodi di validità* del certificato stesso.



Un utente può quindi presentare la sua *chiave pubblica* a una *authority* in *maniera sicura* e ottenere un *certificato firmato*. L'utente a questo punto può *pubblicare* il certificato e ogni volta che qualcuno ha bisogno della sua *chiave pubblica* sarà in grado di *verificarne*, tramite il *certificato firmato allegato*, la sua *validità*.

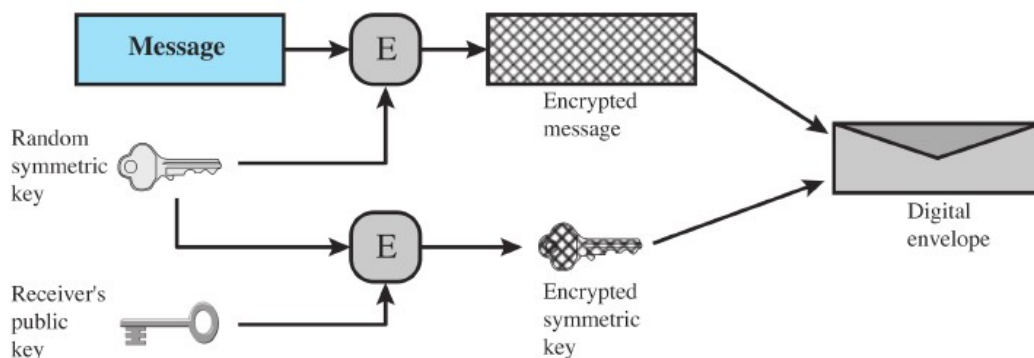
Con la *crittografia simmetrica* sorge quindi la necessità di *comunicare in maniera sicura la chiave segreta condivisa* che verrà utilizzata per *crittografare* i messaggi. Uno degli approcci più usati è *l'algoritmo Diffie-Hellman key-exchange* il quale però *non garantisce authentication*. Per superare questo problema si utilizzano *altri approcci basati algoritmi a chiave pubblica*, tra i quali il *digital envelopes*.

Il *digital envelopes* può essere utilizzato per *proteggere* un messaggio *senza la necessità* che il mittente e il destinatario *si accordino sulla chiave segreta* da utilizzare. Il *meccanismo* è l'equivalente di una *busta chiusa* che contiene una *lettera non firmata*. Supponiamo che Bob voglia mandare un messaggio ad Alice *senza aver condiviso la chiave simmetrica* :

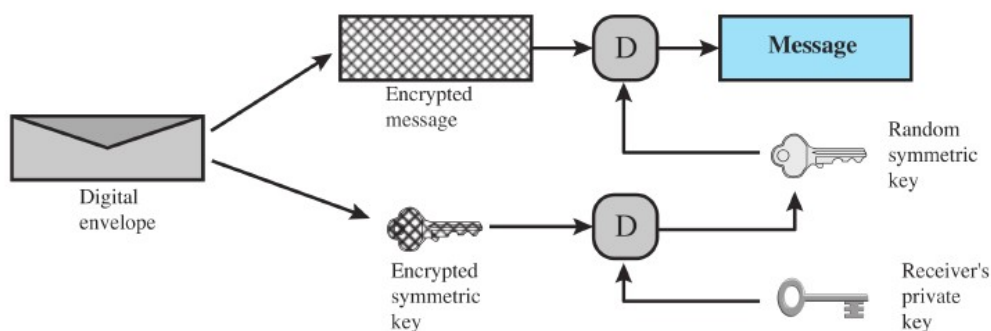
- Bob prepara il messaggio
- genera una *chiave simmetrica random* che utilizzerà *solo questa volta*
- utilizza la *chiave generata* per *crittografare* il messaggio

una volta *crittografato* il messaggio per *evitare di condividere la chiave segreta* si utilizza un approccio a *chiave pubblica*

- Bob utilizza la *chiave pubblica* di Alice (*destinatario*) per *crittografare* la *chiave generata* con la quale ha *crittografato* il messaggio
- Bob spedisce il *messaggio criptato* e in allegato *la chiave simmetrica criptata*



in questo modo *solo Alice* sarà in grado di *decrittografare* la *chiave simmetrica* e di conseguenza *risalire* al plaintext. Inoltre se *Alice* ha una *chiave pubblica certificata* sarà garantita anche *authentication*



i numeri casuali giocano un ruolo importante nell'uso della crittografia nelle varie applicazioni di sicurezza di rete. Molti algoritmi basano la crittografia su numeri casuali :

- la generazione della chiavi adottata da RSA ed altri algoritmi a chiave pubblica
- la generazione della stream key per gli algoritmi di tipo symmetric stream cipher
- durante la creazione del digital envelopes quando si crea la chiave simmetrica di sessione
- in uno scenario di key distribution, i numeri casuali sono utilizzati per fare handshaking per prevenire gli attacchi di tipo replay
- generazione di chiavi di sessione, quando viene eseguita da centro di distribuzione

il problema della generazione di una sequenza di numeri presumibilmente casuali è che tale sequenza sia casuale anche dal punto di vista statistico. Per validare la randomness (casualità) di una sequenza di numeri (e quindi stabilire che è casuale), vengono utilizzati due criteri :

- distribuzione uniforme: la frequenza delle occorrenze di ogni numero dovrebbe essere la stessa. Esistono diversi algoritmi in grado di stabilire se una particolare sequenza di numeri soddisfa questo criterio.
- indipendenza : nessun valore della sequenza deve dipendere dagli altri. A differenza della distribuzione uniforme non esiste un algoritmo per stabilire se una sequenza è indipendente. La soluzione è quella di effettuare tanti test in grado di dimostrarne la non indipendenza fino a quando non si raggiunge un livello di indipendenza sufficientemente forte.

L'uso di una sequenza di numeri apparentemente casuali statisticamente spesso si verifica nella progettazione di algoritmi crittografici. Ad esempio uno dei requisiti fondamentali della crittografia a chiave pubblica RSA è la capacità di generare numeri primi . In sostanza se un problema è troppo complesso o richiede troppo tempo per essere risolto esattamente, si utilizza un approccio più semplice basato sulla randomizzazione in grado di fornire una soluzione che soddisfi qualunque livello di confidenza richiesto.

- Esempio : generalmente è estremamente difficile stabilire se un numero sufficiente grade sia primo. esiste tuttavia un numero di algoritmi in grado di verificare la primalità di un numero usando una sequenza casuale di interi scelti come input a causa della loro semplice computazione. Se questa sequenza è abbastanza lunga, allora la primalità del numero verrà determinata con un discreto livello di certezza. Questo tipo di approccio, conosciuto come randomization (randomizzazione) viene spesso adottato nella progettazione degli algoritmi crittografici

Nelle applicazioni come la autenticazione reciproca o la generazione di chiavi di sessione quello che viene richiesto non è tanto una sequenza di numeri statisticamente casuali ma che l'elemento successivo della sequenza non sia predicibile. In una vera sequenza casuale ogni numero è statisticamente indipendente dagli altri e quindi la sequenza è imprevedibile. Le vere sequenze casuali però non sono sempre usate : piuttosto si utilizzano sequenze di numeri apparentemente casuali generati da un algoritmo. In questo caso bisogna far attenzione che un attaccante non sia in grado di predire l'elemento successivo della sequenza basandosi sui precedenti.

Un generatore di vere sequenze casuali (TRNG) usa una sorgente non deterministica per produrre casualità (Randomness). Molti di essi si basano su processi naturali non predicibili.

Le applicazioni crittografiche utilizzano degli algoritmi per generare le sequenze di numeri casuali. Questi algoritmi però sono deterministici e quindi producono sequenze che a livello statistico non sono casuali; tuttavia se l'algoritmo è buono, allora la sequenza generata sarà comunque in grado di superare una serie ragionevole di test di casualità (randomness) . In questo caso tale sequenza verrà definita una sequenza di numeri pseudocasuale (pseudo randomness)

AUTENTICAZIONE

"l'autenticazione è un processo che verifica una identità *richiesta da oppure di un sistema*"

RFC 2828

il *processo di authentication* consiste in due step :

- *Identification* : viene *presentato un identificatore* al sistema di sicurezza (l'autenticazione di un identità è la base di tutti i servizi di sicurezza, tra i quali il controllo degli accessi e l'user accountability)
- *Verification* : vengono *presentate o generate* informazione di autenticazioni che confermano il legame tra l'identità e il suo identificatore

il processo di autenticazione è la *prima linea di difesa*.

Esempio : l'utente *Alice Toklas* ha come *identificatore (user id)* ABTOKLAS. Questa informazione deve essere *memorizzata su ogni dispositivo* a cui Alice vuole accedere, e inoltre deve essere *conosciuta anche agli amministratori di sistema e agli altri utenti*. Un oggetto tipico utilizzato per *autenticare* le informazioni associate all'identificatore è la *password* .

In questo modo, *essendo noto l'identificatore*, chiunque può ad esempio *mandare e-mail* ad Alice *ma nessuno può pretendere di essere Alice* (in quanto *non conosce la password*).

La *combinazione* dell' *identificatore* di Alice e la sua *password* consente agli amministratori di *impostare i permessi di accesso* e di *controllare le sue attività*.

In sostanza :

- *l'identificazione* è l'azione attraverso la quale un utente *fornisce* al sistema *le informazioni* (user id e password) *sulla identità* con la quale *vuole accedere* .
- *L'user authentication* è il meccanismo attraverso il quale *il sistema stabilisce la validità* delle informazioni *riguardo all'identità richiesta* durante l'identificazione
- *message authentication* è una procedura che permette *alle parti che comunicano* di *verificare* che il contenuto dei messaggi ricevuti *non è stato alterato* e che la fonte è *autentica*

esistono *in generale* 4 modi per *autenticare l'identità* di un utente :

- attraverso qualcosa *che conosce* : chiedendo *password , pin* , o le risposte a una serie di *domande predisposte*
- attraverso qualcosa *che possiede* : si utilizzano dei *token* ad esempio *smartcard, chiavi fisiche, chiavi elettroniche*
- attraverso qualcosa *che è* (*biometrica statica*): ad esempio *impronte digitali, il viso* o la *retina*
- attraverso qualcosa *che fa* (*biometrica dinamica*) : ad esempio viene *registrata la voce* oppure si *ricosce la scrittura* o viene chiesto di eseguire una *sequenza ritmata*

questi modi possono essere usati *singolarmente* oppure venire *combinati tra loro*. Ognuno di essi fornisce *user authentication* ma nessuno è *esente da problemi* :

- un attaccante *può indovinare* la password o il pin di utente, o più banalmente quest'ultimo *se li può dimenticare*
- un attaccante *può rubare* un token oppure un utente *può perderlo*
- per la *biometria* invece c'è ad esempio il problema dei *falsi positivi e negativi*, e soprattutto dei *costi elevati*

la linea di difesa *più utilizzata* contro le intrusioni è quella *basata su password* : il sistema *confronta* la password *digitata* con quella *precedentemente registrata* associata a quello specifico *user id* e mantenuta all'interno di *un file delle password*.

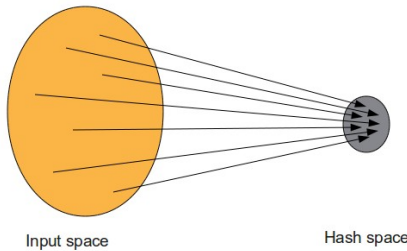
- La *password* serve per *autenticare l'user id* che si sta utilizzando per *collegarsi al sistema*.
- L'*user id* invece fornisce *sicurezza* nei seguenti modi
 - determina se *l'utente* associato è *autorizzato all'accesso* al sistema. Soltanto chi ha il *proprio user id* registrato sul *file ID* all'interno del sistema, può accedervi.
 - Determina i *privilegi* assegnati all'utente. Alcuni sistemi prevedono *account "ospite"* che assegnano *privilegi limitati* rispetto a degli *utenti normali*.
 - Viene utilizzato per *riferirsi* a un utente per *personalizzare il controllo degli accessi*. Ad esempio un utente *potrebbe abilitare i permessi di lettura* dei suoi file a *determinati utenti* indicando i loro *user ID*

un sistema che usa la *password* per garantire *autenticazione*, mantiene un *file delle password* indicizzato per *user ID* . Tipicamente *non viene memorizzata la password* ma il risultato di una *one-way hash function* applicata alla password stessa. I *principali attacchi* effettuati a questi sistemi sono :

- *offline dictionary attack* : l'attaccante riesce a *ottenere il file delle password* e inizia a confrontare gli *hash code* con gli *hash delle password più comuni*. Se riesce a *fare match* allora avrà *ottenuto l'accesso* al sistema con quella combinazione *user id / password*.
 - Le *contromisure* adottate includono il *controllo degli accessi non autorizzati* al file delle password, misure di *intrusion detection* per *identificare compromissioni*, e un meccanismo di *reimmersioni* delle password compromesse
- *specific account attack* : l'attaccante prende di mira *un preciso utente* e cerca di *indovinare la password* effettuando dei *tentativi* finché non ci riesce.
 - La *contromisura* adottata è quella di *bloccare l'accesso* all'account *dopo un limitato numero* di tentativi
- *popular password attack* : si tenta di *indovinare la password* di *una lista di user ID* utilizzando *le più popolari*.
 - Le *contromisure* adottate prevedono *delle politiche* che vietano agli *utenti* di scegliere *password comuni* e meccanismi di *scanning* dell'indirizzo IP che *richiede autenticazione*
- *password guessing against single user* : l'attaccante cerca di *acquisire informazioni* sull'utente e sulle *politiche* adottate dal sistema per le password e le *usa per cercare di indovinare* quella che gli interessa.
 - La *contromisura* adottata riguarda tutti quei meccanismi che rendono una password *difficile da indovinare*
- *workstation hijacking* : l'attaccante *aspetta* che l'utente *abbandoni la postazione* dalla quale è *collegato* al sistema.
 - La *contromisura* adotta prevede il *log out automatico* dopo un certo *periodo di inattività*
- *exploiting users mistake* : utilizzando *password complesse* un utente è portato a *scriversela per ricordarsela*. Un attaccante può sfruttare la *reverse engineering* per ottenerla
 - la *contromisura* adottata prevede di *combinare l'utilizzo di password semplici* con *altri meccanismi di autenticazione*
- *exploiting multiple password use* : un attacco *produce più danni* se la password ottenuta è *utilizzata in diversi sistemi*
 - la *contromisura* adottata prevede il *divieto* di utilizzare *password simili* su dispositivi *diversi*
- *electronic monitoring* : se una password *viaggia in rete* può essere *intercettata* e , anche se *crittografata*, riutilizzata da un attaccante *per accedere al sistema senza decriptarla*

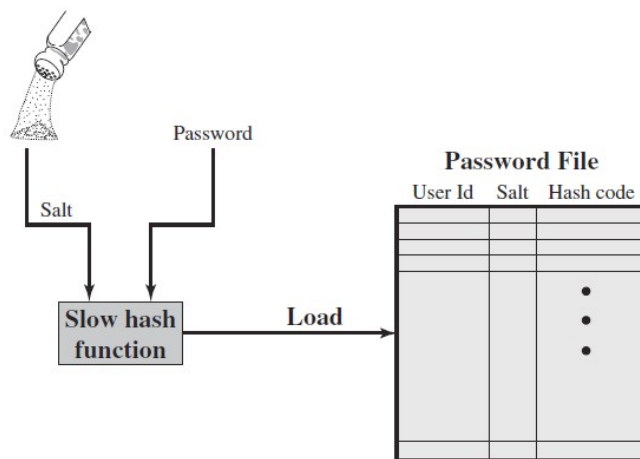
la *tecnica di password security* più utilizzata prevede l'uso di *password hashed* e del *sale* (salt value).

Quando una *nuova password* (scelta oppure assegnata) deve essere *caricata nel file delle password* del sistema, essa *viene combinata con un valore di lunghezza fissa* chiamato *sale* (nelle nuove implementazioni si tratta di un numero *casuale o pseudocasuale*). la *password* e il *sale* vengono dati in pasto a un *algoritmo di hash* che produrrà un *hash code di lunghezza fissa*.



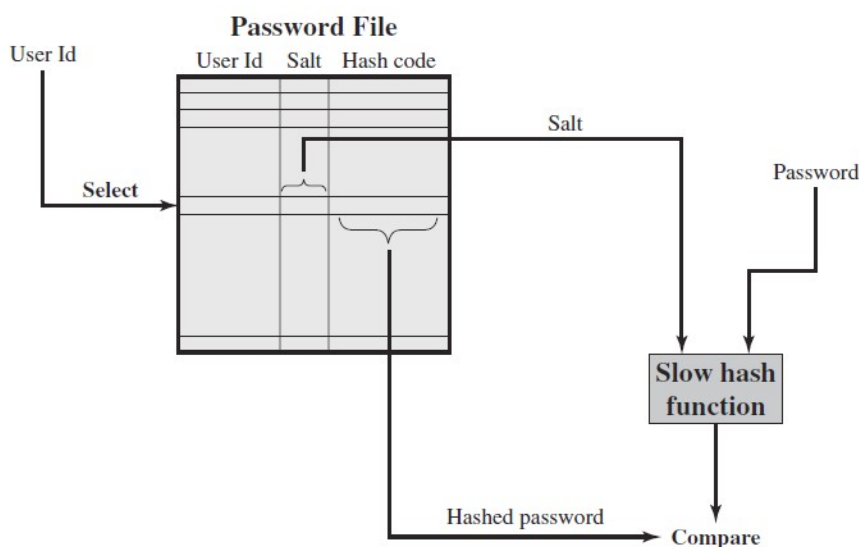
utilizzando una funzione hash che produce un hash code di lunghezza fissa più sono le password da memorizzare e più c'è il rischio che due password diverse ottengano lo stesso hash code (collision)

l'*hash code* viene quindi *memorizzato insieme a una copia in chiaro del sale*, nel *file delle password* in corrispondenza del relativo *user ID*. Ogni password ha un *sale* diverso.



(a) Loading a new password

Quando un *utente* vuole accedere al sistema deve fornire *user ID e password* . Il sistema utilizza l'*user ID* per recuperare all'interno del file delle password il *sale* e la *password crittografata* (hash code). A questo punto il *sale* viene la *password* fornita dall'utente vengono passati all'*algoritmo di hash* che produce un nuovo *hash code*. Se questo *hash code* coincide con quello *memorizzato* all'interno al *file delle password* allora la *password* viene *accettata* e l'*utente ha accesso*



(b) Verifying a password

il *sale* consente di *soddisfare* alcuni obiettivi :

- evita che all'interno del *file delle password* siano visibili *password duplicate* : in questo modo se *utenti diversi* scelgono la stessa password, grazie al fatto che *ogni password* ha un *sale diverso*, verranno generati *hash code diversi*.
- L'utilizzo del *sale* *aumenta la difficoltà* di un attacco *offline dictionary*. Se la lunghezza del *sale* è di b bits allora il *numero di password possibili* aumenterà di un fattore 2^b rendendo *molto difficile indovinarla*
 - se non fosse utilizzato il *sale* : l'attaccante dovrebbe limitarsi a sottomettere alla *funzione hash* un gran numero di *possibili password*. Una volta ottenuta una *corrispondenza*, avrebbe ottenuto direttamente la password.
 - Utilizzando il *sale* : l'attaccante dovrebbe, per ogni possibile password, sottomettere alla *funzione hash* tutte le *possibili combinazioni* con tutti i valori del *sale* presenti nel *file delle password* che ha ottenuto.
- diventa *praticamente impossibile* che la stessa password venga utilizzata in più sistemi diversi

nel mondo UNIX sono state sviluppate *tre implementazioni* di questo schema

- schema *originale*
 - l'utente poteva scegliere una password lunga *fino a 8 caratteri stampabili* che veniva poi convertita in una chiave lunga 56 bit che serviva da *input* per l'algoritmo *crittografico*
 - l'algoritmo *crittografico*, conosciuto come *crypt(3)*, era basato su una *one-way hash function* (DES) e utilizza un *sale* di lunghezza 12 bit
 - per *rallentare l'esecuzione* dell'algoritmo *crittografico*, veniva *ripetuto 25 volte* e come *chiave* per il DES veniva utilizzata una *sequenza di zeri*
 - il *risultato* dell'algoritmo veniva poi *traslato* di 11 caratteri
- schema basato su MD5 (che verrà *bucato*) o SHA512
 - l'utente può scegliere una password di *qualsiasi lunghezza*.
 - L'algoritmo *crittografico*, utilizza un *sale* di 56 bit e *produce un hash code* lungo 128 bit
 - per raggiungere un *buon livello di lentezza*, l'algoritmo veniva *iterato 1000 volte*
- schema basato su BlowFish un *symmetric block cipher*
 - l'utente può scegliere una password lunga *fino a 55 caratteri*
 - l'algoritmo, chiamato *Bcrypt*, utilizza un *sale casuale* lungo fino a 128 bits e *produce un hash code* lungo 192 bit
 - l'algoritmo prevede un *costo variabile* che *incrementa il tempo di esecuzione* dell'algoritmo

l'approccio tradizionale per *indovinare le password*, chiamato anche *password cracking*, consiste nello sviluppare un *dizionario* che contiene *tutte le possibili password* e tentare di *fare match* con quelle presenti nel *file delle password*.

- Ogni password del *dizionario* deve essere *passata alla funzione hash* usando *tutti i possibili sali* presenti nel *file delle password* e tentare di *fare match* con gli *hash code*.
 - Se non *fa match*, il *cracking program* tenta di *variare le password* del *dizionario* includendo ad esempio *numeri, simboli o caratteri speciali* e riprova
- un'alternativa è *precalcolare gli hash code*. In questo modo l'attaccante *si crea già* un *dizionario* composto da gli *hash code* di *tutte le password* combinate con *tutti i sali possibili*. Il risultato è una *tabella di dimensioni enormi* chiamata *rainbow table*
 - una *contromisura* si basa sull'utilizzo di un *sale* e un *hash code* sufficientemente lunghi

il primo approccio occupa *poco spazio* ma chiede *capacità di calcolo elevate* in quanto ogni volta deve *calcolare l'hash code* di ogni password, mentre il secondo occupa *molto spazio* ma richiede *poco sforzo computazionale* in quanto l'*hash* è già *calcolato* e deve fare *solo confronti*.

tentare di *indovinare* una password utilizzando un *banale* attacco *brute force* è *praticamente impossibile* in quanto *tentare tutte le combinazioni possibili di caratteri* richiede un *enorme quantitativo di tempo* (si parla di anni). Invece, i *password cracker* fanno affidamento sul fatto che *molti utenti* scelgono password *facilmente indovinabili*.

Alcuni *utenti*, se gli viene data la possibilità di *scegliere da soli* la password, ne scelgono una *molto corta*. Uno studio del 1992 ha osservato che *quasi il 3%* degli utenti sceglie password *composte da al massimo 3 caratteri*

Length	Number	Fraction of Total
1	55	.004
2	87	.006
3	212	.02
4	449	.03
5	1260	.09
6	3035	.22
7	2917	.21
8	5772	.42
Total	13787	1.0

il *rimedio più semplice* consiste nel *vietare l'utilizzo* di password *inferiore* ai 6 caratteri oppure di *accettare solo password* di 8 caratteri. Questo tipo di *restrizione* da un lato *non incontra resistenze* da parte degli utenti e dall'altro *complica la vita* di un possibile attaccante

Purdue University study on
54 systems and 7000 users

oltre *alla lunghezza* delle password, un'altro problema nasce dal fatto che *spesso* gli utenti scelgono password *facilmente indovinabili* in quanto formate dal *loro nome*, *indirizzo*, oppure da *parole comuni*. È stato dimostrato come *quasi il 25%* delle password UNIX (circa 14.000) sono state *indovinate* da un normale *password cracker*

la strategia adottata prevedeva :

1. *tentava il nome* dell'utente, il nome *dell'account* e altre *informazioni personali* di rilievo
2. tentare parole *provenienti da diversi dizionari*
3. *permutare le parole* dei diversi dizionari, aggiungendo *maiuscole* o *caratteri di controllo*, invertendole o *sostituendo* le lettere coi numeri
4. *effettua altre permutazioni* non considerate precedentemente

Type of Password	Search Size	Number of Matches	Percentage of Passwords Matched	Cost/Benefit Ratio ^a
User/account name	130	368	2.7%	2.830
Character sequences	866	22	0.2%	0.025
Numbers	427	9	0.1%	0.021
Chinese	392	56	0.4%	0.143
Place names	628	82	0.6%	0.131
Common names	2239	548	4.0%	0.245
Female names	4280	161	1.2%	0.038
Male names	2866	140	1.0%	0.049
Uncommon names	4955	130	0.9%	0.026
Myths and legends	1246	66	0.5%	0.053
Shakespearean	473	11	0.1%	0.023
Sports terms	238	32	0.2%	0.134
Science fiction	691	59	0.4%	0.085
Movies and actors	99	12	0.1%	0.121
Cartoons	92	9	0.1%	0.098
Famous people	290	55	0.4%	0.190
Phrases and patterns	933	253	1.8%	0.271
Surnames	33	9	0.1%	0.273
Biology	58	1	0.0%	0.017
System dictionary	19683	1027	7.4%	0.052
Machine names	9018	132	1.0%	0.015
Mnemonics	14	2	0.0%	0.143
King James bible	7525	83	0.6%	0.011
Miscellaneous words	3212	54	0.4%	0.017
Yiddish words	56	0	0.0%	0.000
Asteroids	2407	19	0.1%	0.007
TOTAL	62727	3340	24.2%	0.053

infine, data la *lista delle 100.000* più utilizzate *collezionate* da xato, è stato evidenziato come *oltre il 40%* delle password rientra nella *top 100*, mentre se si estende l'analisi alle *top 500* si nota come la percentuale aumenta fino al 71%

un modo per *contrastare* un attacco alla password consiste nel *vietare* all'attaccante *l'accesso* al file di password. La parte di file contenente *l'hash code* delle password viene *reso accessibile* sono ad *utenti privilegiati*, in modo che un attaccante *per accedervi* deve prima *venire a conoscenza* delle loro password. *Spesso* questi *hash code* vengono memorizzati *in file separati* rispetto ai relativi *user ID* chiamati *shadow password file*, sui quali viene posta *una particolare attenzione* sulla *protezione da accessi non autorizzati*. Sebbene la *protezione* del file delle password sia utile, il file resta *comunque esposto* ad altre vulnerabilità :

- molti sistemi sono *suscettibili* a *effrazioni impreviste* : un attaccante potrebbe *sfruttare una vulnerabilità* all'interno del sistema operativo e *bypassare il sistema di controllo degli accessi* e recuperare il password file oppure *potrebbe trovare delle falle* all'interno del file system o del database management system *che gli consentono di accedere al file* .
- *Errori di permessi* : un *incidente generico* potrebbe *rendere leggibile* il file delle password e quindi *compromettere* tutti gli account
- *stessa password, sistemi diversi* : un sistema *diverso* viene *bucato* e la *password recuperata* in quel sistema è *la stessa utilizzata* per accedere a questo *compromettendolo*
- *mananza o debolezza* nella *physical security* : a volte un attaccante *riesce ad accedere* alle *copie di back up* del file delle password memorizzate su *dischi esterni* o in alternativa *può accedere al file delle password* eseguendo l'avvio di tale disco *da un sistema operativo diverso*.
- *Sniffing della rete* : invece di *catturare il file delle password*, un attaccante può *intercettare in rete* e *collezionare* gli *user ID* e le *relative password*

in sostanza una *politiche di protezione* delle password , deve *abbinare* alle *misure di controllo* anche delle *tecniche* che costringono gli utenti a utilizzare *password difficilmente indovinabili*

quando *non sono costretti*, molti utenti scelgono password *molto corte* o *facilmente indovinabili*. Tuttavia se da un lato una *password casuale* di 8 caratteri ha l'effetto di *evitare attacchi di password cracking* dall'altro *aumenta la probabilità* che l'utente stesso *non se la ricordi*.

L'obiettivo è quello di *eliminare le password facilmente indovinabili* e allo stesso tempo permettere all'utente di scegliere *password che è in grado di ricordare*.

Per raggiungere l'obiettivo si *usano tecniche* che *aiutano l'utente* nella scelta della password

- *User education* : vengono *impartite delle linee guida* che aiutino l'utente a scegliere *password forti*. Il problema è che *molti utenti le ignoreranno* o più semplicemente *non saranno in grado* di capire se quella scelta *si tratta di una password forte oppure no* : una *tecnica molto utilizzata* è quella di scegliere come *password* l'insieme delle *iniziali* delle parole che *compongono una frase* che l'utente *sicuramente ricorda*
(frase : Il mio cane si chiama Rex psw : ImcscR)
- *Computer-generated password* : se la password è *abbastanza casuale* , l'utente *non sarà in grado* di ricordarla. Se tale password è *pronunciabile* l'utente è portato a *scriversela da qualche parte* per poterla rileggere quando se la dimentica. L'*algoritmo standard* utilizzato genera *delle parole* formate da *sillabe pronunciabili* e le *concatena* formando una *parola unica* .

Altre tecniche *consistono* invece *nel valutare* la *password scelta*

- *Reactive password checking* : il sistema esegue *periodicamente su stesso* un *password cracker* (di solito si usa *John the ripper*) con lo scopo di trovare *password indovinabili*. Ogni volta che ne trova una *la cancella e avvisa l'utente*. Il problema di questo approccio è che *per essere eseguito con efficacia* richiede uno *grande sforzo di risorse* in quanto un *vero password cracker* sfrutta *tutta la CPU* che ha a disposizione, a volte anche per giornate intere. Inoltre una *password indovinabile* resta *vulnerabile* fino a quando *non viene individuata* dal sistema
- *Proactive password checking* : al momento della *scelta della password*, il sistema controlla se è una *password ammissibile* (se è una password forte) altrimenti *la rigetta e chiede all'utente* di sceglierne un'altra. La *filosofia* che sta alla base consiste sulla convinzione che con le *sufficienti indicazioni* , un utente sia in grado di scegliere una password che sia *in grado di memorizzare e abbastanza complessa* da non essere *catturata* da un semplice attacco basato su *dizionario*. Il segreto è il *giusto compromesso* tra *accettabilità* e *complessità* della password : se un sistema *rifiuta troppe password*, l'utente potrebbe *giudicare troppo difficile* sceglierne una accettabile, mentre se il *controllo delle password* si basa su un *semplice algoritmo*, questo consente ai *password cracker* di adattarsi e *raffinare le tecniche* che utilizzano per *indovinare la password*, superando l'ostacolo.

Restando all'interno del *Proactive password checking*, un' *approccio* è quello di *applicare delle regole* che , anche se *non sufficienti ad evitare* i password crackers. Ad esempio :

- tutte le password devono essere *composte da almeno 8 caratteri*
- nei *primi 8 caratteri*, deve essere presente almeno una *lettere maiuscola*, una *minuscola*, un *numero* e un *simbolo di punteggiatura*

Questo approccio può essere *abbinato ai consigli* per l'utente, ed è un *approccio più efficace* rispetto alla semplice *educazione*. Il processo può essere *automatizzato* utilizzando un *proactive password crackers* che applica una serie di regole sulle password scelte dagli utenti, ed è configurabile direttamente dall'amministratore del sistema.

Un altro *approccio* consiste nel *creare un dizionario* che contiene *tutte le cattive password* . Quando un utente *sceglie una password*, il sistema *controlla* che essa *non sia presente* all'interno di tale dizionario. Questa modalità però *comporta due problemi*

- *spazio* : il *dizionario* che contiene le *password cattive*, per svolgere *efficientemente il suo compito* ne deve contenere un *numero molto vasto* e di conseguenza *richiedere molto spazio* di memorizzazione
- *tempo* : il *tempo* necessario per *cercare una password* all'interno del *dizionario* aumenta in base *alla dimensione* del dizionario stesso, inoltre bisogna *aggiungere* anche il fatto che bisogna *considerare le permutazioni* della password stessa che *vanno aggiunte* a quelle già *presenti* nel dizionario stesso

i *tokens* sono oggetti fisici *posseduti dall'utente* che vengono utilizzati nei meccanismi di *autenticazione*. I *tokens* più utilizzati sono le *carte* :

Tipo carta	Caratteristiche	Esempi
In rilievo (<i>embossed</i>)	Sul <i>fronte</i> sono stampati <i>caratteri in rilievo</i>	Le vecchie carte di credito
Banda Magnetica (<i>Magnetic Stripe</i>)	Sul <i>fronte</i> sono stampati i caratteri, mentre <i>sul retro</i> è presente la <i>banda magnetica</i>	Le nuove carte di credito, e in generale le <i>carte bancarie</i>
Memory	Hanno una <i>memoria elettronica</i> al loro interno	Le vecchie <i>schede telefoniche prepagate</i>
Intelligenti (<i>smart</i>) a contatto (<i>contact</i>) senza contatto (<i>contact less</i>)	Oltre alla <i>memoria</i> contengono anche un <i>processore</i> i <i>contatti elettrici</i> sono <i>esposti in superficie</i> l' <i>antenna radio</i> è <i>incorporato all'interno</i> della carta	Carte <i>biometriche</i>

- *Memory cards* : sono carte in grado di *memorizzare ma non processare* i dati. Le più *comuni* sono le *carte bancarie* con la *banda magnetica* applicata *sul retro* (alcune di esse hanno anche una *memoria elettronica interna*). La *banda magnetica* *può solo memorizzare* un semplice *codice di sicurezza* che viene *letto* (ed è possibile purtroppo *riprogrammare*) da un *comune lettore* di carte. Possono essere utilizzate per :
 - *accesso fisico* : ad esempio per entrare *nella stanza* di un albergo oppure per *utilizzare* gli *sportelli automatici* per fare *bancomat* (Automatic Teller Machine)
 - *Autenticazione utente* : le carte vengono *abbinate* a una *password* o ad un *personal identification number* (PIN) che permetto di effettuare *l'autenticazione*

le *memory card*, quando sono *abbinate a un Pin o una password*, garantiscono un livello di *sicurezza nettamente maggiore* rispetto alla *sola password*. In questo caso infatti un attaccante *oltre a conoscere il pin*, dovrebbe *entrare in possesso o duplicare* la *memory card*. Tuttavia esistono anche degli svantaggi :

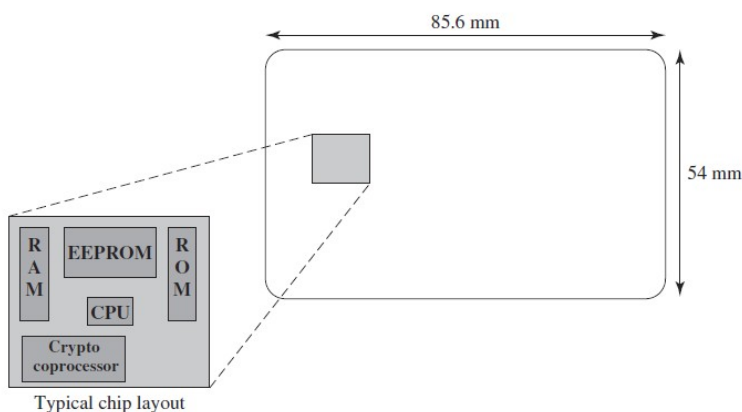
- *requires special readers* : la necessità di usare dei *lettori speciali*, aumenta il *costo* dell'utilizzo delle carte e *crea il bisogno di mantenere sicuri* tali lettori sia a livello *hardware* che *software*
- *token loss* : la *perdita* della carta da un lato *impedisce* all'utente di *ottenere l'accesso* al sistema e di conseguenza la *sostituzione della carta* richiede un *ulteriore costo*. Dall'altro se la *carta è stata rubata*, all'attaccante *basterebbe individuare il PIN* (spesso le informazioni sono *memorizzate in chiaro*) per ottenere *l'accesso non autorizzato*
- *user dissatisfaction* : se l'uso della carta per *accedere agli sportelli automatici* (*bancomat*) è una pratica *comunemente accettata*, il fatto di doverla utilizzare tutte le volte per *accedere a un computer* , per molti utenti *potrebbe risultare scomodo*.

- *smart cards* : esiste una vasta serie di dispositivi classificati come *intelligenti*. Vengono classificati in base a 3 dimensioni le quali non si escludono a vicenda :
 - *caratteristiche fisiche* : hanno un *microprocessore incorporato* , possono essere *carte bancarie, calcolatori, chiavi* o altri *oggetti portabili*
 - *interfaccia* : includono *display e keypad* per l'interazione con l'essere umano e *comunicano* con lettori/scrittori *compatibili*
 - *protocollo di autenticazione* : l'obiettivo è quello di fornire *user authentication* e in sostanza si utilizzano *tre protocolli diversi* :
 - *static* : l'utente *si autentica* al proprio token, e il token *autentica l'utente* al computer *l'autenticazione dell'utente da parte del token al computer* è simile all'operazione che viene eseguita dai *memory token*
 - *Dynamic password generator* : il token genera una *password periodica*. Questa password viene passata *al sistema* , il quale *la confronterà* con quella *fornita manualmente* dall'utente *attraverso il token stesso*. Per poter funzionare il *token e il sistema* devono essere *costantemente sincronizzati* in modo che il *computer* sia sempre a conoscenza della *password periodica generata* dal token e *possa fare authentication*
 - *Challenge-response* : il *systema* genera una *prova* e il *token* deve *rispondere correttamente*. Ad esempio il sistema *crea una stringa (challenge)* e utilizza *crittografia a chiave pubblica*. Il *token* per *autenticarsi* deve *riuscire a decrittare (response)* tale stringa utilizzando la *sua chiave privata*

la categoria delle *smart card* , degli *oggetti intelligenti* che hanno l'aspetto delle *carte di credito*, è la *più importante* per quanto riguarda *l'autenticazione dell'utente* al sistema. Hanno una *interfaccia elettronica* e possono utilizzare *qualsiasi protocollo di autenticazione* .

Una *smart card* contiene al suo interno un *intero microprocessore* composto da un *processore*, la *memoria*, e porte di *input/output*. Alcune versioni includono anche un *circuito apposta* per *velocizzare* i processi *crittografici* di codifica e decodifica dei messaggi o la *generazione* di *digital signature*. Il *chip* delle *smart card* è *incorporato* e non è visibile.

In alcune carte le *porte I/O* sono *accessibili direttamente* dai lettori in quanto i contatti elettrici *sono esposti* in superficie, mentre altre *incorporano* antenne *wireless per la comunicazione*. Una *smart card* tipicamente include *tre tipi di memoria* :



- ROM : contiene dati che *non cambiano* durante la vita della carta stessa come *il numero della carta* e il *nome* del suo possessore
- EEPROM (Electrically Erasable programmable) : contiene i *dati e le applicazioni*, come i *protocolli* che è in grado di eseguire e i *dati che possono variare* nel tempo, come nel caso delle *schede telefoniche*, il *credito residuo* per effettuare chiamate.
- RAM : contiene *dati temporanei* generati durante *l'esecuzione* delle applicazioni

la forma *più semplice* di autenticazione è l'*autenticazione locale*, dove un utente tenta di accedere a un sistema *localmente presente*, come un PC dell'ufficio o uno *sportello del bancomat* (ATM). Il caso più complesso è l'*autenticazione da remoto*, dove l'autenticazione *deve attraversare la rete* e quindi *bisogna affrontare le minacce* alla sicurezza della comunicazione, ad esempio i *tentativi di intercettare la password*, o gli attacchi di tipo *replay* dove l'attaccante *riutilizza la sequenza di autenticazione* (senza doverla decrittare ad esempio) che ha semplicemente *osservato in rete*.

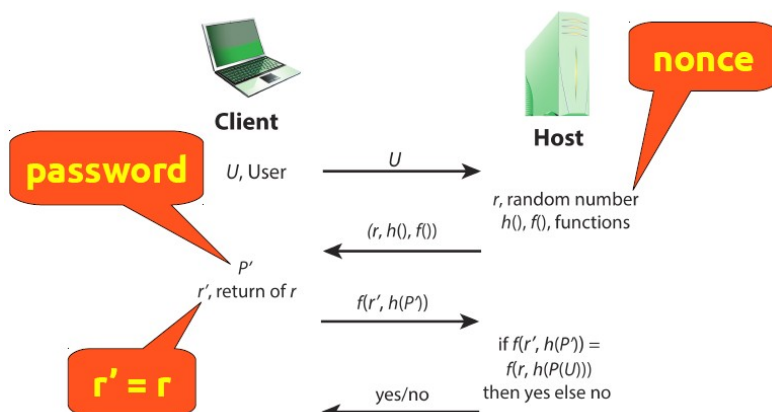
La contromisura *generalmente adottata* è quella che prevede l'utilizzo del *protocollo di autenticazione challenge-response*.

Il protocollo può essere utilizzato per *ottenere autenticazione* ad esempio *tramite password* :

- l'utente *trasmette la sua identità* all' *host remoto*
- l'host genera un *numero casuale* r chiamato *nonce* e lo *spedisce all'utente*
- sempre l'host, durante la trasmissione del *nonce*, specifica *due funzioni* $h()$ e $f()$ che verranno utilizzate *dall'utente* quando dovrà fornire la *response*
- il *challenge* è appunto la *trasmissione del nonce+funzioni* dall'host all'utente
- il *response* è la *funzione hash della password dell'utente combinata con il nonce* (numero casuale generato dall'host) tramite la *funzione* f

Client	Transmission	Host
U , user	$U \rightarrow$	
	$\leftarrow \{r, h(), f()\}$	random number $h(), f()$, functions
P' password r' , return of r	$f(r', h(P')) \rightarrow$	
	$\leftarrow$ yes/no	if $f(r', h(P')) = f(r, h(P(U)))$ then yes else no

(a) Protocol for a password



- l'host registra l'*hash code* della password P di ogni utente U registrato ($h(P(U))$) e passa all'utente la funzione di *hash* h con la quale ha creato l'*hash code*, una funzione f e un numero casuale *nonce*.
- quando arriva il *response*, ossia $f(\text{nonce}, h(P))$, l'host *ricalcola* la funzione $f(\text{nonce}, h(P(U)))$ e li *confronta*: se si *verifica il match* allora l'utente U viene *autenticato*

in questo modo ci si *protegge da numerosi attacchi* :

- il sistema *registra solo l'hash code* della password di ogni utente, in questo modo anche se un attaccante riuscisse ad *accedervi* non potrebbe in alcun modo recuperare la password in chiaro.
- l'*hash code* non viene nemmeno *trasmesso direttamente*, ma come *parametro* della funzione f , quindi se si tratta di una *buona funzione* diventa *impossibile* da intercettare durante la *trasmissione*
- il secondo parametro della funzione f , essendo un *numero casuale*, rende inefficace ogni tipo di *attacco replay*

come ogni *servizio di sicurezza*, anche l'*user authentication* e specialmente la *remote authentication* è soggetta a una *varietà di attacchi* :

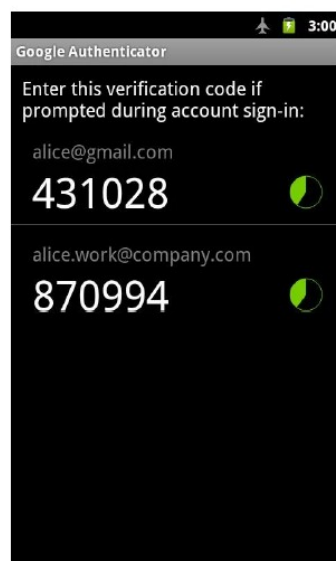
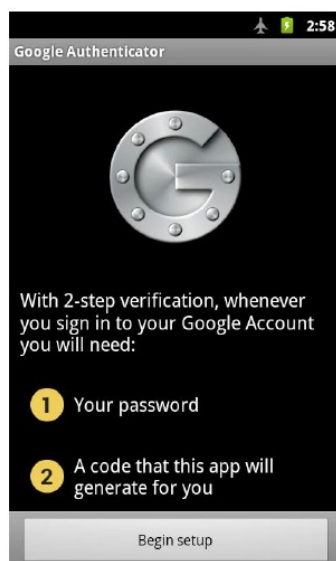
- *client attacks* : in questo tipo di attacco, l'attaccante *cerca di ottenere l'autenticazione* senza avere *accesso al sistema remoto* e senza *intervenire sul canale di comunicazione*. Sui sistemi *basati su password*, l'attaccante cerca di *indovinare la password* dell'utente *provandole tutte* fino ad avere successo. Un modo per *evitare* questo tipo di attacco è sicuramente quello di utilizzare una *password lunga e imprevedibile*. Un altro modo è quello di *limitare il numero di tentativi possibili* in un *certo periodo* e da una *determinata sorgente*. Se invece il sistema si basa *sull'utilizzo di un token*, l'attaccante oltre a *scoprire il PIN* dovrebbe anche *impossessarsi o duplicare* il token stesso.
- *Host attacks* : in questo tipo di attacco, l'attaccante cerca di *accedere al file utente* del sistema, dentro al quale sono memorizzate le *password, i token passcode e i biometric templates*. Per i tokens, un *linea difensiva* è rappresentata dall'utilizzo di *passcode di sessione*, che quindi *non vengono memorizzati* all'interno di questo file. Le *caratteristiche biometriche* sono difficili da *difendere* in quanto sono *caratteristiche fisiche* dell'utente. Una *misura di sicurezza* per le *caratteristiche statiche* è rappresentata dall'*autenticazione del device*, mentre per quelle *dinamiche* si utilizza un protocollo di sicurezza *challenge-response*
- *intercettazione* : l'attaccante tenta di *imparare la password osservando l'utente, trovandone una copia scritta da qualche parte*, e in generale tutti quegli attacchi che *prevedono la vicinanza fisica* tra utente e attaccante. Un'altra forma di intercettazione è il *keystroke logging* dove un sistema hardware o software *malevolo* viene installato in modo che l'attaccante *sia in grado di catturare tutto quello che viene digitato* dall'utente e analizzarlo. Per quanto riguarda i *dispositivi biometrici* l'attaccante tenta di *copiare o imitare i parametri biometrici* utilizzati per l'autenticazione. Chiaramente per le *caratteristiche dinamiche* è molto difficile, mentre per quelle *statiche* è sufficiente aggiungere anche l'*autenticazione del device*. In generale un sistema che adotta *più protocolli di sicurezza* e si basa su *più fattori* (es password + biometrica) avrà un grado di resistenza maggiore
- *replay* : un attaccante *replica la risposta dell'utente* che ha *precedentemente catturato*. La contromisura più *adottata* è l'utlizzo del *protocollo challenge-response* che utilizzando un *numero casuale* rende vano qualsiasi attacco di questo tipo
- *trojan horse* : un applicazione o un dispositivo fisico *si maschera da applicazione autentica* con l'obiettivo di *catturare la password* dell'utente. In questo modo l'attaccante con le *informazioni catturate* può essere in grado di *fingersi l'utente legittimo*.
- *denial of service* : questo tipo di attacco cerca di *disabilitare* il sistema di autenticazione *inondandolo di richieste*. Un attacco più *specifico e mirato a un determinato utente* può consistere nel *bloccargli l'accesso* per un certo periodo di tempo a causa *dei troppi tentativi effettuati e falliti*. Un modo per *evitare questo tipo* di attacchi è adottare un protocollo di autenticazione *multifattore*, ad esempio introducendo anche l'utilizzo di un token, l'attaccante dovrebbe prima *venirne in possesso*.

Il sistema di *verifica* basato su un solo fattore è risultato *fragile* : l'idea infatti è quella di rendere la *user authentication* più sicura e quindi di basarsi su *protocolli multi fattore*.

Normalmente, nei moderni sistemi, si fa affidamento su qualcosa che uno *sa* (es. *Nome utente e password*) abbinato a qualcosa che uno *ha* (es. Un token) per *verificare l'identità* di un utente che sta *richiedendo l'accesso* al sistema.

Il *token* più utilizzato, visti anche i tempi, è lo *smartphone* quindi ad esempio una possibile *implementazione* di un sistema a *due fattori*:

- *primo fattore* : nome utente e password
- *secondo fattore* : un PIN *dinamico* generato da un applicazione *installata sul proprio smartphone*



l'idea alla base è quella di *smaterializzare* il sistema, eliminando ad esempio *le smartcard*, e sostituendole ad esempio con *applicazioni installate* sui cellulari, che una volta inserito *nome utente e password* (primo fattore) generano una OKP (*one kind password*) che *per un periodo limitato di tempo* potrà essere utilizzata per *accedere al sistema*.

La compatibilità dei *sistemi che generano numeri casuali* dipenderà sostanzialmente *dall'algoritmo utilizzato*. Il sistema basato sulla OKP però funziona solo se il *dispositivo* in possesso dell'utente (ad esempio *l'applicazione su smartphone*) e il server riescono *in un preciso momento*, *generare la stessa sequenza casuale*.

Per riuscire a generare la *stessa sequenza casuale* :

- l'applicazione e il sistema devono essere *sincronizzati* , quindi condividere qualcosa inteso come *concetto di tempo*. Se non sono sincronizzati, due sistemi non *produrranno mai lo stesso stream* e quindi *sarà impossibile fare match*. Il NTP (*network time protocol*) è un protocollo che viene utilizzato per *collegare i sistemi* a particolari computer che si collegano a un sistema molto *sofisticato* di segnale orario
- al momento dell'*installazione dell'applicazione* (ad esempio sullo smartphone) viene *generato il seme* (anche chiamato *segreto*) . Dato un *seme* si ottiene infatti la *stessa sequenza numerica* e quindi una *sua copia* viene *salvata anche sul server*.
- Ogni volta che l'applicazione *genera una sequenza* , richiede *username e password* (primo fattore) e una volta *verificati*, *spedisce* la richiesta al server (con il quale condivide il *seme*) e se le sequenze generate *coincidono* (secondo fattore) allora *ottengo authentication*

anche in questo caso *non si è esenti* da possibili attacchi :

- l'attacco *più semplice* è quello che prevede l'*accesso fisico* al token, nel caso si tratti ad esempio di una *chiave sigillata*, e *manomettere* il sistema di *sincronizzazione*. In questo modo il dispositivo *non sarà più in grado* di generare in nessun caso *la stessa sequenza* del server e quindi anche *il proprietario* non otterrà mai *authentication*
- se si tratta di un *applicazione*, anche in questo caso si possono effettuare attacchi *basati sul tempo*, in modo che *applicazione e server* misurino un *tempo diverso*. Oppure cercare di *individuare la OKP* ed accedere *durante la finestra temporale* per la quale la password temporanea è ancora valida

in generale la *two step authentication* ha migliorato nettamente il livello di sicurezza. Esistono tuttavia ancora degli aspetti migliorabili. Ad esempio se io dovessi *cambiare smartphone* non posso, per motivi di sicurezza, copiare *tutto in una volta* tutti i dati di applicazioni che utilizzano questo meccanismo ma dovrei *spostarli uno alla volta*.

Esistono altri *punti critici* :

- il più *evidente* è il *seme* o *segreto*. Tralasciando il *server*, se il *segreto* è salvato su *smartphone* , qualsiasi altra applicazione li installata che *ha accesso* all'app che lo genera è *in grado* di rubarlo. Questo non accade perché di solito le applicazioni su smartphone hanno il *vincolo* che le *fa vivere solo all'interno del loro spazio*, ma se ad esempio l'applicazione che genera il segreto *fosse salvata sulla memoria esterna*, chiunque fosse in grado di accedere a tale memoria avrebbe di conseguenza accesso a tutte le informazioni li sopra salvate
- di solito la *finestra temporale* di validità di una *sequenza* cambia da *smartphone a server*. Infatti sul *server* la *finestra temporale* è più lunga rispetto a quella *prevista* dallo smartphone e di conseguenza *esisterà un periodo limitato di tempo* per il quale esisteranno *due sequenze valide contemporaneamente* a causa ad esempio di problemi di rete.

Anche se si tratta di un sistema di *autenticazione a due fattori* bisogna prestare attenzione al fatto che *entrambi i fattori* vengono trasmessi *sullo stesso canale di comunicazione* (ad esempio si utilizza *lo stesso browser* , *la stessa tastiera* e *lo stesso sistema operativo*). Se uno di questi *canali viene compromesso* (ad esempio viene installato un *keystroke logger*), un eventuale attaccante avrebbe *accesso ad entrambi i fattori* per tutto il tempo in cui il canale *rimane compromesso*

CONTROLLO DEGLI ACCESSI

è un sistema che *previene* l'uso *non autorizzato* di una risorsa, e che include anche la *prevenzione* dell'uso *sbagliato* (non consentito) di una risorse di cui si è *autorizzati* ad usare.

ITU-T

l'*access control* può essere visto come l'*elemento centrale* della *computer security*. Gli obiettivi *principali* della computer security sono :

- *impedire l'accesso* alle risorse ad *utenti non autorizzati*
- *impedire* che un *utente autorizzato* utilizzi una risorsa *in modo sbagliato*
- *consenta* ad un *utente autorizzato* di utilizzare una risorsa *in modo legittimo*

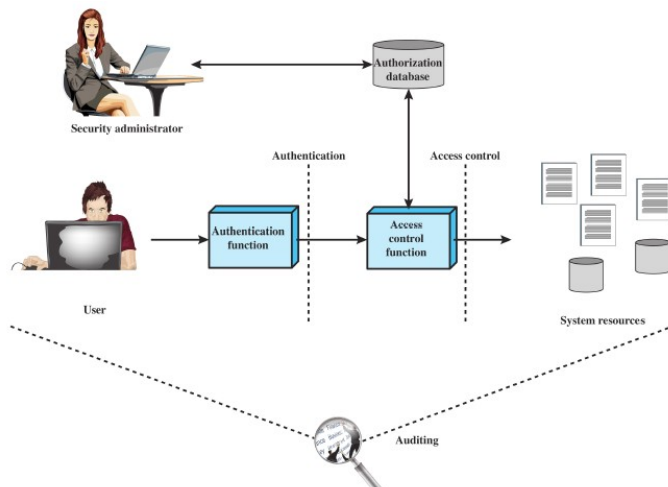
in un *senso molto ampio*, tutta la *computer security* ruota attorno al *controllo degli accessi*

la *computer security* è un insieme di *misure* che *implementano* e *assicurano* un *servizi* di *sicurezza* all'interno di un sistema e *in particolare* assicurano un *servizio di controllo degli accessi*

RFC 2828

l'*access control* implementa una *politica di sicurezza* che specifica *chi* o *cosa* può *avere accesso* una specifica risorsa di sistema e il *tipo di accesso* permesso e in *quali circostanze*.

In uno scenario reale, l'*access control* si *relaziona* con altre *funzioni di sicurezza* (es. *firewall*)

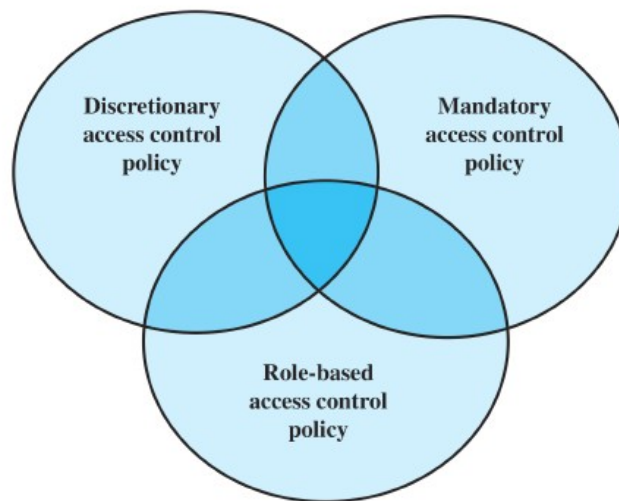


- *Authentication* : è la *funzione* che *verifica* le *credenziali* fornite da un *entità* siano *valide* (ossia *che è chi dice di essere*)
- *Authorization* : è la *funzione* rilascia i *permessi* all'*entità autenticata* che *accede* a una risorsa di sistema.
- *Audit* : è una *funzione indipendente* che *revisiona ed esamina* le attività del sistema in modo da *testare l'adeguatezza* dei sistemi di controllo

il *meccansimo* di *controllo degli accessi* deve *mediare* tra l'*utente* e la *risorsa a cui vuole accedere* :

- per *prima cosa* deve *autenticare* l'*entità che richiede l'accesso* alla risorsa
- una volta *autenticata l'entità*, deve *verificare* che essa *abbia il permesso* di accedere alla *risorsa richiesta*
 - per fare questo l'*amministratore* consulta l'*authorization database* che *specifica* , per ogni utente e per ogni risorsa, il *tipo di accesso* consentito .
- La *funzione di controllo* quindi *determina se concedere l'accesso*
- la *funzione di auditin* *monitora e registra* tutti gli *accessi alle risorse* effettuati da ogni *utente*
 - *assicura* che *ci sia conformità* con le *politiche di sicurezza* e le *procedure operazionali*
 - *individua falle* di *sicurezza*
 - *suggerisce* *cambiamenti*

una *politica* di controllo sugli accessi detta il *tipo di accesso* permesso alla risorsa, *in quale circostanza* e *a chi* . Generalmente esistono *tre categorie* (un meccanismo di controllo *può implementarne più di una* in quanto *non si escludono a vicenda*) :



- *Discretionary access control (DAC)* : il controllo si basa sull'*identità* dell'entità che ha effettuato la richiesta e sulle *regole di accesso* (autorizzazioni) che gli sono permesse. È un tipo di controllo *discrezionale* perché un'entità che ha il permesso di accesso può di sua volontà, permettere ad un'altra entità di accedere alla risorsa
- *Mandatory access control (MAC)* : il controllo si basa sulla *comparazione* tra l'*etichetta* di sicurezza (*security labels*) che indica il *livello di sensibilità o criticità* di una risorsa e la *security clearances* che indica il *livello di criticità della risorsa* a cui l'utente può accedere. L'utente quindi (di solito) potrà accedere a tutte le risorse *etichettate a un livello pari o inferiore* alla sua *security clearances*. È un tipo di controllo "Di Mandato" perché un'entità che può accedere a una risorsa, non può nemmeno se vuole, permettere a un'altra entità (con una *security clearances* non sufficiente) di accedervi
- *Role-Based access control (RBAC)* : il controllo si basa sul *ruolo* che un utente ha all'interno dei sistemi e dalle *regole di accesso* (autorizzazioni) che vengono concesse a chi accede con quello specifico ruolo

di solito un sistema di controllo degli accessi deve poter supportare i seguenti concetti/caratteristiche :

- *reliable input* : il meccanismo di controllo assume che l'utente sia *autenticato*, per cui affinché funzioni ha bisogno di un *meccanismo di autenticazione* e in generale che tutti i dati in input siano affidabili .
- *Support for fine and coarse specification* : il controllo deve supportare la possibilità di regolare gli accessi a livello del singolo campo di un record all'interno di un file. Ogni accesso deve essere controllato come se si trattasse di una *sequenza di richieste*. L'amministratore deve comunque essere in grado di scegliere il *livello di dettaglio* per alcune classi di risorse
- *least privilege* : ogni sistema deve concedere il *privilegio minimo* che consenta all'utente di svolgere il proprio compito. In questo modo si limitano i possibili danni o accessi non autorizzati
- *separation of duty* : in sostanza si tratta della *separazione dei poteri*. Questa pratica consiste nel suddividere i poteri di autorizzazione per una certa funzione tra vari individui (es. Amministratori). In questo modo per poter eseguire una certa funzione, un utente deve ricevere l'ok da più amministratori, i quali possiedono una parte della chiave, che serve per autenticarsi e quindi dare il via alla funzione stessa.

- *open and closed policies* : in una politica definita *aperta*, devono essere *specificati quali accessi sono proibiti*, altrimenti tutti gli altri *vengono considerati permessi* (ad esempio posso *specificare* che il browser *non può accedere* a una determinata cartella). In una politica definita *chiusa*, devono essere *specificati quali accessi sono permessi*, altrimenti tutti gli altri *vengono considerati proibiti* (ad esempio posso *specificare* che il browser *può accedere* a una determinata cartella, esempio quella dei *download*)
- *policy combination and conflict resolution* : su una *certa classe* di risorse è possibile *applicare più politiche*. In questo bisogna fare attenzione *che non entrino in conflitto* (es. Una *permette l'accesso* e l'altra *lo vieta*). In caso di conflitto quindi *stabilire una procedura* che si incarichi di risolverlo.
- *Administrative policies* : sono *politiche* che specificano *chi può modificare, aggiungere o cancellare i permessi* di accesso
- *dual control* : un'azione *potrebbe richiedere* che *più individui lavorino in tandem*.

Gli *elementi* che stanno alla base del sistema *di controllo degli accessi* sono :

- *soggetto* : è un *entità* in grado di *accedere a un oggetto*. È *responsabile* di tutte le *azioni che intraprende* e l'audit *può memorizzare* tutte le *azioni rilevanti* ai fini della *sicurezza che il soggetto ha effettuato su un oggetto*. Esistono *tre classi* di soggetti , ognuna delle quali *ha permessi differenti* :
 - *Owner* : è il *creatore (proprietario) della risorsa* (es. *File*). il proprietario delle *risorse di sistema* è l'*amministratore*
 - *Group* : oltre ai *privilegi assegnati al proprietario*, certi *permessi possono essere assegnati a un gruppo*, ed ogni *utente che ne fa parte* può usufruirne. Un *utente può appartenere a più gruppi*.
 - *World* : il *resto degli utenti che ha accesso al sistema ma non appartiene* nè al gruppo a cui sono *stati assegnati determinati privilegi* ne tantomeno è *proprietario*, riceve *i privilegi minimi* per accedere alla risorsa.
- *Oggetto* : è la *risorsa* il cui *accesso è controllato*. Un oggetto in generale è un *entità* usata per *contenere e/o ricevere informazioni*. Il *numero e il tipo degli oggetti il cui accesso è protetto* da un sistema di *controllo degli accessi* dipende dall'*ambiente* in cui opera, e il *compromesso* che si vuole ottenere tra *sicurezza e facilità d'uso*.
- *permessi di accesso (access right)* : descrive *il modo* in cui in *soggetto può accedere a un oggetto* :
 - *Read* : l'utente con questo permesso è in grado di *visualizzare le informazioni* relative a quella risorsa. Il *permesso di lettura* include la possibilità di *copiare o stampare* la risorsa (se ad esempio si tratta di un *file di testo*)
 - *Write* : con i *permessi di scrittura* l'utente può *aggiungere, modificare o cancellare* i dati appartenenti a una risorsa. Il permesso di *scrittura* include quello di *lettura*
 - *Execute* : un *utente può eseguire* un programma (*risorsa*) specifico
 - *Delete* : un *utente può cancellare una risorsa* (ad esempio un *file*)
 - *Create* : un *utente può creare* una nuova risorsa
 - *Search* : un *utente è in grado* , ad esempio, di *ottenere la lista* dei file all'interno di una *directory*

uno schema del *sistema degli accessi discrezionale* è un meccanismo in cui un *entità* che ha *i permessi per accedere a una risorsa* è in grado, di *sua volontà*, di *permettere a un'altra entità* (alla quale il sistema *non aveva concesso i permessi*) di *accedere a quella risorsa*.

Viene implementato da una *matrice degli accessi* :

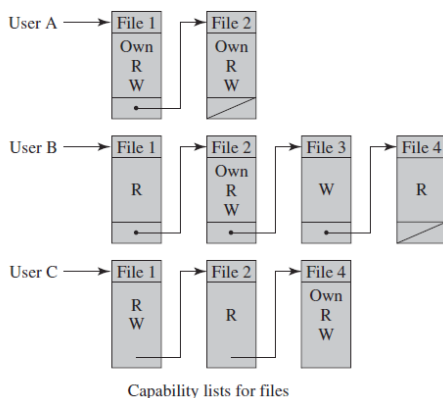
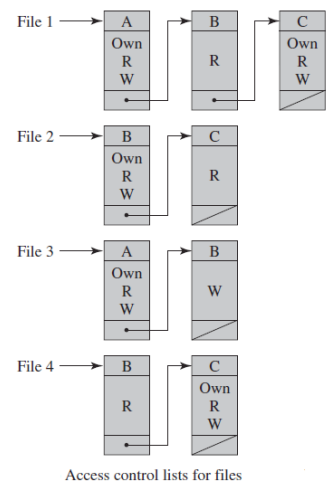
		OBJECTS			
		File 1	File 2	File 3	File 4
SUBJECTS	User A	Own Read Write		Own Read Write	
	User B	Read	Own Read Write	Write	Read
	User C	Read Write	Read		Own Read Write

- *prima dimensione* : contiene gli *identificatori dei soggetti* (soggetti) che *possono richiedere* l'accesso a una risorsa
- *seconda dimensione* : contiene la *lista delle risorse* (oggetti) a cui *si può accedere*
- *incrocio delle dimensioni* : indica il *tipo di permesso concesso* ad un utente per quella *particolare risorsa*

la *matrice degli accessi* normalmente è una *matrice sparsa* per cui viene *decomposta* in uno dei due seguenti modi :

- **Access Control List (per colonne)**: per *ogni oggetto* viene creata una *lista di utenti* e i loro *permessi di accesso*. Inoltre contiene una *entry pubblica* che comprende i restanti utenti *che non fanno parte della lista* a quali vengono assegnati *permessi standard* (di solito si tratta di permessi di *sola lettura*).

Questa implementazione è *conveniente* nel caso si desidera sapere *quale soggetto ha un tipo di permesso* dato un *oggetto*. Ogni Access Control List fornisce *tutte le informazioni* su una data risorsa. Non è conveniente invece se *dato un utente* devo determinare i suoi *privilegi* in quanto dovrei *scorrere tutte le ACL*.



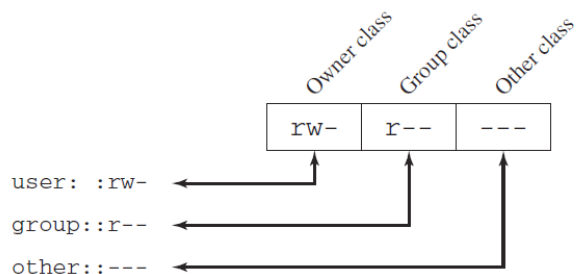
- **Capability tickets (per righe)** : per *ogni utente* vengono specificate le *risorse a cui ha accesso* e per ognuna di esse i *permessi concessi*. A causa della *dispersione* degli utenti in tutto il sistema, questo metodo rispetto al precedente *presenta grossi problemi di sicurezza*. Per ogni *ticket* ne va protetta e garantita *l'integrità* : il ticket *non può essere falsificato*. Per ottenere questo scopo i *ticket* vengono memorizzati in un area del sistema operativo *non accessibile agli utenti*, oppure viene *aggiunto un token* il cui valore verrà *verificato ogni volta* che viene richiesto l'accesso a una *risorsa importante*. L'utilizzo di un token, ad esempio *random password* o *cryptographic message authentication code* , è molto utilizzato in ambito *distribuito* dove *non si garantisce* la sicurezza del contenuto del singolo token. Questa implementazione è *conveniente* quando si vuole determinare , dato un utente, *l'insieme dei suoi privilegi*. Mentre è molto scomodo se il mio obiettivo è determinare *la lista degli utenti* che hanno un *tipo di privilegio* data una risorsa.

Tutti i tipi di file UNIX sono amministrati dal sistema operativo tramite *inodes* (index node). Un *inode* è una *struttura di controllo* che contiene le *informazioni chiave* necessarie al sistema operativo riguardo a un *file particolare*. Più *nomi dei file* possono essere associati allo stesso inode, ma un *inode attivo* è associato ad *esattamente un file*, e ogni *file* è *controllato da esattamente un inode*, al cui interno vengono *memorizzati gli attributi* , i *permessi concessi* che lo riguardano e altre *informazioni di controllo*.

Su disco invece è presente una *inode table* o *inode list* che contiene *tutti gli inode* relativi a *tutti i file* presenti nel *file system*. Quando un file viene *aperto*, il suo *inode* viene *spostato in memoria centrale* e memorizzato sulla *inode table*. Una *directory* è un semplice file che contiene una *lista di nome dei file* con i *relativi puntatori agli inode*.

Ad ogni utente UNIX viene assegnato un *identificativo univoco* (User ID) ed è membro di un *gruppo primario* ed eventualmente *altri gruppi* ognuno identificato da un Group ID. Ogni volta che un *file viene creato*, viene *etichettato* con l'User ID del suo *owner* (creatore) e inoltre di *default* viene anche *assegnato* al suo *gruppo primario* (ed eventualmente ad *altri gruppi* a cui il proprietario appartiene, se ha il SetGID *attivato*). Ad ogni file poi vengono associati 12 bit di *protezione*. L'Owner ID , il Group ID e i 12 bit vengono memorizzati *dentro l'inode*.

- 9 dei 12 bit di *protezione* sono utilizzati per *specificare* quali sono i *permessi concessi* al proprietario (*read, write, execute*)



(a) Traditional UNIX approach (minimal access control list)

*l'owner può leggere, scrivere ma non eseguire
chi appartiene al gruppo può solo leggere
i restanti (world) non possono far nulla*

nel caso il file sia una *directory*, i permessi di *lettura e scrittura* consentono di *creare, cancellare e rinominare* i file all'interno della cartella, mentre *execute* permette di *scendere lungo la directory* oppure cercare un file *in base al nome*.

- 2 bit invece servono per *settare* i *set User ID* (SetUID) e i *group User ID* (SetGID).

Se questi bit sono *attivi* su un *file eseguibile*, allora quando l'*utente* li esegue il sistema *concede* a tale utente *temporaneamente i privilegi* che spetterebbero al suo *owner* (SetUID *attivo*) oppure al suo *gruppo* (SetGID *attivo*) . Questo cambiamento ha effetto *solamente durante l'esecuzione del programma* e permette di *usufruire di privilegi* che normalmente l'utente che lo sta eseguendo non avrebbe. In alternativa se invece di un *file eseguibile* lo si applica a una *directory*, il SetGID indica che il *nuovo file creato eredita* il gruppo della *directory*.

- L'ultimo bit è lo *sticky bit* (bit appiccicoso). Quando viene *attivato su un file*, indica che il sistema *dovrebbe mantenere* il contenuto del file *in memoria* mentre viene eseguito. Quando invece viene *attivato su una directory*, indica che *soltanto il proprietario* dei file al suo interno *può rinominarli, spostarli o cancellarli* (utile in caso di *cartella condivisa*).

Un particolare User ID può essere denominato *superuser* (root) :

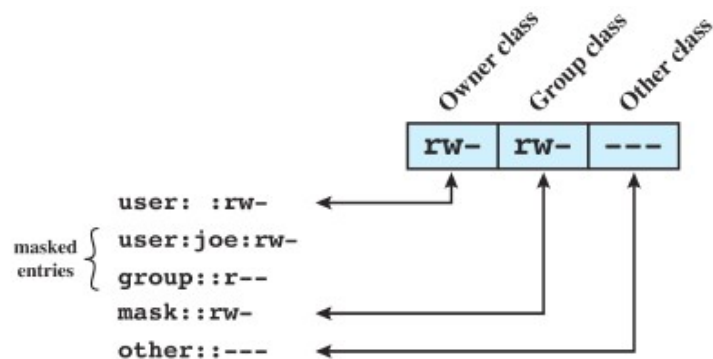
- è *esente* dalle *normali restrizioni* adottate dal *controllo degli accessi*
- ha *accesso ovunque* all'interno del sistema

nei *moderni sistemi UNIX* il *sistema degli accessi discrezionale* è implementato tramite *access control list* (la *matrice degli accessi* viene quindi *frammentata per colonne*)

FreeBSD permette all'amministratore di assegnare a un file una lista (Access control list) di User ID e Group ID tramite il comando *setfacl*. La possibilità di associare a un file un certo numero di utenti e gruppi, ognuno con i propri 3 bit di protezione (owner, group, world e lettura, scrittura, esecuzione) offre un meccanismo di assegnazione degli accessi molto flessibile. Un file se non ha una Access control list viene protetto dal meccanismo degli accessi tradizionale previsto da Unix. I FreeBSD file (che includono gli access control list) includono un bit aggiuntivo che indica appunto il fatto che esso possiede tale lista.

L'implementazione FreeBSD usa la seguente strategia :

- i permessi concessi al proprietario e a tutti gli altri utenti di un FreeBSD file sono gli stessi (lettura/scrittura/esecuzione) che verrebbero assegnati da un meccanismo di access control list tradizionale
- il campo *group* funge da maschera e specifica il massimo set di permessi che può essere assegnato ad un utente o ad un gruppo (owner escluso) appartenente all'access control list di quel file
- per ogni utente o gruppo che appartiene all'access control list vengono controllati i permessi assegnati e quelli che non sono presenti nella maschera vengono disabilitati



quando un processo richiede di accedere a un oggetto :

1. il sistema sceglie l'entry (owner/namer user/namer group , others) dell' access control list più appropriata
2. una volta scelta l'entry, controlla se ha i permessi sufficienti per accedere all'oggetto richiesto

un processo può far parte di più gruppi :

- uno o più gruppi può risultare appropriato per la richiesta fatta
 - ne viene scelto comunque uno solo
- se nessuno risulta avere i permessi necessari allora l'accesso viene negato

il *mandatory access control* è una *meccanismo* che associa ad ogni *file* una *label* e ad ogni *entità* una *clearance*



un soggetto può leggere un oggetto solo se lo domina :

- un soggetto si dice che *domina* un oggetto se la *clearance* del soggetto è di livello superiore rispetto a quella dell'oggetto
 - il *rank* del soggetto deve essere superiore rispetto al *rank* dell'oggetto
 - il *compartimento* dell'oggetto deve essere un *sottocompartimento* di quello del soggetto

l'informazione classificata può essere associata a uno o più progetti che la suddividono in *compartimenti* . L'informazione quindi sarà composta da un :

- *rank* : indica il livello di sicurezza richiesto per accedere all'informazione
- *compartimento* : indica il progetto a cui *bisogna far parte* per poter accedere

Esempio :

data l'informazione classificata : < secret ; Singapore >

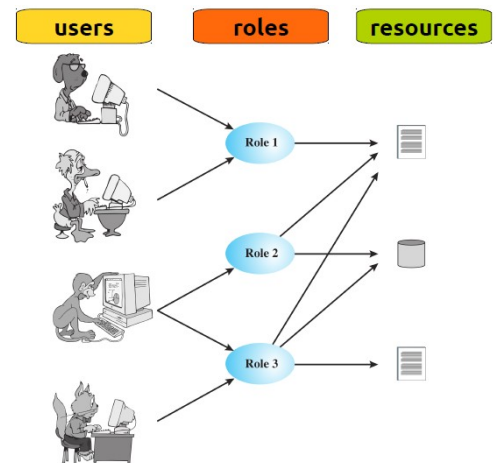
data la *clearance*, i seguenti soggetti :

- < top secret ; {Singapore } >
 - *rank* soggetto > *rank* oggetto
 - *compartiment* soggetto = *compartiment* oggetto
 - può leggere
- < secret ; {crypto } >
 - *rank* soggetto > *rank* oggetto
 - *compartiment* soggetto != *compartiment* oggetto
 - non può leggere
- < confidential ; {Singapore, crypto } >
 - *rank* soggetto > *rank* oggetto
 - non può leggere
 - *compartiment* oggetto è un *sub-compartiment* di soggetto



il *role-based access control* invece si basa sul *ruolo* che un utente utilizza al posto della propria identità per accedere a una risorsa. Il sistema quindi *assegna i privilegi ad un ruolo* invece che a un singolo utente. Ad ogni utente quindi, in base alle proprie responsabilità, verrà assegnato un ruolo in maniera statica oppure dinamica

- l'insieme degli utenti nel tempo cambia frequentemente in certi ambienti, quindi l'assegnamento di uno o più ruoli a un determinato utente avviene dinamicamente
- ogni ruolo ha dei permessi specifici per una o più risorse. L'insieme delle risorse e i permessi di accesso a loro associate non cambiano frequentemente, quindi i permessi concessi a un ruolo vengono assegnati staticamente



tipicamente il numero di utenti è maggiore rispetto a quello dei ruoli e il numero di risorse è maggiore rispetto a quello dei ruoli. L'accesso a una risorsa viene implementato da due tabelle

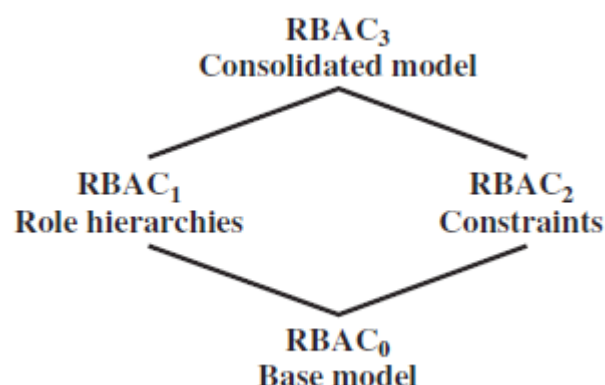
		roles			
		R ₁	R ₂	...	R _n
users	U ₁	×			
	U ₂	×			
	U ₃		×		×
	U ₄				×
	U ₅				×
	U ₆				×
	U _m	×			

- per ogni utente vengono sopecificati i ruoli che può assumere
- per ogni ruolo vengono sopecificati i permessi per ogni risorsa

		objects								
		R ₁	R ₂	R _n	F ₁	F ₁	P ₁	P ₂	D ₁	D ₂
roles	R ₁	control	owner	owner control	read *	read owner	wakeup	wakeup	seek	owner
	R ₂		control		write *	execute			owner	seek *
	...									
	R _n			control		write	stop			

ogni ruolo dovrebbe seguire la regola del *privilegio minimo*. Un utente a cui viene assegnato un ruolo deve essere in grado di eseguire solo quello previsto per chi utilizza quel ruolo.

Per rafforzare la standardizzazione del *role-based access control* sono stati definiti quattro modelli che formano un unica famiglia :



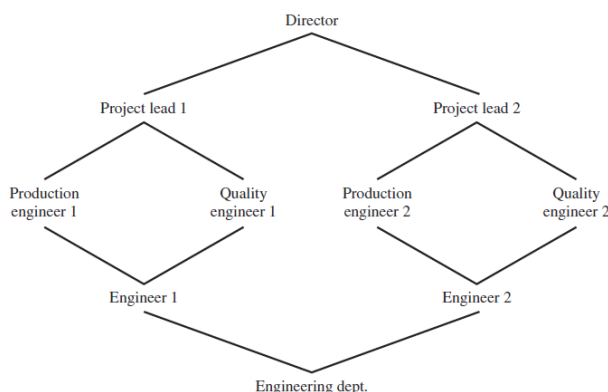
Modello Base (RBAC₀) : è il modello che *non prevede* ne la *gerarchia* tra ruoli e nemmeno dei *vincoli* . Il sistema è quindi *composto* da :

- *user* : è un *entità* che *ha accesso* al sistema alla quale *viene associato* un *userID*
- *role* : è uil *nome* associato a una *funzione* all'interno *dell'organizzazione* che *controlla* il sistema. *Associato* a un ruolo c'è la *descrizione* dell'*autorità* e delle *responsabilità* conferite *al ruolo* e ad ogni *utente* che lo assumerà
 - un *utente può avere più ruoli* e un *ruolo può essere assegnato a più utenti*
- *permission* : è un *particolare modo di accesso* a una o più risorse, *approvato* da chi *gestisce* il sistema
- *session* : è il *mapping* tra un *utente* e i *ruoli attivi* che gli sono *stati assegnati*
 - una *sessione definisce una relazione temporanea* tra l'*utente* e *uno o più ruoli* che esso può assumere : un *utente stabilisce una sessione* con assumendo i *ruoli necessari* per compiere un *determinato compito*.

La *possibilità* per un *utente di avere più ruoli*, di *assegnare un ruolo a più utenti*, di *garantire a un ruolo più permessi*, e di *assegnare lo stesso permesso a più ruoli* consente una *maggior flessibilità e granularità* rispetto al *discretionary access control*.

- Potendo *controllare il tipo di accesso* si evita di *assegnare più privilegi* di quelli *necessari per eseguire un determinato task* (Es. A un *certo utente* è possibile *consentire la lettura* di un *file* senza *consentirgli la scrittura*)

Modello Gerarchio (RBAC₁) : questo modello *prevede una struttura gerarchica* dei ruoli. Un ruolo (job function) *subordinato* potrebbe avere un *sottoinsieme* dei *privilegi* assegnati al ruolo *superiore*. Si utilizza il concetto di *ereditarietà* per consentire a un ruolo di *includere implicitamente* i privilegi assegnati a un suo ruolo *subordinato*



un ruolo può ereditare i privilegi da più ruoli subordinati

- *project lend 1* eredita i ruoli di *production engineer* e *quality engineer*

più ruoli possono ereditare dallo stesso ruolo subordinato

- *production engineer* e *quality engineer* ereditano entrambi da *engineer*

Modello *Vincolante* (RBAC₂) : un *vincolo* è una *relazione* tra un *ruolo* e le *condizioni* poste su di esso. In questo modo è *possibile adattare il modello* alle *specifiche amministrative* o alle *politiche di sicurezza* dell'organizzazione che lo implementa.

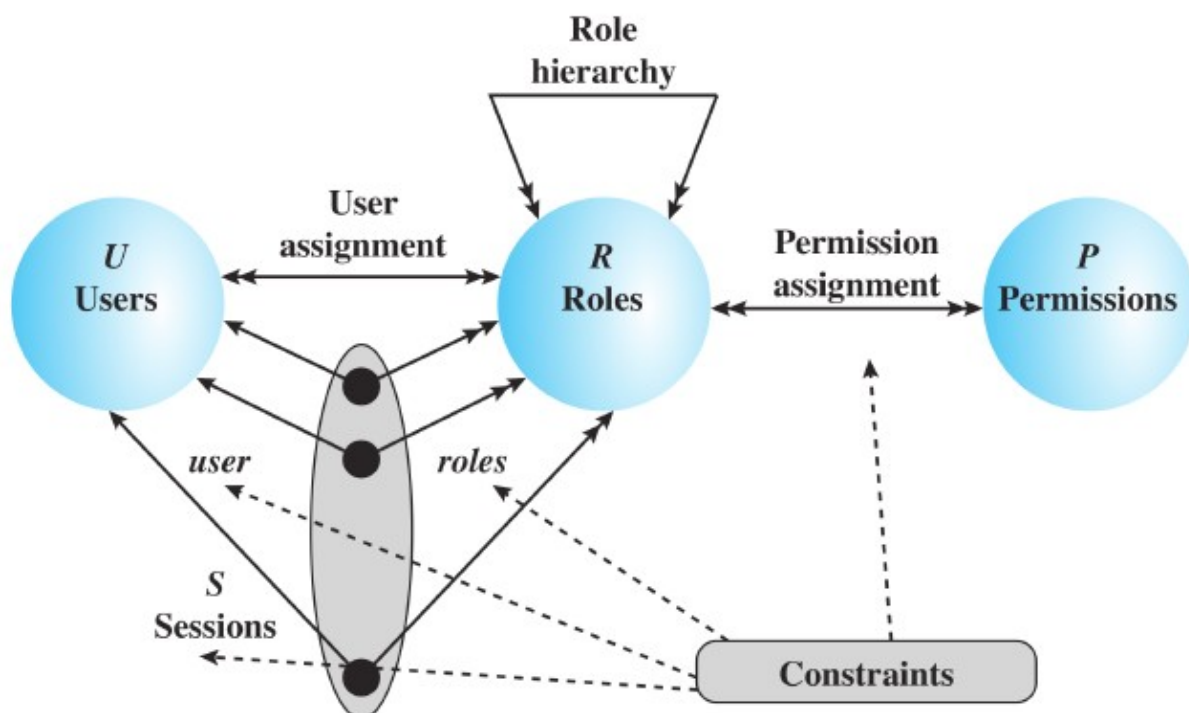
I *vincoli* previsti sono i seguenti :

- *mutual exclusive role* : all'interno di una *sessione* ad utente viene assegnato un solo ruolo. Questo vincolo supporta la *separazione dei poteri* all'interno dell'organizzazione e può essere *rafforzato* aggiungendo l'*assegnamento in mutua esclusione* dei *privilegi*. In questo modo :
 - ad un utente può essere assegnato solo un ruolo (*ruoli in mutua esclusione*)
 - un privilegio può essere concesso solo da un ruolo (*privilegi in mutua esclusione*)
 se a due utenti vengono assegnati due ruoli diversi, allora non avranno dei *privilegi in comune* (no overlapping). Questi vincoli *aumentano la difficoltà* di *collusione* tra utenti con *abilità* o *ruoli differenti* che *tentano di aggirare* le politiche di sicurezza
- *cardinality* : limita il numero massimo di utenti che possono assumere un determinato ruolo e il numero massimo di ruoli che possono concedere un particolare privilegio in modo da *mitigare il rischio* che potrebbe essere causato da un *errato utilizzo* di un *privilegio sensibile*
- *pre-requisite role* : un utente può assumere un determinato ruolo solo se è già in possesso di un altro specifico ruolo. In un *contesto gerarchico*, ad esempio, un utente può assumere un ruolo *senior* solo se gli è già stato assegnato il ruolo di *livello immediatamente inferiore*. Questo vincolo implementa il concetto del *privilegio minimo*

nel 2001 il Nist propose un modello di RBAC (RBAC₃). La *principale innovazione* è rappresentata dall'introduzione della *RBAC system and administrative functional specification* : un *benchmark* che indica le *funzionalità* che dovrebbero essere fornite agli utenti e l'*interfaccia* di programmazione generale da adottare. Questa *specifica* raggruppa le *funzioni* in tre categorie :

- *Administrative* : sono le funzioni che permettono di *creare, cancellare e mantenere* gli elementi e le relazioni che compongono il modello
- *Supporting system* : sono le funzioni in grado di *gestire una sessione* e di *prendere decisioni* in ambito del *controllo degli accessi*
- *Review* : sono le funzioni che *permettono di eseguire query* sul modello

il modello RBAC₃ comprende *quattro modelli* : il *core RBAC*, il *hierarchical RBAC*, *static separation of duty relations* (SSD) e *dynamic separation of duty relations* (DSD)



- Core RBAC : è l'equivalente del RBAC₀ , al quale il NIST ha aggiunto due entità subordinate
 - Object : sono tutte le risorse soggette al controllo degli accessi
 - Operation : sono gli eseguibili che se invocati, svolgono delle funzioni per l'utente
 - Permission : è l'approvazione che deve essere concessa per eseguire un'operazione su uno o più oggetti del RBAC

all'interno del Core RBAC le funzioni :

- administrative : permettono di creare/cancellare utenti o ruoli, associazioni utente/ruolo e privilegio/ruolo
 - System support : permettono di creare sessioni, aggiungere/eliminare un ruolo da una sessione e controllare se un soggetto ha privilegi adatti per operare su un oggetto.
 - Review : permette all'amministratore di monitorare ma non modificare tutti gli elementi e le relazioni del modello
- hierarchical RBAC : include i concetti di ereditarietà del modello RBAC₁. Il NIST definisce due tipi di gerarchie :
 - General role Hierarchies : permette l'eredità multipla, dove più ruoli possono ereditare dallo stesso ruolo subordinato e un ruolo può avere più ruoli subordinati
 - Limited role Hierarchies : un ruolo può avere più di un padre ma un solo subordinato dal quale eredita i privilegi (struttura ad albero)

l'ereditarietà permette di semplificare le relazioni tra ruoli. Utenti che hanno ruoli diversi potrebbero avere comunque dei privilegi in comune : è una situazione molto tipica per un'azienda che molti utenti condividano un insieme di privilegi standard.

Hierarchical RBAC aggiunge funzioni rispetto al Core RBAC:

- administrative : permettono di aggiungere/eliminare relazioni di ereditarietà tra ruoli, creare un nuovo ruolo e aggiungerlo come padre/subordinato di un altro ruolo
- Review : permette all'amministratore di vedere i permessi e gli utenti associati ad ogni ruolo sia quelli diretti che quelli che lo ereditano

SSD e DSD sono due componenti che aggiungono ulteriori vincoli al modello. Si tratta di separare i ruoli in modo da prevenire l'assegnamento di poteri eccessivi ad un ruolo che per eseguire il proprio lavoro gliene basterebbero di meno (implementa la proprietà del privilegio minimo)

- static operation of duty relations : se un utente è assegnato a un ruolo, non può essere assegnato a nessun'altro ruolo in questa sessione. SSD è una coppia (ruoli, n) dove a nessun utente vengono assegnati più di n ruoli appartenenti all'insieme.
- Dinamic operation of duty relations : limita la disponibilità di un privilegio aggiungendo dei vincoli sui ruoli che possono essere attivati durante la sessione di un utente. DSD è una coppia (ruoli, n>=2) dove una sessione utente non può attivare più di n ruoli.

DATABASE SECURITY

lo *standard query language* (SQL) è un linguaggio utilizzato per *definire* lo schema, *manipolare* e *interrogare* i dati all'interno di un *database relazionale* . Le *istruzioni SQL* sono usate per :

- creare tabelle
- inserire o eliminare dati
- creare viste
- recuperare dati attraverso le query

L'*SQL injection* è uno tra i più *pericolosi* e *diffusi* attacchi che si basano *sulle minacce alla sicurezza di rete* : è progettato per *sfruttare la natura dinamica delle applicazioni web* .

- I *contenuti dinamici* spesso vengono *trasferiti da/spediti* a un *database* che contiene un *elevato volume di informazioni*
- una *pagina web* è in grado quindi di *interrogare il database* e *spedire/ricevere* informazioni *critiche* per *agevolare l'user experience*

L'*SQL injection* quindi è progetto per *spedire* comandi SQL *malevoli* al *database* presente *sul server* di una *applicazione web*. Gli *obiettivi* più comuni per questo attacco sono :

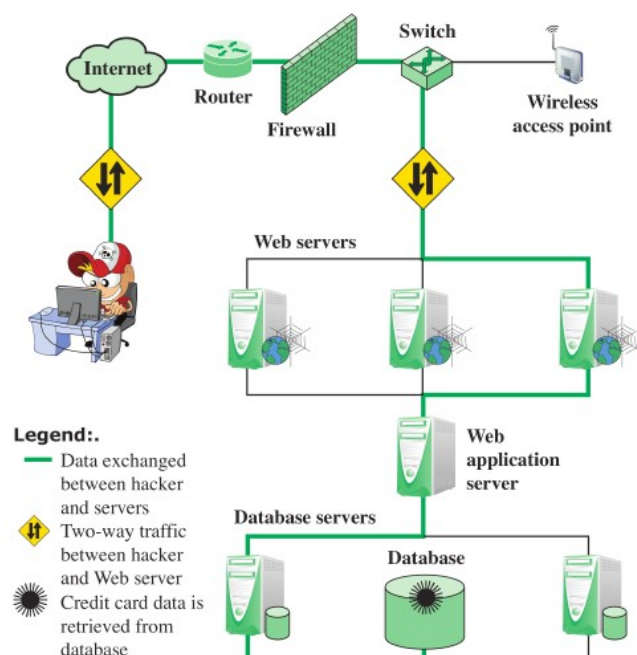
- *estrarre* di una *grande quantità* di dati
- *sporcare* le tabelle *aggiungendo migliaia* di tuple

a seconda dell'*ambiente* un attacco *SQL injection* può essere *sfruttato* anche per :

- *modificare* o *eliminare* dati
- *eseguire* comandi del sistema operativo
- lanciare attacchi *Denial of Service*

L'attacco *SQL injection* sfrutta le *vulnerabilità* del *database* di una *applicazione web*. L'attaccante quindi sarà in grado di *estrarre o manipolare* i dati dell'*applicazione*. Tale attacco è possibile quando l'*input dell'utente* non è *filtrato correttamente* (esempio permette l'inserimento di *caratteri di escape SQL*) oppure non è *fortemente tipato* permettendo *esecuzioni inaspettate*

1. l'attaccante trova una *vulnerabilità* nella *web application* e *inietta* dei comandi SQL al database *attraverso* l'invio di comandi al *web server*. Tali comandi verranno *accettati* anche da un *eventuale firewall* (in quanto *immessi nel traffico regolare*)
2. il *web servers* riceve il comando *malevolo* e lo invia al *server della web application*
3. la *web application* spedisce il comando *malevolo* al database
4. il *database* esegue il codice *malevolo*
5. il *server della web application* genera *dinamicamente* una pagina con i risultati del codice *malevolo* (es. Numeri carte di credito)
6. il *web server* spedisce questi risultati all'attaccante



tipicamente un attacco *SQL injection* viene messo in atto *terminando prematuramente* una stringa di testo e *aggiungendo infondo* un nuovo comando.

Esempio :

```
var Shipcity;
ShipCity = Request.form ("ShipCity");
var sql = "select * from OrdersTable where ShipCity = '" +
ShipCity + "'";
```

l'intenzione di questo script data la *shipcity* (*user input*) è quello di *restituire* gli ordini ad essa *legati*

```
SELECT * FROM OrdersTable WHERE ShipCity = 'Redmond'
```

se però *l'attaccante* invece di inserire *una città* inserisse *la seguente stringa* :

```
Boston'; DROP table OrdersTable--
```

lo script eseguirebbe la *seguente query* :

```
SELECT * FROM OrdersTable WHERE ShipCity =
'Redmond'; DROP table OrdersTable--
```

- il ";" *suddivide* il comando in *due comandi sperati*
 - *primo comando* : restituisce *gli ordini* legati alla città di *Redmond*
 - *secondo comando* : *elimina la tabella* degli ordini
- il "--" *commenta* il resto del *testo* rendendolo *non eseguibile*

gli attacchi *SQL injection* possono essere suddivisi in base *al canale* e *al tipo* di attacco :

- *User input* : l'attacco avviene *modificando opportunamente* l'*user input*. Di solito *l'input dell'utente* deriva da *un form* che viene *spedito* all'applicazione web tramite *una GET o un POST*.
- *Server variables* : l'attacco *modifica* le variabili che contengono *headers HTTP o protocolli di rete* oppure che contengono *altre variabili* di ambiente. L'attaccante *può nascondere* quindi al loro interno del *codice malevolo* che verrà eseguito ad esempio quando la *web application* leggerà un *header HTTP*.
- *Second-order injection* : l'attaccante potrebbe *fare affidamento* su dati *già esistenti* nel database. In questo modo l'attacco *SQL injection* non deriverebbe *da un utente* ma sarebbe causato dal *sistema stesso*
- *Cookies* : i *cookies* sono controllati dal *client*, quindi un attaccante *può modificarne il contenuto* aggiungendo del *codice malevolo* in modo che ogni query che si basa sui *cookies* venga alterata dall'attacco *SQL injection*
- *Physical user input* : l'attacco avviene tramite *l'utilizzo* di oggetti leggibili da un *lettore ottico* che poi invia il codice al *database*

gli attacchi SQL *injection* vengono raggruppati in tre grandi *categorie* :

- *inband attack* : viene utilizzato lo *stesso canale di comunicazione* sia per l'attacco SQL *injection* sia per il *recupero del risultato*.
 - *Tautology* : questo tipo di attacco *inietta il codice* utilizzando *più istruzioni* la cui condizione è *sempre verificata*

```
$query = "SELECT info FROM user WHERE name =  
'$_GET['name']' AND pwd = '$_GET['pwd']'";
```

questo script richiede nome e password

un attacco di tipo *tautology* consiste nell'inserire nel *campo name* la seguente *stringa*

```
SELECT info FROM users WHERE name = ' ' OR 1=1 -- AND pwd = ' '
```

in questo modo viene *disabilitato il controllo sulla password* e la *clausola WHERE* è *sempre vera* (*tautologia*) e quindi la *query risulta verificata* per ogni *tupla della tabella user* e ne restituirà *le informazioni*

- *end of line comment* : il comando "--" serve per *commentare* e quindi *non far eseguire il codice che segue* . Viene utilizzato dopo *aver iniettato del codice malevolo*.
 - *Piggybacked queries* : l'attaccante *aggiunge una query* oltre a quella *legittima*, in modo che l'attacco si *appoggi su una richiesta legittima*
- *inferential attack* : l'attaccante è in grado di *ricostruire l'informazione* spedendo particolari richieste e *osservando il comportamento* del website/database server.
 - *Illegal/logical incorrect queries* : è considerato come attacco *preliminare* in previsione di quello *vero e proprio* e consente all'attaccante di ottenere informazioni sul *tipo* e sulla *struttura* del database. Si basa sul *trarre informazioni* dalla *pagina di errore* che si genera all'invio di una *query errata*. Il semplice fatto che venga generato un messaggio di errore *rivela possibili parametri vulnerabili* a un attacco di SQL *injection*
 - *Blind SQL injection* : permette all'attaccante di *dedurre quali dati* sono presenti all'interno del database anche nel caso in cui il sistema è *abbastanza sicuro* da non *mostrare messaggi di errore* che potrebbero rivelare *vulnerabilità*. Se l'*istruzione iniettata* risulta *vera* allora il sito continuerà a *funzionare correttamente*, altrimenti la pagina *modificherà significativamente* il suo comportamento.
 - *out of band attack* : i dati vengono *recuperati* attraverso un *diverso canale di comunicazione*. Questo tipo di attacco viene adottato quando ci sono *delle limitazioni* sul *recupero delle informazioni*, ma la *connettività in uscita* dal database è *vulnerabile*

molti attacchi di SQL *injection* hanno successo a causa della *poca sicurezza* adottata in fase di *codifica* . Le *contromisure* adottate vengono raggruppate in 3 *categorie* :

- *defensive coding*
- *detection*
- *run-time prevention*

il *defensive coding* è l'unica pratica in grado di *ridurre drasticamente* la *minaccia* del *SQL injection*.

- *Manual defensive coding practise* : la *vulnerabilità più sfruttata* è quella riguardante la *carenza di controllo durante la validazione dell'input*. Per eliminare questa *vulnerabilità* bisogna adottare *pratiche difensive adeguate*
 - *type checking* : controlla che un *input* che si suppone sia *numerico* non contenga *caratteri*. Questa tecnica evita gli attacchi che si basano sulla *forzatura degli errori* nel *dbms*
 - *pattern matching* : tenta di *distinguere* un *input* normale da uno *anormale*
- *parameterized query insertion* : permette allo sviluppatore di *specificare la struttura* delle query SQL e di *passare i valori dei parametri separatamente* in modo da *non permettere a nessun'altro* di *cambiare la struttura* della query stessa
- *SQL DOM* : è un insieme di tecniche che permettono la *validazione automatica* dei tipi di dato. Lo sviluppatore può *sistematicamente applicare* nuove tecniche di *input filtering* e *type checking*

esistono diverse *misure di detection* :

- *signature based* : sono tecniche che *tentano di riconoscere un pattern specifico*. Queste tecniche devono essere *costantemente aggiornate* e non funzionano contro attacchi *in grado di auto-modificarsi*
- *anomaly based* : questa tecnica cerca di *definire un comportamento normale* e quindi prova a *individuare quelli anomali*. Esistono due fasi
 - *training fase* : il sistema *impara* quali sono i *comportamenti normali*
 - *detection phase* : una volta terminata la prima fase, il sistema è in grado di *individuare i comportamenti anomali*
- *code analysis* : si esegue un *test* in grado di *individuare le vulnerabilità sfruttabili* da un attacco di tipo *SQL injections*

le tecniche di *run-time prevention* invece *controllano* le query in fase di *runtime* e stabiliscono se sono *conformi* al modello di query *che si aspettano*

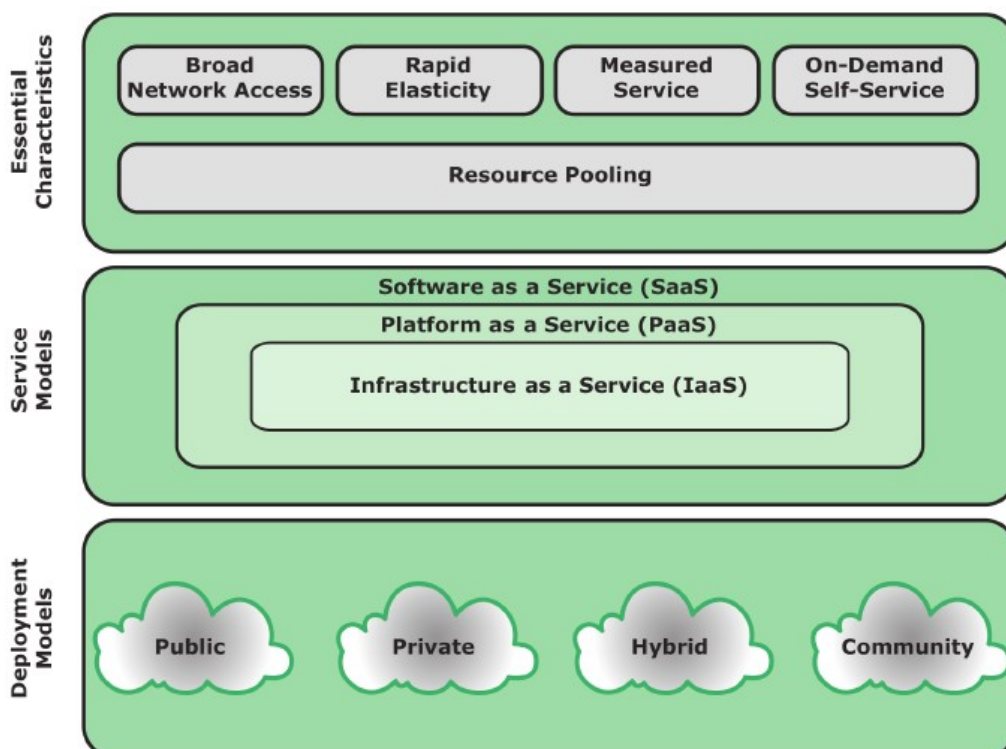
CLOUD COMPUTING

"il *cloud computing* è un modello che permette, *su richiesta*, di accedere in modo *ubiquo e conveniente* ad una *piscina condivisa* di *risorse computazionali configurabili* che possono essere fornite *rapidamente* e *rilasciate* con il *minimo costo di management* o *interazione* con il *service provider*. Il *modello cloud* promuove la *disponibilità* e si compone di *5 caratteristiche essenziali*, *tre modelli di servizio* e *quattro modelli di impiego*"

il cloud computing – NIST

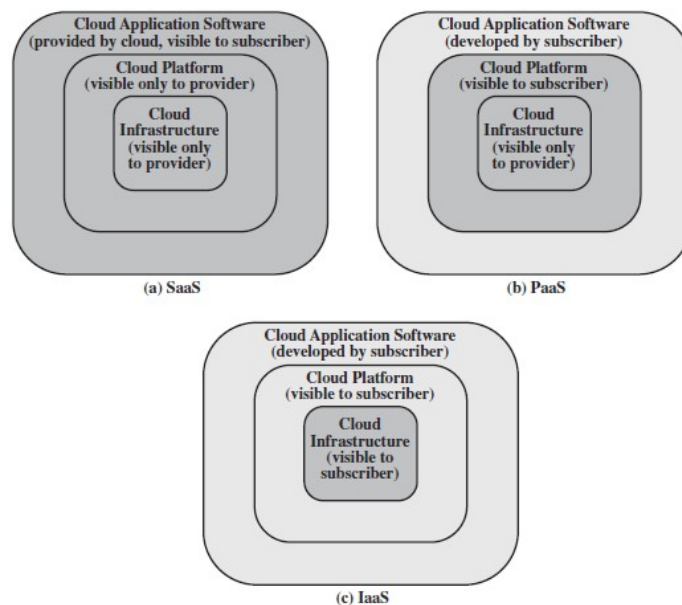
le *caratteristiche essenziali* del *cloud computing* sono :

- *Broad network access* : è la capacità di essere *disponibile in rete* e di essere *accessibili* attraverso i *meccanismi standard* che promuovono l'uso di *piattaforme eterogenee* (*mobile, laptop*)
- *rapid elasticity* : è la possibilità di *espandere* oppure *ridurre* le risorse a disposizione *a seconda del bisogno* (ad esempio *per un task* ho bisogno di *molte risorse*, che poi al termine verranno *rilasciate*). Questa caratteristica è chiamata *killer feature of cloud computing*
- *Measured service* : il *cloud system* controlla e ottimizza *automaticamente* le risorse sfruttando un *livello di astrazione appropriato* per il tipo di servizio. Le *risorse utilizzate* sono *monitorate e controllate*, garantendo *trasparenza* sia per chi le fornisce che per chi le utilizza
- *On-Demand Self-Service* : il *consumatore* può *ottenere un aumento* della capacità di calcolo (esempio *server time, network storage*) senza *alcuna iterazione umana* con nessun *server provider*. Siccome il servizio è *on-demand* (a richiesta) la risorse non *fanno parte permanentemente* dell'infrastruttura dell'utente
- *Resource pooling* : le *risorse di calcolo* messe a disposizione sono *in comune* e quindi sono a disposizione di *pù utenti* utilizzando un modello *multi-tenant*, dove le risorse *fisiche e virtuali* sono *dinamicamente assegnate e riassegnate* a seconda della *richiesta* dei consumatori. Il *consumatore* in generale *perde il controllo* sulla *locazione delle risorse* fornite, anche se può *specificarlo* ad un *livello più alto di astrazione* (indicando ad esempio lo stato, il paese o il datacenter)



i modelli di servizio sono in realtà *innestati* uno sopra l'altro :

- *Software as a Service (SaaS)* : può essere visto come il *livello più esterno* e il servizio offerto riguarda *applicazioni software* eseguibili ed accessibili all'interno del cloud. Il *SaaS* permette di utilizzare le *applicazioni* eseguibili *sulla infrastruttura* messa a disposizione dal cloud. Queste applicazioni sono *accessibili* attraverso una *semplice interfaccia web*. Il *cloud provider* (es Gmail) inoltre si occupa dalla *installazione, aggiornamento, mantenimento e patch* dell'applicazione software
- *Platform as a Service (PaaS)* : è il *livello intermedio* e fornisce una *piattaforma* su cui l'utente può far eseguire delle applicazioni software. Il *cloud provider* (es Google App Engine) fornisce una *soluzione stack*, ossia una *serie di programmi software*, dei *tool di programmazione*, un ambiente *runtime* e altri strumenti per realizzare nuove applicazioni. Il *PaaS* in sostanza fornisce un *sistema operativo* all'interno del cloud
- *Infrastructure as a Service (IaaS)* : è il *livello base* e permette all'utente di *accedere* alla *infrastruttura* del cloud. Il *cloud provider* (es Amazon EC2 o Windows Azure) fornisce *macchine virtuali*, tutto l'*hardware astratto* e i *sistemi operativi* che possono essere controllati attraverso le *application programming interfaces (API)*. In questo modo l'utente è in grado di *combinare* i servizi computazionali di base e *costruirsi il proprio sistema* all'interno del cloud



il Nist inoltre *definisce* quattro modelli di impiego :

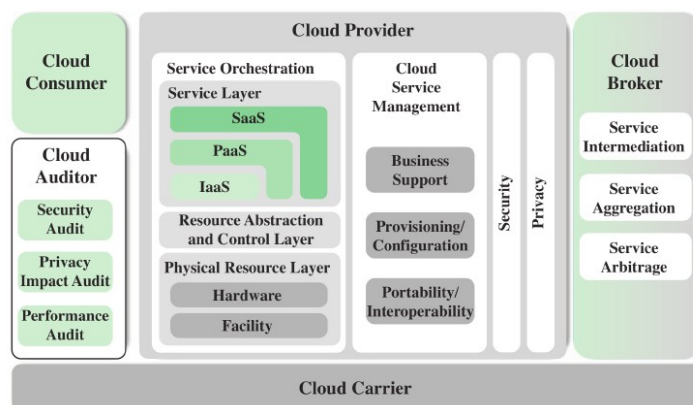
- *public cloud* : l'*infrastruttura* del cloud è *disponibile* al pubblico oppure ad un *elevato numero* di compagnie ed è di *proprietà* di un'organizzazione che *vende servizi cloud*. Il *cloud provider* (es Microsoft Azure) è *responsabile* sia dell'*infrastruttura* e sia del *controllo* sui dati e operazioni all'interno del cloud
- *private cloud* : l'*infrastruttura* del cloud è utilizzata *esclusivamente* da un'organizzazione. Può essere *gestita da una terza parte*, la quale sarebbe *responsabile* solo dell'*infrastruttura* ma non del *controllo* sui dati e le operazioni che avvengono su di essa
- *community cloud* : l'*infrastruttura* del cloud è *condivisa* tra più organizzazioni le quali fanno parte della *stessa comunità* oppure hanno delle *caratteristiche che le accomuna* (es *mission* o *security policies*).
- *hybrid cloud* : l'*infrastruttura* del cloud è una *composizione* delle precedenti legate insieme da *tecnologie standard* o *proprietarie* che permettono la *portabilità* dei dati e delle applicazioni

secondo il NIST, *l'architettura di riferimento* dovrebbe focalizzarsi *sul cosa* deve fornire un *servizio cloud* e *non sul come* la soluzione deve essere *progettata* o *implementata*. L'obiettivo dell'*architettura di riferimento* secondo il NIST è quello di :

- illustrare i *vari servizi cloud*
- fornire un *punto di riferimento* per permettere agli utenti di *comparare* i *vari servizi cloud* offerti
- facilitare *l'analisi* per la scelta di *implementazioni standard* riguardanti la *sicurezza, l'interoperabilità e la portabilità* .

L'architettura è composta da *cinque attori principali* :

- *cloud consumer* : è un *utente* o un *organizzazione* che *utilizza* i *servizi* forniti dal *cloud provider*
- *cloud provider* : una *persona* o un *organizzazione* che *mette a disposizione* il servizio e del quale *ne è responsabile*
- *cloud auditor* : è una *parte indipendente* che si occupa di *valutare* il *servizio cloud* offerto (*sicurezza, performance*)
- *cloud broker* : è un'entità che si occupa della *negoiazione* tra il *cloud provider* e il *cloud consumer*.
- *Cloud carrier* : è un *intermediario* che *fornisce la connettività* e il *trasporto dei servizi cloud* tra *fornitore e consumatore*



per ogni modello di servizio il *cloud providers* deve fornire la *memoria* e la *semplificazione dei processi* necessarie per *supportare* il modello, insieme ad un *interfaccia* per l'*utente consumatore* :

- per il *SaaS* : implementa, configura, mantiene e aggiorna le *operazioni* delle *applicazioni software* sulla *infrastruttura cloud* in modo che il *servizio offerto* sia al *livello delle aspettative* di chi lo utilizza
- per il *Paas* : il *cloud provide* gestisce l'*infrastruttura della piattaforma* e lancia i *software* che forniscono i *componenti della piattaforma* (es *database* o altri *middleware*)
- per il *Iaas* : il *cloud provide* si occupa di *acquisire le risorse fisiche*

il *cloud broker* è una figura *molto utile* nel caso in cui la *gestione di un servizio cloud* risulta essere *troppo complessa* per un *cloud consumer* :

- *Service intermediation* : aggiunge *servizi di gestione, valutazione performance e rafforzamento della sicurezza*
- *Service aggregation* : è un servizio che *combina più servizi cloud* in modo da *soddisfare le esigenze* del consumer sfruttando servizi offerti da *clod providers diversi*
- *Service arbitrage* : simile al *service aggregation* con l'*eccezione* che il broker *può scegliere i servizi* da *agenzie diverse* .

In termini *general*i, i controlli di *sicurezza* in ambito *cloud* sono *simili* a quelli presenti in *ogni altro ambiente IT*. Tuttavia in un *ambiente cloud*, le aziende sostanzialmente *perdono parte del controllo* sulle risorse, servizi e le applicazioni le quali *devono comunque garantire* un certo grado di *sicurezza e privacy* e questo comporta una *nuova serie di minacce* :

- *abuse and nefarious use of cloud computing* : per un *cloud provider* è relativamente semplice *offrire* alcuni *periodi gratis* di prova del servizio. Questo permette ad un *attaccante* di *entrare gratuitamente* all'interno del servizio *cloud* e lanciare *diversi attacchi* tra i quali lo *spamming*, codice *malevolo* o attacchi *DoS*. Il compito di *proteggere* il sistema da questi attacchi *ricade sul cloud provider*, ma sono i *clienti* che devono *monitorare l'attività* che riguarda i *dati e risorse* in modo da *individuare comportamenti malevoli*.
 - Le *contromisure* includono :
 - *controllo stretto* sulle *registrazioni e validazione* dei processi
 - *rafforzamento del monitoraggio* sulle *frodi* con carta di credito
 - *controllo sul traffico di rete* del consumatore
 - *monitoraggio della blacklist pubblica* per ogni *blocco di rete*
- *insicure interfaces and APIs* : la *sicurezza* di un servizio *cloud* dipende dall'*interfaccia* e dalle *APIs base* che permettono al *cliente* di *gestire e interagire* con i *servizi cloud*. Dall'*autenticazione e controllo degli accessi* fino alla *crittografia* e al *monitoring*, queste *interfacce* devono essere *protette* da *danni accidentali* o *tentativi malevoli* di *aggirare le security policy*.
 - Le *contromisure* includono :
 - *analisi del modello di sicurezza* adottato dall'*interfaccia* del *cloud provider*
 - *assicurarsi* che sia stato *implementato* un *forte sistema* di *autenticazione e controllo degli accessi* unito alla *crittografia* delle trasmissioni
 - *comprensione della catena di dipendenze* tra le vari API
- *Malicious insiders*: molti aspetti della *sicurezza* gravano *direttamente sul cloud provider* al quale deve essere conferito un *elevato grado di affidabilità*. In particolare bisogna *fidarsi* sulla questione del *rischio di attività malevole interne* al *cloud provider* stesso. Le *architetture cloud* richiedono di *assumere certi ruoli* estremamente *rischiosi*, ad esempio quello di *amministratore di sistema* e la *gestione dei servizi di sicurezza*.
 - Le *contromisure* includono :
 - un *rafforzamento* della *catena di gestione* e una *valutazione dei venditori*
 - *specificare all'interno* del contratto anche i *requisiti* legati alle *risorse umane*
 - *richiedere totale trasparenza* sulle *pratiche di gestione*, sulle *informazioni* relative alla *sicurezza*
 - *determinare un processo di notifica* delle *breccie nel sistema di sicurezza*
- *Shared technology issues*: spesso i *componenti che sorreggono l'infrastruttura cloud* (es CPU cache) *non sono progettati* per offrire una *proprietà di isolamento forte* per una architettura *multi-tenant*. Di solito i *cloud provider* per ogni cliente *utilizzano delle macchine virtuali* isolate tra loro. Questo approccio però *rimane vulnerabile agli attacchi* ed è solo una *parte della strategia* da adottare.
 - Le *contromisure* includono :
 - *implementare le best practice* di *sicurezza* per *installazione/configurazione*
 - *monitoraggio dell'ambiente* per *identificare attività non autorizzate*
 - *rafforzamento dei processi di autenticazione e controllo accessi* per l'amministratore
 - *rafforzamento dei service level agreement* per *tappare le vulnerabilità*
 - *condurre una scansione delle vulnerabilità* e *configurare un servizio di audit*

- *Data loss or leakage* : per molti clienti, l'impatto più devastante riguarda la perdita o il furto dei dati tramite lo sfruttamento di una falla nella sicurezza.
 - Le contromisure includono :
 - implementazione di un forte controllo degli accessi delle API
 - crittografia e protezione dell'integrità dei dati in transito
 - analisi del livello di protezione dei dati sia in fase di progettazione che runtime
 - implementa pratiche forti di generazione di chiavi, gestione e memorizzazione, e distruzione dei dati
- *Account or service hijacking* : di solito si tratta di furto delle credenziali e rimane la minaccia principale. L'attaccante quindi potrebbe accedere alle aree critiche dove si trovano i servizi di cloud computing e permettergli di compromettere la confidenzialità, integrità e disponibilità di questi servizi
 - le contromisure includono :
 - proibire la condivisione delle credenziali tra utenti e servizi
 - dove possibile implementare l'autenticazione a due fasi
 - implementare un attività di monitoraggio proattivo per individuare attività non autorizzate
 - capire le politiche di sicurezza del cloud provider e le security license agreement
- *Unknown risk profile* : usando una infrastruttura cloud il cliente deve necessariamente cedere il controllo al cloud provider di una serie di problemi che possono minare la sicurezza. Per questo motivo il cliente deve prestare attenzione e definire in modo chiaro i ruoli e le responsabilità per la gestione di tali rischi.
 - Le contromisure includono :
 - divulgazione di log e dati
 - parziale o completa divulgazione dei dettagli infrastrutturali
 - monitorig e notifiche di informazioni necessarie

esistono molti modi per compromettere un dato all'interno di una infrastruttura cloud :

- eliminazione o modifica dei dati i quali non hanno copie di backup
- scollegare un record dal proprio contesto in modo da renderlo irrecuperabile
- memorizzazioni su dispositivi non affidabili
- perdere la chiave di codifica di elementi crittografati
- accesso non autorizzato a dati sensibili

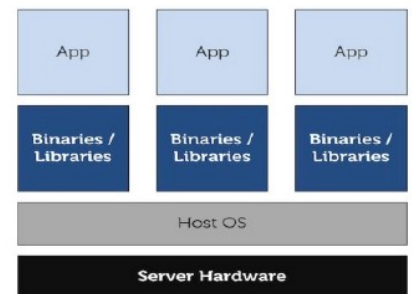
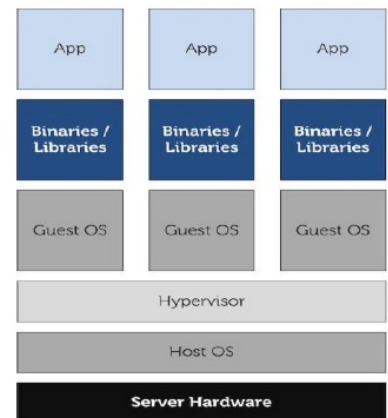
la minaccia di compromissione dei dati aumenta nel cloud a causa del numero di interazioni, i rischi presenti all'interno del cloud stesso e dalle caratteristiche architetturali e operazionali dell'ambiente cloud.

L'ambiente cloud può cambiare in modo significativo e in generale esistono due modelli :

- *multi-istanza* : viene fornito un unico DBMS che viene eseguito su ogni macchina virtuale assegnata a un singolo cliente del servizio cloud.
Il cliente in questo modo ha il controllo completo su :
 - definizione dei ruoli
 - autorizzazione utenti
 - azioni amministrative in ambito di sicurezza
- *multi-tenant* : ad ogni cliente viene fornito un ambiente predefinito che però è condiviso con gli altri . Ogni dato sarà etichettato con l'user identifier del cliente proprietario. Il sistema di etichettamento fa apparire l'uso dell'istanza esclusivo e non condiviso ma in realtà si basa sul cloud provider che deve fornire un database sicuro e mantenerlo tale

in ambiente cloud esistono due tecniche di virtualizzazione :

- *hypervisor* : consente a *diversi sistemi operativi* (macchine virtuali) di *condividere lo stesso hardware*. Ad ogni *macchina virtuale* sembrerà di avere *una propria memoria*, e un *proprio processore* (che in realtà *appartengono all'host* che fornisce *l'hardware comune*) . L'*hypervisor* controlla l'*host* che *condivide l'hardware*, assegnando le risorse *necessarie* ad ogni *macchina virtuale* facendo anche in modo che *non interferiscano tra loro*
- *container-based* : con questo approccio *non c'è overhead* derivatne dal fatto che *ogni macchina virtuale* debba installare un *proprio sistema operativo*. Esiste un *solo sistema operativo* che si preoccupa di rispondere alle *chiamate hardware*.



I dati devono essere *protetti* sia in fase di *transito*, che di *utilizzo* e *inutilizzo* e l'*accesso* ad essi deve essere *controllato*. Il *cliente* può adottare la *crittografia* per progettare i dati durante il *transito* delegando la *gestione* delle chiavi al *cloud provider*, può inoltre *rafforzare* le tecniche di *controllo* degli *accessi* lasciando poi l'*onere* al *cloud provider* di *estendere* tale tecnica in base al *modello del servizio* utilizzato. Per i dati che al momento non sono *ne in transito* e *ne in uso* la scelta migliore è quella di *crittografare* il database e *all'interno del cloud* di *memorizzare* solo dati *crittografati* senza fornire al *cloud provider* la chiave *crittografica* usata, evitando così che il *cloud provider* abbia la possibilità di *leggere i dati* (restando comunque *vulnerabili* ad attacchi DoS o *corruption*).

La *crittografia* viene dunque usata per :

- rendere *sicura* la comunicazione tra il *cliente* e il *cloud provider*
- rendere *sicuro* il *trasferimento* dei dati tra *diversi provider*
 - nel 2014 la NSA *sniffava* il traffico tra i *server di google*

l'idea di *crittografare tutti i dati* che vengono spediti al cloud risolve le seguenti *minacce* :

- *malicious insider*
- problemi legati alla *condivisione* di tecnologie
- *furto* dei dati

ma ne crea di nuove :

- *gestioni* delle chiavi *crittografiche*
- il *server side* avendo a che fare con *dati crittografati* non è in grado di *eseguire operazioni* su di essi (esempio fare la *somma* tra due celle in un foglio di calcolo *criptato*)

la *crittografia omomorfica* è una forma di *crittografia* che permette *alcune specifiche computazioni* di essere estratte dal *ciphertext* e *generare un risultato crittografato* , tale che una volta *decryptato* il risultato coincida con quello ottenuto eseguendo la stessa operazione sui dati in chiaro

con il termine *security as a service* generalmente ci si riferisce a un *pacchetto* di servizi di sicurezza offerti dal *cloud provider* che scarica la responsabilità in termini di sicurezza "dalle spalle" dell'azienda consumatrice

nel contesto del *cloud computing* la *cloud security alliance* ha identificato le seguenti categorie di servizi di *cloud security as a service* :

- *identity and access management* : assicura che l'identità di un'entità che utilizza/gestisce le risorse è verificata e in base a tale verifica assegna il corretto livello di accesso.
- *data loss prevention* : viene implementata dal *cloud client* e si occupa di monitorare, proteggere e verificare la sicurezza dei dati in transito, uso, inutilizzati. Il *cloud security provider* può stabilire le regole che riguardano le operazioni possibili sui dati a seconda dei contesti
- *web security* : è una protezione in tempo reale che può essere offerta sia da applicazioni installate che dal *cloud* tramite il reindirizzamento del traffico al *cloud provider*
- *e-mail security* : protegge i canali in entrata e uscita, cercando di evitare attacchi di phishing, allegati malevoli e spam
- *security assessments* : facilita il lavoro di auditing da parte di terze parti fornendo strumenti e punti di accesso
- *intrusion management* : consente nell'implementare nel punto di ingresso al *cloud* e sul server dei sistemi di *intrusion prevention and detection* .
- *security information and event management* : aggrega i log e gli eventi provenienti da applicazioni/sistemi/reti sia virtuali che fisiche e fornisce un report in tempo reale e implementa un meccanismo di alert nel caso sia richiesto un intervento o un diverso tipo di report
- *encryption* : è un servizio pervasivo che coinvolge i dati, il traffico mail, informazioni specifiche e quelle che riguardano l'identità dell'utente
- *business continuity and disaster recovery* : comprende le misure e i meccanismi che assicurano la resilienza delle operazioni in caso di interruzione del servizio
- *network security* : consiste nel rendere sicuri i servizi che allocano gli accessi, distribuiscono, monitorano, e proteggono le risorse del servizio

FIREWALL

il *firewall* è uno *strumento* in grado di *proteggere* un *sistema locale* o un *insieme* di sistemi da *minacce* alla *sicurezza di rete*. Il *firewall* permette al *sistema* di *accedere al mondo esterno* tramite *reti geografiche* o più comunemente *internet*.

Le informazioni di sistema hanno subito una *costante evoluzione* :

- il sistema *che processa* i dati è *centralizzato* ed è composto da un *mainframe* che funge da *supporto* a numerosi *terminali* ad esso *collegati*
- un *local area network* (LAN) *interconnette* i PC e i terminali *tra di loro* e al *manframe*
- un *rete interna* è formata da un *insieme* di LAN
- una *enterprise wide network* è costituita da *numerosa rete interne* sparse *geograficamente* e collegate tra loro da una *private wide area network* (WAN)
- la *connettività a internet* , dove possono *comunicare* tutte le *reti interne* può o non può essere *garantita* da una *WAN privata*

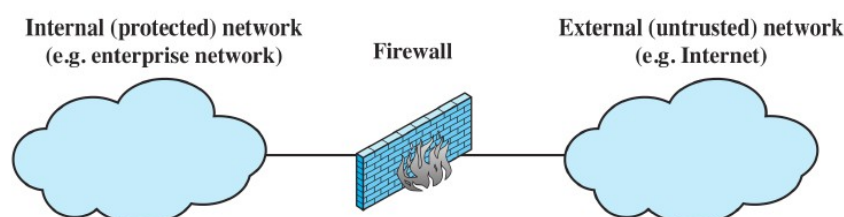
in realtà la *connettività ad internet* è diventata *praticamente obbligatoria* in quanto le *informazioni reperibili* in rete sono *essenziali* per un' azienda. Se la LAN *non fornisce* tale accesso, i dipendenti infatti ricorrono le *funzionalità wireless* dei propri dispositivi per *collegarsi* a un *internet service provider* (ISP). Se da un lato internet *fornisce benefici* a un azienda, dall'altro *permette* al mondo esterno di *raggiungere* e *interagire* con le risorse della *rete locale*.

Questo espone l'azienda a *nuove minacce* : per ogni *workstation* e *server* facenti parte della *rete locale* devono essere *rafforzate* le *caratteristiche di sicurezza*, ad esempio con meccanismi di *intrusion protection*, anche se a volte *risultano insufficienti* o *non convenienti economicamente*.

Quando viene riscontrata una *falla nella sicurezza* , deve essere *aggiornato* ogni sistema *potenzialmente infetto*. Questo richiede una *configurazione scalabile* e un sistema *aggressivo di patching* efficientissimo ed è *necessario* se la *sicurezza* si basa ed è incentrata *sul singolo host* (host-based security). Un meccanismo *alternativo* o *complementare* prevede l'*utilizzo di un firewall* tra la *rete interna* e *internet* che funge da *collegamento* e da *muro perimetrico* di sicurezza contro l'esterno. L'obiettivo è quello di *proteggere* la *rete interna* dagli *attacchi provenienti da internet* e di fornire un *singolo punto di ingresso* (*choke point*) dove possono essere imposti meccanismi di *sicurezza* e *auditing*. In sostanza il *firewall* fornisce un *ulteriore strato difensivo* che *isola* la *rete interna* dall'esterno

chi *progetta* firewall di solito *si pone* i *seguenti obiettivi* :

1. tutto il *traffico* deve *passare attraverso* il *firewall*. Tutti gli altri accessi sono *fiscamente bloccati*
2. solo il *traffico autorizzato* dalle politiche di sicurezza *può passare*
3. il *firewall* è immune alle *penetrazioni*. i sistemi *affidabili* sono *adatti* per ospitare (*hosting*) un *firewall*



il *punto critico* durante lo sviluppo e l'implementazione di un *firewall* è rappresentato dalle *politiche di accesso* :

- quale *tipo di traffico* è autorizzato a passara attraverso il *firewall*
- quali *protocolli, indirizzi, applicazioni e contenuti*

queste politiche devono essere sviluppate *specificando ampiamente* il *tipo di traffico* che sostiene l'azienda in modo da poter *definire i dettagli* sugli elementi *filtranti da implementare* all'interno del *firewall*.

Per *filtrare il traffico* di solito ci si basa sulle *seguenti caratteristiche* :

- *ip address and protocol values* :usato per *limitare l'accesso* a servizi *specifici*
- *application protocol* : usato per *trasmettere e monitorare* lo *scambio di informazioni* tra *protocolli di specifiche applicazioni*
- *user identity* : è utilizzato per *gli utenti interni* la cui *identificazione* funge da *autenticazione* come *utente sicuro*
- *network activity* :tiene in considerazione il *momento* in cui viene *effettuata* una *richiesta*

un *firewall* deve essere in grado di :

1. *definire un singolo punto di accesso* (choke point) in modo da *proteggere la rete locale* da accessi *non autorizzati* o *attacchi* provenienti dalla *rete esterna*. L'uso di un *singolo punto* permette di *semplificare la gestione* della *sicurezza* in quanto *ci si concentra* in una *sola zona* del sistema
2. deve fornire un sistema di *monitoraggio* di eventi legati alla *sicurezza*. Sul *firewall* bisogna poter implementare meccanismi di *alarms e audits*
3. deve essere una *piattaforma conveniente* per *funzioni internet insicure*. Ad esempio su un *firewall* può essere aggiunta la funzione *network address translate* che si occupa di *mappare gli indirizzi locali* con quelli di *internet*.
4. Può essere usato per *implementare* una *virtual private network* (VPN)

esistono però delle *limitazioni* :

1. il *firewall* non è in grado di *proteggere* il sistema da *attacchi* capaci di *bypassarlo*
2. il *firewall* non è in grado di *proteggere da tutte* le minacce *interne*, ad esempio da *impiegati scontenti* o che *collaborano* con un *attaccante esterno*
3. un *firewall* non è in grado di *proteggere* il sistema da *attacchi provenienti* da *comunicazioni wireless* diverse da quelle *sui cui è installato* il *firewall interno*.
4. Un dispositivo *portatile* può essere *infettato* quando ci si trova *all'esterno* e poi *collegato alla rete interna* evitando il *controllo del firewall*

un *firewall* può *monitorare* il *traffico di rete* a *diversi livelli* . La *scelta del livello appropriato* viene determinato dalle *politiche di accesso* desiderate :

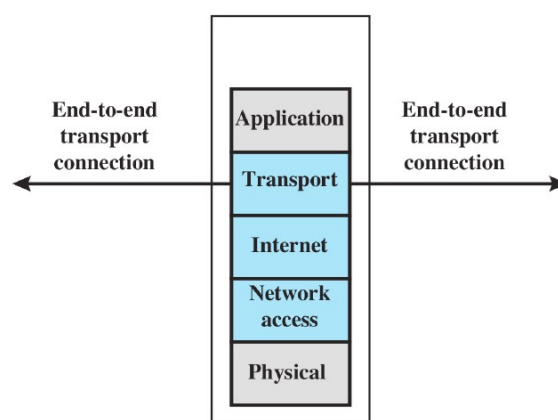
- *positive filter* : concedono di *passare solo* ai pacchetti che *rispettano criteri specifici*.
- *Negative filter* : vengon *bloccati* i pacchetti che *presentano specifici criteri*

a seconda del *tipo di firewall* possono venire esaminati :

- uno o più *protocol headers* per ogni *pacchetto*
- il *carico utile* (payload) per ogni *pacchetto*
- il *percorso generato* da una *serie di pacchetti*

il *packet filtering* firewall applica un insieme di regole ad ogni pacchetto IP in entrata o uscita dalla rete e sceglie se farli passare oppure scartarli . Le regole di *filtering* si basano sulle informazioni contenute all'interno del pacchetto :

- *source IP address* : l'indirizzo del sistema che ha creato il pacchetto
- *Destination IP address* : l'indirizzo che il pacchetto sta tentando di raggiungere
- *source and destination transport-level address* : il livello di trasporto (TCP o UDP) e il numero di porta che si cerca di raggiungere
- *IP protocol field* : a seconda del livello di trasporto adottato
- *interface* : se il *firewall* ha più ingressi, sapere da quale proviene (pacchetto in uscita) o a quale è destinato (pacchetto in entrata)



il *filtro* sui pacchetti di solito si basa su una lista di regole che riguardano gli *headers* IP o TCP :

- se una di queste regole trova corrispondenza allora tale regola stabilirà se il pacchetto verrà scartato oppure no
- se non si trova nessuna corrispondenza allora verrà intrapresa l'azione di default
 - *discard* : tutto quello non espressamente permesso è vietato. Questa scelta è conservativa. Inizialmente tutto è bloccato e i servizi permessi devono essere aggiunti mano a mano.
 - *forward* : tutto quello non espressamente vietato è permesso . Questa scelta semplifica l'utilizzo da parte degli utenti ma riduce la sicurezza, in quanto l'amministratore deve reagire ad ogni nuova minaccia come se la conoscesse bene.

Esempio : insieme di regole per il traffico SMTP (*simple mail transport protocol*)

l'obiettivo è quello di permettere il traffico in entrata ed uscita di mail , e di bloccare tutto il resto. Le regole sono applicate dall'alto al basso ad ogni pacchetto

Rule	Direction	Src address	Dest address	Protocol	Dest port	Action
1	In	External	Internal	TCP	25	Permit
2	Out	Internal	External	TCP	>1023	Permit
3	Out	Internal	External	TCP	25	Permit
4	In	External	Internal	TCP	>1023	Permit
5	Either	Any	Any	Any	Any	Deny

1. il traffico in entrata verso porta 25 (porta per il traffico SMTP) è permesso
2. questa regola permette di rispondere a una connessione SMTP in entrata
3. il permette di spedire mail in uscita verso una sorgente esterna
4. questa regola permette di rispondere a una connessione SMTP in entrata
5. questa regola esplica l'istruzione di default (in questo caso è *discard*). Tutti gli insiemi di regole devono prevederla e viene implicitamente chiamata l'ultima regola

questo insieme di regole comporta *diversi problemi* :

- la *regola 4* permette al traffico esterno di accedere a *qualunque porta* di numero superiore a 1023
 - un attaccante potrebbe sfruttare questa regola per aprire una connessione dalla sua porta 5150 a un server web proxy interno che si trova a porta 8080 ed essere quindi in grado di attaccare il server.
 - Una *contromisura* è quella di configurare un campo per ogni regola che indichi la porta di ingresso (settandola a >1023 però saremmo comunque ancora vulnerabili)
- le regole 3 e 4 specificano che *qualunque host* della rete interna può spedire mail all'esterno
 - una *macchina esterna* potrebbe essere configurata in modo di avere altre applicazioni collegate a porta 25. in questo modo un attaccante potrebbe ottenere l'accesso a una macchina interna spedendo un pacchetto TCP su porta 25
 - una *contromisura* è quella di aggiungere un campo *ACK flag* . Quando viene stabilita una connessione viene settato l'ACK flag sul segmento TCP in modo da riconoscere i segmenti inviati dall'altra parte. In questo modo la *regola 4* fa passare solo i pacchetti TCP, con porta di origine 25 che hanno il flag ACK nel segmento TCP

uno dei vantaggi dei *packet filtering* firewall è la semplicità, trasparenza verso gli utenti e la velocità dovuta al *poco overhead* dovuto al fatto che devono essere controllati tutti i pacchetti in entrata e uscita (CPU e RAM) il che rende preferibile implementare *firewall hardware*

questo tipo di *firewall* però ha le sue debolezze :

- non esaminando i dati a un livello superiore ai pacchetti, non possono prevenire attacchi che sfruttano vulnerabili presenti all'interno di una applicazione o una funzione . Se il *firewall* permette di passare a un applicazione, il permesso vale per tutte le funzioni interne dell'applicazione
- sono necessarie regole di registrazione per stabilire quali dati devono essere memorizzati in quanto le uniche informazioni contenute nei logs del firewall sono le stesse usate per il controllo degli accessi
- non sono supportate funzioni avanzate di *user authentication* in quanto opera solo a livello di pacchetti e non ad un livello più alto
- è vulnerabile ad attacchi che sfruttano dei bug all'interno del protocollo TCP/IP. Non riesce ad individuare eventuali alterazioni.
- Se viene configurato male, questa è considerata una falla di sicurezza

gli attacchi più frequenti ai *packet filtering* firewall sono :

- *IP address spoofing* : un intruso trasmette un pacchetto dall'esterno dichiarando nel campo del pacchetto di utilizzare l'indirizzo IP di un host della rete interna . In questo modo spera di penetrare un sistema che accetta tutti i pacchetti spediti da un host interno.
 - Una *contromisura* è quella di scartare i pacchetti che hanno un indirizzo interno, che però arrivano dall'esterno
- *source routing attack* : specifica il percorso che un pacchetto può intraprendere attraverso internet sperando di bypassare la sicurezza che non analizza l'origine del percorso
 - una *contromisura* è quella di bloccare tutti i pacchetti che usano questa opzione
- *tiny fragment attack* : un intruso può usare la frammentazione per creare piccolissimi frammenti che contengono una piccola parte di informazioni sull'header TCP. La speranza è che quella che venga analizzato solo il primo frammento , e insieme ad esso, fatti passare tutti gli altri senza esaminarli
 - una *contromisura* è quella di forzare il primo frammento e accettarlo solo se contiene informazioni sufficienti sull'header TPC

un *packet filtering firewall* tradizionale applica i filtri basandosi sui singoli pacchetti e non prende in considerazione contesti di livello superiore.

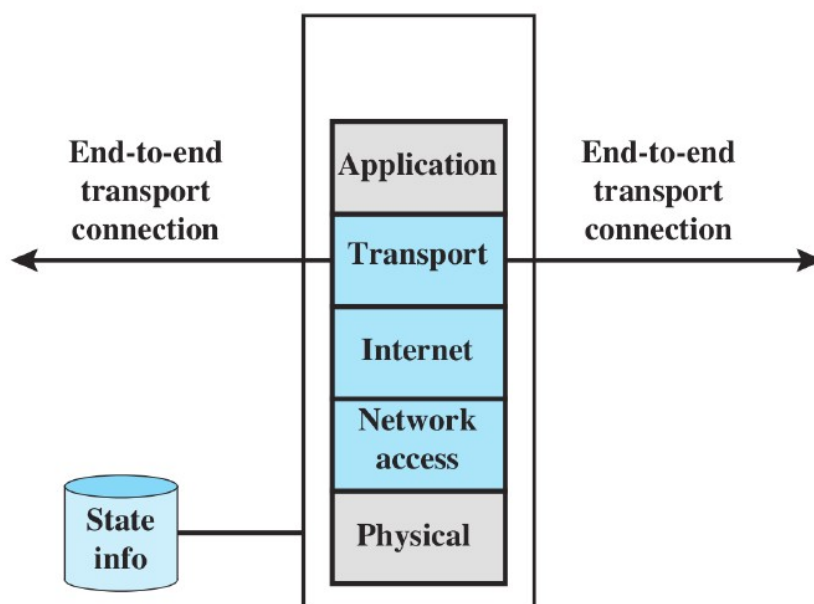
Quando un'applicazione che usa TCP crea una sessione con un host remoto, stabilisce una connessione TCP dove la porta TCP della applicazione remota (server) ha un numero <1024 mentre la porta dell'applicazione locale (client) è compresa tra 1024 e 65535. questi numeri sono generati dinamicamente e hanno un significato temporaneo solo per la durata della connessione.

A causa della generazione dinamica, un firewall di tipo *packet filtering* è obbligato a permettere tutto il traffico in entrata per tutti i numeri possibili e questo crea delle vulnerabilità sfruttabili da un attaccante.

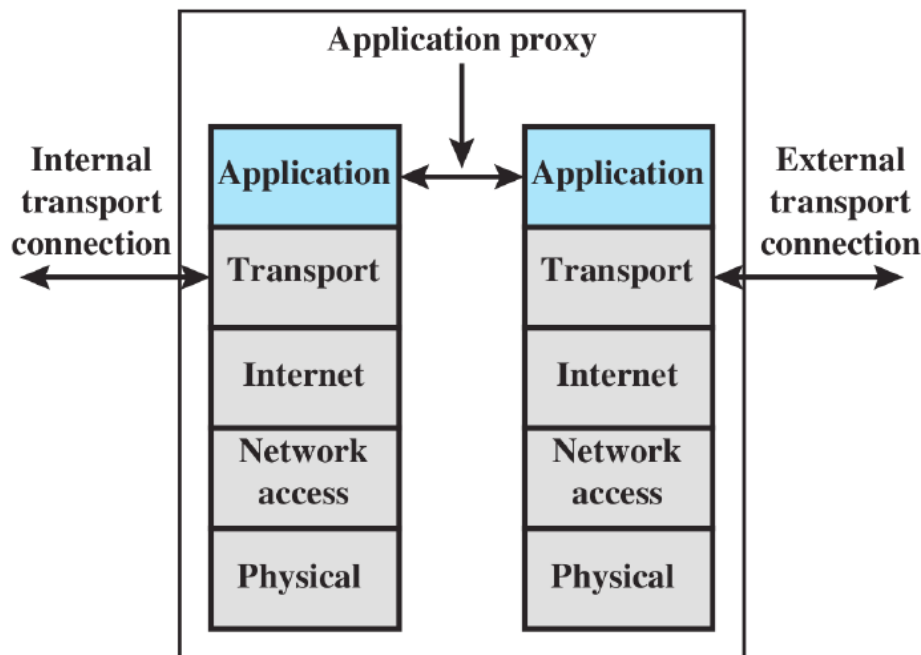
Lo *stateful packet inspection firewall* stringe le regole sul traffico TCP creando una cartella dove memorizza tutte le connessioni TCP in uscita. Ogni riga rappresenta una connessione attualmente stabilita. In questo modo il firewall accetterà il traffico in entrata con un numero di porta elevato solo per quei pacchetti che rispecchiano il profilo di una delle entità all'interno della cartella (ossia fanno parte di una delle connessioni stabilite).

Source Address	Source Port	Destination Address	Destination Port	Connection State
192.168.1.100	1030	210.9.88.29	80	Established
192.168.1.102	1031	216.32.42.123	80	Established
192.168.1.101	1033	173.66.32.122	25	Established
192.168.1.106	1035	177.231.32.12	79	Established
223.43.21.231	1990	192.168.1.6	80	Established
219.22.123.32	2112	192.168.1.6	80	Established
210.99.212.18	3321	192.168.1.6	80	Established
24.102.32.23	1025	192.168.1.6	80	Established
223.21.22.12	1046	192.168.1.6	80	Established

Oltre a raccogliere le informazioni dei *packet filtering*, gli *stateful packet* memorizzano anche le informazioni riguardo la connessione TCP e mantengono traccia della sequenza numerica TCP per prevenire attacchi di tipo *session hijacking*



un *application-level gateway* è un *tipo di firewall* chiamato anche *application proxy* che opera a *livello applicazione* sul *traffico dei pacchetti*.



Quando l'utente *che ha contattato* il proxy fornisce un *UserID valido* e informazioni per l'autenticazione, il proxy (*application-level gateway*) *contatta l'applicazione sull'host remoto* e trasmette i *segmenti TCP* che contengono i *dati dell'applicazione* tra i due *estremi* della comunicazione (ossia tra *utente* e *server host remoto*).

- Il *gateway* può essere configurato in modo da *supportare* solo le applicazioni *che presentano specifiche caratteristiche* che le rendono *accettabili* mentre tutte le altre *vedranno negato* il passaggio
- se il *gateway non implementa* il *codice proxy* di una applicazione, tale servizio *non è supportato* e quindi *non potrà attraversare* il firewall

l'*application-level gateway* tende ad essere *più sicuro* di un *packet filters firewall*

- Invece di *avere a che fare con numerose combinazioni* a livello TCP o IP , l'*application level gateway* necessita di *scrutinare solo poche applicazioni*. Inoltre i *log* e l'*auditing* del traffico *in entrata* a *livello applicativo* è molto più semplice

lo *svantaggio principale* è dato dall'*overhead* che si aggiunge *per ogni connessione*

- *di fatto una connessione* *viene spezzata in due* : dall'*utente al gateway* e dal *gateway all'host remoto*. Il *gateway* per entrambe queste *sottoconnessioni* deve *esaminare e far passare* il traffico *sia in entrata che uscita*

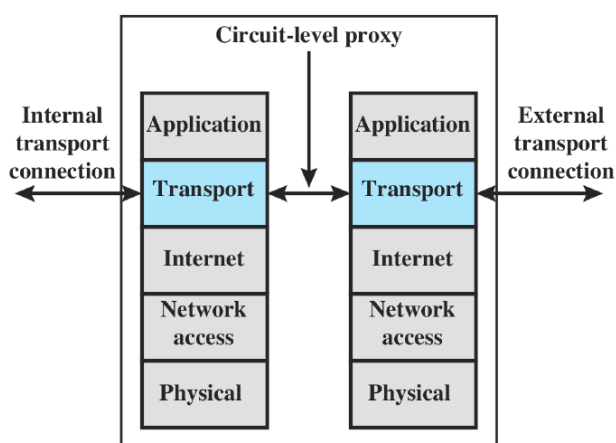
un altro tipo di *firewall* è il *circuit-level gateway*, il quale come nel caso dell'*application-level* crea per ogni connessione, due connessioni TCP :

- tra l'utente TCP e il *circuit-level gateway* su un *host interno*
- tra il *circuit-level gateway* e l'utente TCP su un *host esterno*

a differenza dell'*application-level*, il *circuit-level* esamina i pacchetti a livello di trasporto quindi una volta stabilite le connessioni, il gateway trasmette i segmenti TCP da una connessione all'altra però senza poterne esaminare i contenuti.

- A livello di trasporto è possibile solo conoscere il contenuto dell'*header* del segmento quindi indirizzo/porta di origine/destinazione

la funzione di sicurezza consiste nel determinare se permettere una connessione oppure no.



Tipicamente il *firewall* di tipo *circuit-level gateway* viene adottato in situazioni in cui si fida degli utenti interni (l'utente interno è *trusted*). Di solito il gateway viene configurato in questo modo :

- per la *connessione interna* si utilizza un'*application-level proxy*
- per la *connessione esterna* si utilizza il *circuit-level* il quale ha un *overhead* minore

lo standard *de facto* che implementa il *circuit-level gateway* è il SOCKS : uno *shim-layer* tra il livello applicativo e il livello di trasporto e come tale non fornisce servizi di rete.

Il protocollo SOCKS è composto da :

- il server SOCKS, solitamente eseguito su un *firewall* basato su UNIX
- una libreria client SOCKS, che esegue su un *host interno* protetto da *firewall*
- una versione SOCKS di un programma client standard (es FTP).

L'implementazione di un protocollo SOCKS di solito coinvolge o la *re-compilazione* di una applicazione client TCP oppure l'uso di librerie caricate dinamicamente

quando un *client TCP* desidera stabilire una connessione con un oggetto raggiungibile solo tramite *firewall* deve :

- aprire una *connessione TCP* su una porta appropriata (1080) sul SOCKS server system
- se la richiesta ha successo, deve autenticarsi
- il SOCKS server a questo punto valuta la richiesta e decide se stabilire la connessione

un *firewall* può essere implementato :

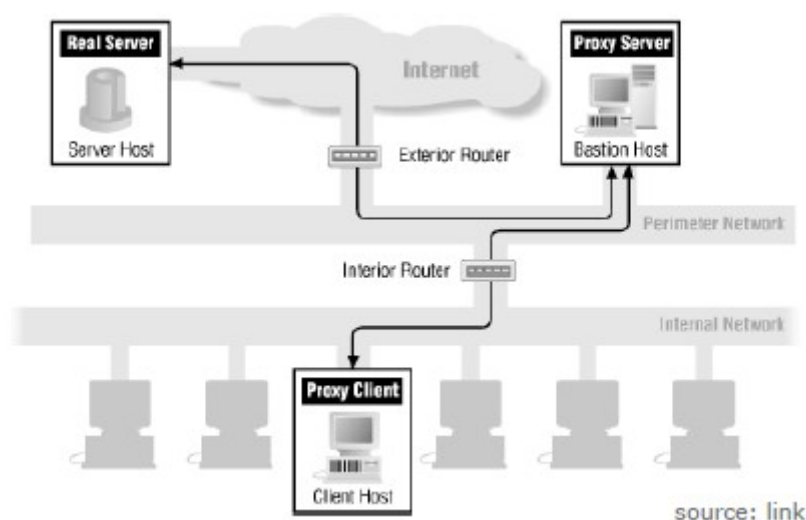
- livello *hardware* : una *macchina* con un *proprio sistema operativo*
- livello *software* : un *modulo* che viene implementato ad esempio sul *router periferico*

il *bastion host* è un sistema identificato dall'*amministratore del firewall* come un *punto critico* della *sicurezza di rete*. Il *bastion host* funge da *piattaforma* per un' *application* o *circuit level gateway* e ha le seguenti caratteristiche :

- il suo *sistema operativo* deve essere *sicuro* rendendo più *duro* il sistema (*hardening*)
- su di esso devono essere installati *solo i servizi essenziali*
 - ad esempio applicazioni *proxy* per il DNS,FTP, HTTP, e SMTP
- può richiedere *ulteriore autenticazione* prima di *concedere l'accesso* ad un servizio *proxy* (o ad un *host*), il quale *può avere un proprio* meccanismo di autenticazione *che garantisce l'accesso* ad un utente
- un *proxy* generalmente *non può accedere al disco* se non per leggere il *file di configurazione iniziale*. Il *codice eseguibile* rimane *accessibile in lettura* in modo da evitare possibili *installazioni* di trojan horse, sniffers o altro sul *bastion host*

ogni *proxy* deve :

- essere *configurato* per *supportare un sottoinsieme* delle comandi delle standard application
- essere *configurato* per *consentire l'accesso* solo a uno *specifico sistema host*
- *mantere informazioni dettagliate* sui *log* le *connessioni* e la loro *durata* per determinare *intrusioni*
- essere un *modulo piccolo e progettato apposta* per la *sicurezza di rete*
- essere *indipendente* dagli altri presenti sul *bastion host*
- operare *senza privilegi* e in *directory sicure e private*



un *host-based firewall* è un *modulo software* usato per *proteggere un singolo host*, in grado di *filtrare e restringere il flusso di pacchetti in transito* su quell'*host* :

- le *regole di filtraggio* possono essere *fatte su misura* per *l'ambiente dell'host* (server o workstation). Possono essere *applicate regole diverse* per *applicazioni diverse su server diversi*
- la *protezione* è *indipendente dalla topologia*, quindi gli *attacchi interni ed esterni* devono *per forza passare dal firewall*
- se *combinato con un firewall stand-alone* , il *host-based firewall* offre *uno strato protettivo aggiuntivo*

un *personal-firewall* è *modulo software* molto *meno complesso* dei *firewall server-based* o *stand-alone* e di solito viene installato su un *personal computer* dove il suo compito è quello di *controllare il traffico* tra *internet* e il *computer stesso*.

- Il *ruolo primario* è quello di *vietare gli accessi remoti non autorizzati* al sistema, e quello di *monitorare le attività in uscita* con l'obiettivo di *individuare e bloccare worms e malware*

usalmente, *abilitando il personal-firewall*, tutte le *connessioni in entrata* che non *ottengono un permesso esplicito dall'utente* vengono *bloccate*.

Per *aumentare la protezione* , possono essere *configurate caratteristiche avanzate* . I *pacchetti UDP* possono *venir bloccati* e il *traffico di rete* dei *pacchetti TCP* può essere *ristretto* solo per le *porte aperte*. I *firewall* possono *supportare il logging*, ossia un *tool* che gli consente di *controllare le attività illegali*, mentre un *personal firewall* può *concedere all'utente di utilizzare solo applicazioni prestabilite* oppure *firmate da una certification authority*

il *firewall* viene *posizionato* in modo da fornire una *barriera continua* tra il *traffico dei sorgenti esterni* (potenzialmente *inaffidabili*) e la *rete interna*

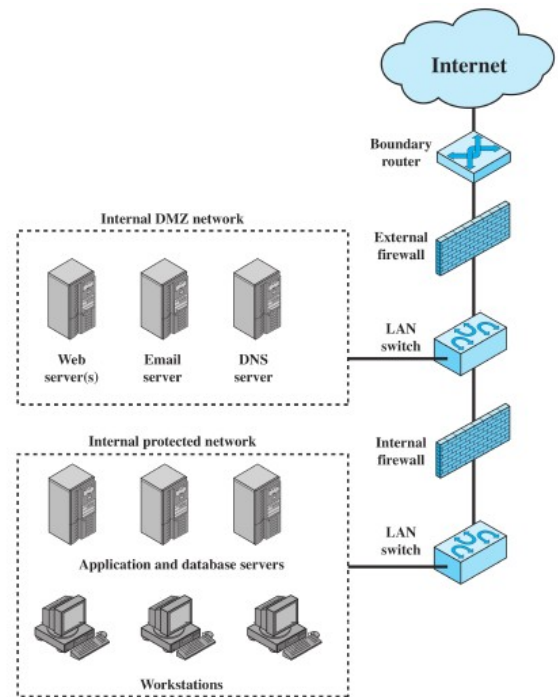
- l'*amministratore di sicurezza* ha il compito di *stabilire quanti firewall adottare e dove posizionarli*

una *configurazione comune* include un *ulteriore segmento di rete* tra il *firewall interno* e il *firewall esterno*.

- Firewall *esterno* : è posto *al confine* delle *rete interna*, appena *prima* del *router* che permette di *connettersi* ad Internet (*rete esterna*)
- Firewall *interno* : sono *più di uno* e *proteggono* la maggior parte della *rete interna*

questa *regione* che si trova *tra i due tipi* di firewall e che contiene *alcuni dispositivi di rete* è chiamata DMZ (*delimitarized zone*)

- fanno parte della *DMZ* quei sistemi che *devono essere accessibili dall'esterno*, ma che *necessitano* di qualche *protezione*
- questi sistemi *richiedono* la *possibilità di connettersi con l'esterno* (es *al sito aziendale, e-mail o DNS server*)



il *firewall interno* serve a *tre scopi* :

- aggiunge *filtri più stretti* rispetto al *filtro esterno*, in modo da *proteggere* i *server aziendali* e le *workstation* da *attacchi esterni*
- fornisce *protezione doppia* per la DMZ
 - protegge il *resto della rete interna* da *eventuali attacchi lanciati dalla DMZ*
 - protegge la DMZa *eventuali attacchi lanciati dalla rete interna*
- più *firewall interni* possono essere utilizzati per *proteggere porzioni* di *rete interna* l'una *dalle altre*

un *punto critico* di questo modello è *rappresentato dalle workstation* :

- non essendo *isolate*, in questo caso dai *database server*, se un *attaccante riesce a prendere il controllo* di una *workstation* è *in grado di accedere direttamente* ai *database* senza dover *superare nessun'altro controllo*

un'altra soluzione, specialmente in un ambiente distribuito come quello odierno, è rappresentata dalle virtual private network (VPN).

- Una VPN consiste in un insieme di computer interconnessi da una rete relativamente insicura che utilizza la crittografia e protocolli speciali per offrire sicurezza.
 - Come rete può essere usata internet (o qualunque rete pubblica) in modo da abbassare i costi derivanti dall'uso di una rete privata e delegando la gestione della rete a un network provider pubblico.

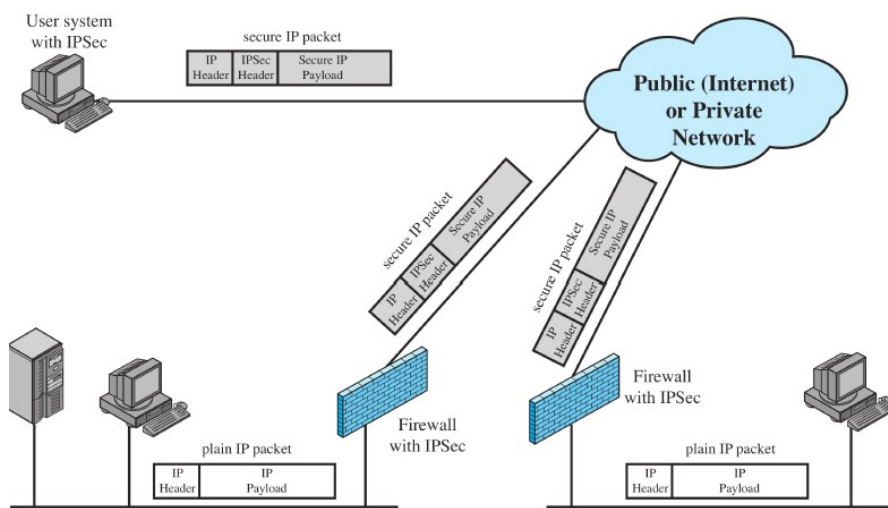
L'utilizzo di una rete pubblica espone il traffico della VPN a intercettazioni e fornisce un punto di ingresso per autenti non autorizzati.

- La VPN ricorre quindi alla crittografia e autenticazione per i protocolli più bassi in modo da fornire una connessione sicura attraverso una rete insicura
- la crittografia viene eseguita dal firewall o dal router

il meccanismo più utilizzato è il protocollo Ipsec :

Scenario : un organizzazione mantiene della local area network sparse in diverse località. All'interno di ogni LAN viene utilizzato un traffico IP non sicuro. Per il traffico esterno alle LAN , che viaggia attraverso una WAN pubblica si utilizza il protocollo Ipsec

- i dispositivi Ipsec tipicamente criptano e comprimono tutto il traffico destinato alla WAN (in uscita) e decriptano e decomprimono tutto quello in entrata sempre dalla WAN (possono richiedere anche autenticazione)
- ogni utente, la cui workstation implementa Ipsec sarà in grado di comunicare in modo sicuro attraverso la WAN
 - ad alto livello però questi host sono connessi alla rete esterna, e questo li rende ottime prede per chi volesse ottenere accesso alla rete interna



ipsec dovrebbe essere implementato in un firewall (i router sono meno sicuri)

- se fosse implementato ad esempio alle spalle di un firewall interno, tutto il contenuto sarebbe criptato e quindi impossibile da esaminare per il firewall stesso ed applicarci i vari filtri

una *configurazione distribuita* prevede l'utilizzo combinato di *firewall stand-alone* e *firewall host-based* sotto lo stesso ed unico controllo amministrativo (la cui gestione e configurazione risulta comunque essere molto complessa).

- L'amministratore può configurare dei *firewall host-based* su centinaia di server o workstation così come configurare un *personal firewall* su ogni sistema utente che sia *locale* o *remoto*
- questi *firewall* proteggono da *attacchi interni* e forniscono *protezioni su misura* per *specifiche* macchine e applicazioni

è possibile creare delle *DMZ interne* ed *esterne* :

- *DMZ interna* : si trova prima del *firewall esterno* e ospita ad esempio *e-mail* e i *DNS servers* i quali hanno bisogno di *aver accesso all'esterno* e che comunque *devono essere protetti*
- *DMZ esterna* : si trova *oltre il firewall esterno* e ospita ad esempio i *web server* che necessitano di *protezione minore* in quanto *non contengono informazioni critiche*

un *aspetto fondamentale* è l'attività di *monitoraggio* : è necessario *aggregare i log* , le *analisi* , le *statistiche* relative ai *firewall* e gli *alerting* che *provengono da tutti i singoli host*

